

프로젝트 보고서-중간

날짜: 2026-05-03, 학번: 243135, 이름: 윤지연

1. 프로젝트 개요

- 프로젝트 이름 : "SecPipeline : Flask 기반 학습 기록 애플리케이션의 DevSecOps 파이프라인 구축 프로젝트"
- 프로젝트 목적 : 본 프로젝트는 단순한 웹 애플리케이션 구현을 넘어서, 개발(Dev)-> 보안(Sec) -> 운영(Ops)의 전 과정을 하나의 자동화된 파이프라인으로 직접 설계, 구축하는 것을 목표로 한다. 특히 단순히 도구를 나열하는 것이 아니라, Schuckert et al.(2023)의 "Insecurity Refacotring" 기법을 응용하여 5가지 보안 취약점(VULN 01~05)을 의도적으로 주입시켜 Red Run하게 한 뒤, 각 보안 게이트가 이를 정확히 탐지, 차단하는지 확인하고, 이후 시큐어 코딩으로 리팩토링하여 모든 게이트를 통과시키는(Green Run) 검증 사이클을 핵심 방법론으로 삼았다.
- GitHub 주소 : <https://github.com/de1ayyy/SecPipeline>

2. 소프트웨어 주요 기능

- 본 프로젝트의 애플리케이션은 Flask 기반 학습 로그 관리 웹 서비스로, 파이프라인 검증을 위해 구성하였다.
- 회원가입/로그인

SecPipeline

로그인 회원가입

회원가입

사용자명

사용할 아이디

비밀번호

비밀번호

비밀번호 강도: 입력해주세요

가입완료

이미 계정이 있으신가요? [로그인](#)

- 학습 기록 CRUD : 사용자별 학습 시간, 과목, 내용을 생성, 조회, 수정, 삭제

SecPipeline

jiyeon0623님 기록 과목 검색 대시보드 로그아웃

📖 내 학습 기록

+ 새 기록 작성

필터: 모든 과목 ▾

정렬: 최신순 ▾

소프트웨어개발론 7주차

오늘 소프트웨어개발론에서 폭포수 모델에 대해

폭포수 모델은..

🕒 1.0시간 | 2026-05-02 17:33:52

수정 삭제

오늘 render로 배포해봤어요!

render로 배포해봤더니 편리하네요. 파이프라인

🕒 2.0시간 | 2026-05-02 17:33:05

수정 삭제

- 과목 관리 : 학습 카테고리 등록 및 분류

SecPipeline

jiyeon0623님 기록 과목 검색 대시보드 로그아웃

📁 과목 관리

새로운 과목 이름을 입력하세요

추가하기

● 소프트웨어개발론

삭제

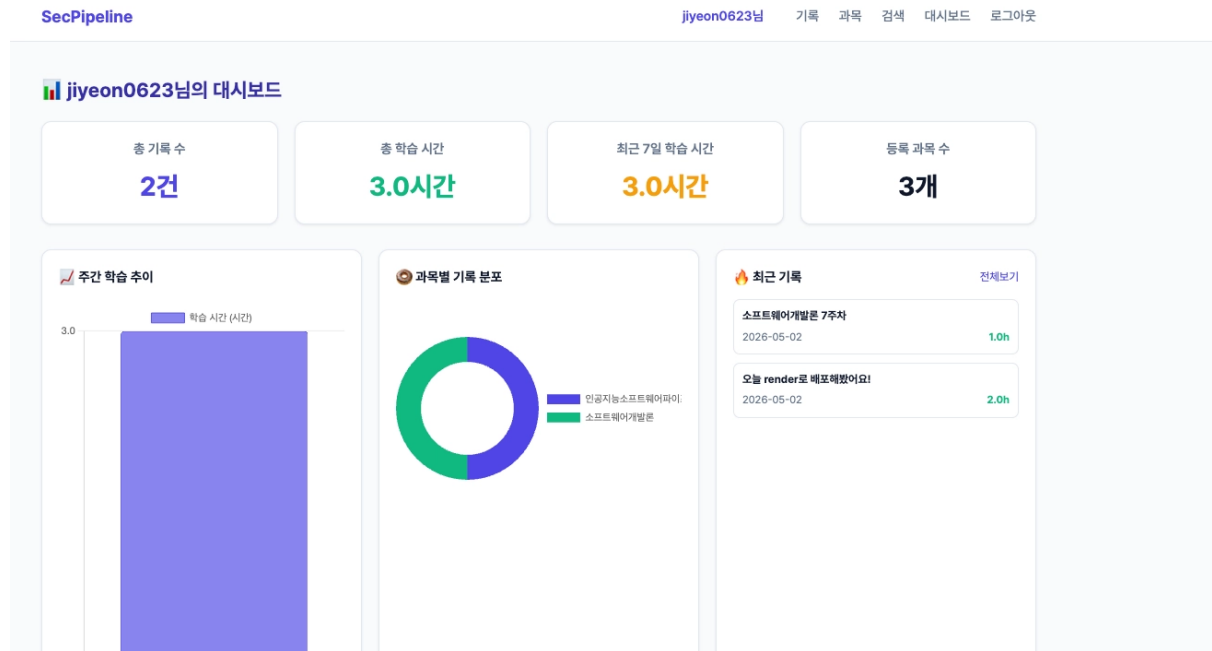
● 알고리즘

삭제

● 인공지능소프트웨어파이프라인개발

삭제

- 대시보드 : 누적 학습 시간 및 과목별 분포 등 통계 시각화



- 검색 기능 : 키워드 기반 학습 기록 검색

- CSV 내보내기 : 자신만의 학습 데이터를 외부 파일로 추출하기 (주요 기능만 뽑아서 웹 사진 첨부하자면, 이 정도임!)

3. 실행 환경

- 사용 OS 및 버전: MacOS
- Git 버전: git 2.46.2
- Python 버전 : 3.12
- Docker : Docker Desktop, 베이스 이미지 : python:3.12-slim
- 배포 플랫폼 : Render
- GitHub 계정: <https://github.com/de1ayyy/SecPipeline>

4. 전체 DevOps 파이프라인 구조

- 전체 파이프라인은 Code Commit -> CI(보안 게이트 + 단위 테스트) -> Build(Docker) -> Deploy(Render) -> Monitor(DAST)의 총 5단계로 구성된다. 특히 보안 게이트 안에 5Gate가 또 있기에 각 단계마다 명확한 보안 책임을 부여한 것이 큰 특징이다!

1. 개발자가 "git push origin main" 명령어를 입력하면, (Trigger가 push / pull_request)

2. CI-Gate 1(Secret Scan)으로, Gitleaks가 하드코딩된 시크릿을 탐지한다. 이때 시크릿키가 그냥 적혀있는 경우 탐지되며, 보안 관련 이슈가 있을 경우 배포되지 않는다.
3. CI-Gate 2(SCA)으로 pip-audit가 의존성 CVE를 점검한다.
4. CI-Gate 3(SAST)으로 Semgrep과 Bandit으로 정적 코드를 분석한다.
5. 이때 Unit Test로 6가지 케이스에 대해 pytest가 진행된다.
6. CI-Gate 5(Dockerfile Lint)로 hadolint를 통해 분석한다.
7. CI-Gate 4(Container Scan)로 Trivy 이미지 취약점이 있는지 분석한다.
8. 2번부터 7번까지 하나라도 실패한다면 BUILD/DEPLOY가 차단된다.(Red Run) 만약 모든 게이트를 통과했다면 Render의 Deploy Hook을 호출해 배포한다.
9. 마지막으로 DAST로 OWASP ZAP Baseline Scan을 통해 보안 결함이 Runtime 환경에서 있는지 확인한다.

단계	활동	본 프로젝트 도구
Code Commit	시크릿/이력 검사	Gitleaks
Build	의존성/빌드 보안	pip-audit, pytest
Test	코드/이미지 분석	Semgrep, Bandit, Trivy, Hadolint
Deploy	자동 배포	Render Deploy Hook
Monitor	런타임 보안	OWASP ZAP (DAST)

5. Git 기반 개발 과정

5.1 개발 흐름

1. GitHub CLI(gh repo create)로 public 저장소 생성
2. 로컬에 python -m venv .venv로 가상환경 구성
3. Flask Application Factory 패턴 + Blueprints 구조로 모듈 분리
4. 의도적 취약점(VULN-01~05) 주입 → Red Run으로 보안 게이트 동작 검증
5. 탐지 결과 분석 후 시큐어 코딩 리팩토링 → Green Run으로 통과 확인
6. 모든 게이트 통과 후 Render 자동 배포 + OWASP ZAP DAST 검증

5.2 브랜치 전략

- 개인 프로젝트 특성상 main 단일 브랜치 중심의 GitHub Flow 변형
- push / pull_request 두 이벤트 모두 파이프라인 트리거
- 큰 단위 변경(예: VULN 주입, 게이트 추가) 시 의미 단위로 별도 커밋

5.3 커밋 메시지 규칙

- 작업 내용을 한국어로 명확히 요약
- 날짜+의미 단위 커밋 (예: "프로젝트 정의 + 환경 세팅 + 파일 구조 세팅 완료")
- VULN/Gate 관련 변경은 "[VULN-01] config.py SECRET_KEY 하드코딩 추가"처럼 태그 형태로 구분
- Red Run / Green Run 비교를 위해 의도적 취약 버전과 패치 버전을 분리 커밋
- 커밋 전략: 커밋 기준과 메시지 규칙

6. CI (Continuous Integration)

6.1 CI 구성

- 도구 : GitHub Actions
- 워크플로우 파일 : .github/workflows/devsecops.yml
- 실행 조건

```
name: SecPipeline DevSecOps
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

6.2 수행 작업

1. unit-test: pytest로 6개 단위 테스트 (인증 흐름, 헬스체크, 검색, CRUD)
2. secret-scan (Gate 1): Gitleaks (fetch-depth: 0으로 전체 이력 검사)
3. dependency-scan (Gate 2): pip-audit (requirements.txt 기반 CVE 탐지)

4. sast (Gate 3): Semgrep(--config auto + 커스텀) + Bandit (병렬 실행)

5. container-scan (Gate 4): Trivy (Docker 이미지 OS/패키지 스캔)

6. dockerfile-lint (Gate 5): Hadolint (Dockerfile 모범 사례)

7. deploy: Render Deploy Hook 호출 (CD)

8. dast: OWASP ZAP Baseline Scan

- Workflow 설명: 주요 단계 및 yaml 일부를 첨부해 설명

```

name: SecPipeline DevSecOps

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # — Gate 1: Secret Scan (Gitleaks) —————
  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

  # — Gate 2: Dependency Scan (pip-audit) —————
  dependency-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: |
          pip install pip-audit
          pip-audit -r requirements.txt --format json --output dep-scan.json
    n || true
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: dependency-scan
          path: dep-scan.json

  # — Gate 3: SAST (Semgrep + Bandit) —————
  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: |
          pip install semgrep bandit
          semgrep scan --config auto --config .semgrep/ app/ --json --output
t semgrep.json || true
          bandit -r app/ -f json -o bandit.json || true
      - uses: actions/upload-artifact@v4
        if: always()

```

```

# — Gate 3: SAST (Semgrep + Bandit) —————
sast:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: |
        pip install semgrep bandit
        semgrep scan --config auto --config .semgrep/ app/ --json --output
t semgrep.json || true
        bandit -r app/ -f json -o bandit.json || true
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: sast-results
        path: |
          semgrep.json
          bandit.json

# — Unit Test (pytest) —————
unit-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: |
        pip install -r requirements.txt
        python -m pytest tests/ -v --junitxml=test-results.xml
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: unit-test-results
        path: test-results.xml

# — Gate 4: Build + Container Scan (Trivy) —————
build-and-scan:
  runs-on: ubuntu-latest
  needs: [secret-scan, dependency-scan, sast, unit-test]
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t studylog:${{ github.sha }} .
    - name: Trivy image scan
      uses: aquasecurity/trivy-action@v0.35.0
      with:
        image-ref: studylog:${{ github.sha }}
        format: json
        output: trivy.json
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: trivy-results
        path: trivy.json

```

```
# — Gate 5: Dockerfile Lint (hadolint) —————
dockerfile-lint:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Lint Dockerfile
      uses: hadolint/hadolint-action@v3.3.0
      with:
        dockerfile: Dockerfile

# — Deploy: Render Webhook 배포 —————
deploy:
  runs-on: ubuntu-latest
  needs: [unit-test, build-and-scan, dockerfile-lint]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  steps:
    - name: Trigger Render Deploy
      run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"

# — Gate 7: DAST (OWASP ZAP) - 운영 환경 동적 스캔 —
dast:
  runs-on: ubuntu-latest
  needs: deploy
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  steps:
    - name: Checkout
      uses: actions/checkout@v4
    - name: OWASP ZAP Baseline Scan
      uses: zaproxy/action-baseline@v0.14.0
      with:
        target: 'https://secpipeline.onrender.com'
        fail_action: false
```

- 실행 결과

6.5.1 Red Run 결과 (취약점 주입 상태)

Job	상태	탐지/오류
unit-test	✅ pass	6/6 통과
secret-scan (Gitleaks)	❌ fail	1건 (SECRET_KEY 하드코딩)
dependency-scan (pip-audit)	❌ fail	16건 CVE
sast (Semgrep)	❌ fail	16건 (ERROR 3건)
sast (Bandit)	❌ fail	B105, B608 다수
container-scan (Trivy)	❌ fail	HIGH 다수 (EOL python:3.8-slim)
dockerfile-lint (Hadolint)	❌ fail	DL3002, DL3025, DL3045
deploy	⏭ skipped	이전 게이트 실패로 차단

pip-audit 출력 일부 (JSON 발췌):

```
{
  "dependencies": [
    {
      "name": "flask",
      "version": "2.2.5",
      "vulns": [
        {"id": "CVE-2023-30861", "fix_versions": ["2.3.2"]}
      ]
    },
    {
      "name": "werkzeug",
      "version": "2.2.3",
      "vulns": [
        {"id": "CVE-2024-34069", "description": "Debugger PIN bypass"},
        {"id": "GHSA-2g68-c3qc-8985"}
      ]
    }
  ],
  "summary": {"total_vulns": 16}
}
```

Bandit 출력 일부:

Issue: [B105:hardcoded_password_string] Possible hardcoded password
Severity: Low Confidence: Medium
Location: ./app/config.py:8

Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction
Severity: Medium Confidence: Low
Location: ./app/routes/search.py:24

6.5.2. Green Run 결과 (리팩토링 후)

Job	상태	결과
unit-test	✅ pass	6/6 통과, 커버리지 정상
secret-scan	✅ pass	0건
dependency-scan	✅ pass	0 CVE
sast (Semgrep + Bandit)	✅ pass	0건
container-scan (Trivy)	✅ pass	HIGH/CRITICAL 0건
dockerfile-lint	✅ pass	0건
deploy	✅ success	Render Webhook 호출 성공
dast (OWASP ZAP)	✅ pass	High 0, Medium 0

OWASP ZAP Baseline Scan 출력 발췌:

PASS: Cross Domain JavaScript Source File Inclusion
 PASS: X-Frame-Options Header Set
 PASS: X-Content-Type-Options Header
 PASS: Cookie Secure Flag
 WARN-NEW: Informational - 0
 WARN-NEW: Low - 0
 FAIL-NEW: Medium - 0
 FAIL-NEW: High - 0

Red Run은 5개 게이트 모두에서 의도된 취약점을 정상 탐지하여 배포 단계로 진행되지 못하였고, 이후 시큐어 코딩 리팩토링을 거친 Green Run에서는 모든 게이트를 통과한 뒤 Render 배포 및 OWASP ZAP DAST까지 통과하였다. 이로써 본 파이프라인이 ① 취약점이 있을 때 정확히 차단하고, ② 안전한 코드는 자동 배포까지 끊김 없이 흐른다는 양방향 동작이 모두 검증되었다.

7. Docker 기반 환경 구성

```
# Fix-04: 최신 보안 패치가 적용된 Python 3.12 사용
FROM python:3.12-slim

# Fix-05: 작업 디렉토리(WORKDIR) 명시
WORKDIR /app

# Fix-05: Non-root 권한 분리용 사용자 생성 및 curl 설치 (Healthcheck 용도)
# hadolint ignore=DL3008
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/* \
    && adduser --disabled-password --gecos "" appuser

COPY requirements.txt requirements.txt
# Fix-05: 패키지 다운로드 캐시 제거(--no-cache-dir)
RUN pip install --no-cache-dir -r requirements.txt

# Fix-05: 코드 복사 시 소유권(chown) 지정
COPY --chown=appuser:appuser . .

# Fix-05: 애플리케이션 실행 권한 전환 (CWE-250 방어)
USER appuser

EXPOSE 5000

# Fix-05: 컨테이너 내에서 DB 파일을 누구나 쓰기 가능한 /tmp 경로에 생성하도록 환경변수 설정
ENV DATABASE_PATH=/tmp/studylog.db

# Fix-05: 컨테이너 상태 모니터링을 위한 HEALTHCHECK
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1

# Fix-05: exec form 배열 사용 및 운영용 서버 gunicorn으로 실행
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "run:app"]
```

특히 보안 취약점 (VULN-04, 05)을 방어하기 위해 유의하며 Dockerfile을 작성하였습니다.

- 최소 권한의 원칙대로 Run adduser ... appuser 명령으로 전용 시스템 사용자를 생성하고, USER appuser로 컨테이너가 root가 아닌 제한된 권한으로 실행되도록 설정하였습니다.

- 실행 방법

- 1) 이미지 빌드 : 터미널에서 docker build -t secpipeline:latest .

- 2) 컨테이너 실행 : docker run -d -p 5000:5000 --name secpipeline_server secpipeline:latest

8. 배포 및 실행

- 배포 방법 : Render로 GitHub Actions의 마지막 단계에서 Render Deploy Hook URL로 "curl -X POST" 호출하면 Render가 새 이미지를 pull 후 자동 배포한다. Secrets는 모두 환경 변수로 지정했다.

- 서비스 정보 : 서비스명은 StudyLog, 외부 접근 URL :

<https://secpipeline.onrender.com/login>, 헬스 체크도 가능하다. GET /health ->


```
{"status": "ok"}
```

- 실행 확인

8. 문제 해결 경험

8.1 Gitleaks의 False Negative

- app/config.py에 하드코딩한 SECRET_KEY = "super-secret-key-12345"를 Gitleaks가 탐지하지 못함
- 원인: 문자열 엔트로피가 낮고 hyphen 포함 등으로 "generic-api-key" 규칙 임계값 미달
- 해결: 고엔트로피 무작위 문자열로 교체(fl4sk_s3cr3t_8Kx9Qm2vPwLnR7jY5tZa) → True Positive 탐지 확인 → 최종적으로 환경변수 분리(os.environ.get("SECRET_KEY"))

8.2 의존성 16개 CVE (pip-audit)

- Flask 2.2.5, Jinja2 3.1.2, Werkzeug 2.2.3, Gunicorn 21.2.0 등에서 16건 CVE 탐지,, 의도적 구버전을 사용했음에도 발견안되는 경우가 있었음. (이미 해결된 경우에)
- 해결 : Flask 3.1.0+, Jinja2 3.1.4+, Werkzeug 3.1.0+, Gunicorn 22.0.0+로 업그레이드 → CVE 0건

8.3 Dockerfile 안티패턴

- USER 누락, EOL 베이스 이미지, shell form CMD 등 다수 경고
- python:3.12-slim 교체, appuser 비특권 사용자 추가, exec form CMD, HEALTHCHECK 추가

9. 느낀 점 및 개선 방향

- DevOps 관점에서 배운 점 : Shift-Left Security 가치를 직접 검증할 수 있었다. 기존에는 보안이 "마지막 검토 단계"로 인식되었지만, 이번 프로젝트에서 Gate 1~5를 통과하지 않은 코드는 빌드조차 되지 않는 환경을 직접 경험하니 개발의 보안에 대해서 미리 피해를 방지할 수 있는게 정말 좋다는 것을 직접 알게 되어 좋았다.
- 개선하고 싶은 부분 : Render 배포 후 Prometheus/Grafana 또는 Sentry 연동으로 런타임 모니터링, 알림 체계 추가로 구축해보기. 또한 ZAP Baseline Scan은 표면적 검사이므로, ZAP Full Scan 또는 인증 토큰 기반 Authenticated Scan을 도입하여 로그인 후 화면까지 점검할 필요가 있다.
- 수업 피드백 : 좋은 내용 많이 배우고 있어서, 정말 없습니다..!

참고 자료

- [1] Schuckert, F. et al. (2023). Insecurity Refactoring: Automated Generation of Vulnerable Code Samples.
- [2] Fu, M. et al. (2024). DevSecOps 5단계 프레임워크 관련 문헌
- [3] NVD — CVE-2023-30861: <https://nvd.nist.gov/vuln/detail/CVE-2023-30861>
- [4] 12-Factor App — Config 원칙: <https://12factor.net/config>
- [5] OWASP Top 10 (2025) — A07: Authentication Failures
- [6] CWE Top 25 (2023) — CWE-798 Hardcoded Credentials (#18)
- [7] Gitleaks 공식 문서: <https://github.com/gitleaks/gitleaks>
- [8] pip-audit: <https://pypi.org/project/pip-audit/>
- [9] Semgrep Rules: <https://semgrep.dev/r>
- [10] Bandit: <https://bandit.readthedocs.io/>
- [11] Trivy: <https://aquasecurity.github.io/trivy/>
- [12] Hadolint: <https://github.com/hadolint/hadolint>
- [13] OWASP ZAP: <https://www.zaproxy.org/>
- [14] Render Docs — Deploy Hooks: <https://render.com/docs/deploy-hooks>
- [15] Docker Desktop on macOS — 아키텍처 문서: <https://docs.docker.com/desktop/mac/>

제 프로젝트를 보고서에 담기는 내용 전달이 제대로 안되어, 제가 개발하면서 겪었던 것들, 분석했던 것들 정리한 내용을 추가로 첨부합니다! 파이프라인 고도화 과정입니다. 이 부분이 핵심이니 꼭 읽어주십시오! 파이프라인 고도화 과정이 담겨있습니다. (보고서만으로는 어떻게 디벨롭했는지 담기지 않아 추가로 첨부합니다.)

0324 개발

▼ 0. SecPipeline 프로젝트를 기획하게 된 이유

0-1. 왜 DevSecOps 파이프라인을 직접 구축하는가?

전통적인 소프트웨어 개발에서는 개발(Dev)이 끝난 후 별도의 보안 점검(Sec)을 거치고, 그 다음에 운영으로 넘겼음. 이 방식은 보안 문제를 개발 후반부에야 발견하게 되어 수정 비용이 높고, 배포 속도를 떨어뜨리게 됨.

DevSecOps는 이 문제를 해결하기 위해 보안을 개발 프로세스의 모든 단계에 통합하는 패러다임임. 코드를 작성하는 순간부터 빌드, 테스트, 배포까지 자동화된 보안 검사가 돌아가도록 했음.

0-2. 해당 프로젝트는 기존 방법론을 이해하면서 구축한다.

기존 DevSecOps 방법론을 직접 체험하고, 전체 파이프라인을 구현하면서 발생하는 버그/장애의 발생 → 해결 과정을 기록할 것임.

의도적 취약점 6가지

ID	취약점	위치	탐지 Gate	도구
VULN-01	하드코딩 시크릿	app/config.py	Gate 1	Gitleaks
VULN-02	취약 종속성 (CVE-2023-30861)	requirements.txt	Gate 2	pip-audit
VULN-03a	SQL Injection (f-string)	app/routes/search.py	Gate 3	Semgrep+Bandit
VULN-03b	XSS (render_template_string)	app/routes/dashboard.py	Gate 3	Semgrep+Bandit
VULN-04	오래된 베이스 이미지 + root 실행	Dockerfile	Gate 4	Trivy
VULN-05	Dockerfile 베스트프랙티스 위반	Dockerfile	Gate 5	hadolint

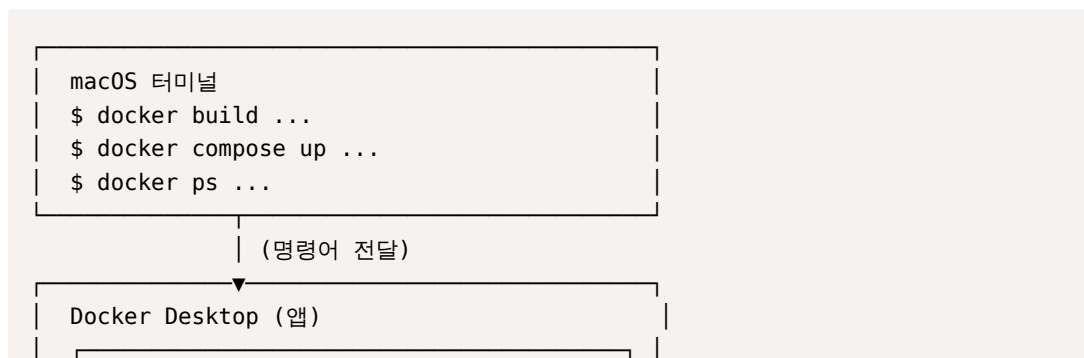
▼ 1. 환경 세팅

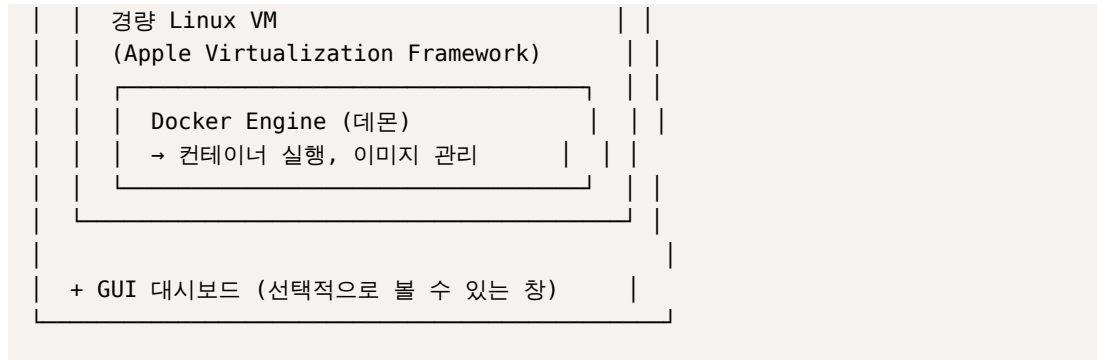
```
/Users/yoongi@yoongi-macbook-pro:~/Documents/0324$ python --version
Python 3.12.7
(base) yoongi@yoongi-macbook-pro:~/Documents/0324$ git --version
git version 2.46.2
(base) yoongi@yoongi-macbook-pro:~/Documents/0324$ code --version
zsh: command not found: code
(base) yoongi@yoongi-macbook-pro:~/Documents/0324$ docker --version
Docker version 28.3.3, build 980b856
(base) yoongi@yoongi-macbook-pro:~/Documents/0324$ gh --version
zsh: command not found: gh
(base) yoongi@yoongi-macbook-pro:~/Documents/0324$ docker run hello-world
docker: Cannot connect to the Docker daemon at unix:///Users/yoongi/.docker/run/docker.sock. Is the docker daemon running?
Run 'docker run --help' for more information
```

1.1 필수 소프트웨어 설치

- Python 3.12 +
- Git
- VS Code (CLI)
- Docker Desktop

▼ Docker Desktop 관련 조사 (Docker Desktop을 명령어로..?)





Docker Desktop = Linux VM + Docker Engine + GUI 대시보드

macOS는 Darwin 커널 기반이라 Linux 컨테이너를 직접 실행할 수 없습니다. Docker 데몬 자체가 macOS 위에서는 네이티브로 돌아갈 수 없다는 뜻입니다. 그래서 누군가가 중간에 **Linux VM**을 깔아주고, 그 안에서 **Docker 데몬**을 띄워주고, macOS 터미널과 연결해줘야 비로소 `docker` 명령어가 동작합니다.

그 "누군가"가 바로 **Docker Desktop**이거나 **Colima** 같은 도구입니다. 이것들이 하는 일은 결국 한 가지로, Linux VM을 만들고 → 그 안에 Docker Engine을 띄우고 → macOS 터미널의 `docker` 명령어가 그 Engine과 통신할 수 있도록 소켓(`/var/run/docker.sock`)을 연결해주는 것입니다.

그래서 흐름을 비교하면 이렇습니다.

리눅스에서는 `docker build .` 을 치면 그 명령어가 같은 OS 안에 있는 Docker 데몬에 **직접** 전달됩니다. 중간 단계가 없습니다.

macOS에서는 `docker build .` 을 치면 그 명령어가 Linux VM 안에 있는 Docker 데몬에 **소켓을 통해 우회** 전달됩니다. 한 단계가 더 있지만, 사용자 입장에서 명령어도 결과도 똑같기 때문에 그 차이를 체감할 일은 거의 없습니다.

결국 Docker Desktop이나 Colima를 설치하지 않은 순수 macOS 터미널에서 `docker` 를 치면 **"command not found"**이거나 **"데몬에 연결할 수 없다"**는 예러가 나옵니다. 연결해주는 도구를 설치해야 비로소 리눅스와 동일하게 쓸 수 있게 되는 것

- GitHub CLI
 - 확인해보니 없어서 `brew install gh` 로 설치했음.
 - gh에 로그인 성공!

```

yoonjiyeon ~ - zsh - 80x24
Removing: /Users/yoonjiyeon/Library/Caches/Homebrew/bootsnap/aa63b2bf9ba1db3c59%
8867e38a8c8156f3643776b6b443a3475a2f58fc4bb... (648 files, 5.7MB)
Removing: /Users/yoonjiyeon/Library/Logs/Homebrew/git-lfs... (648)
Removing: /Users/yoonjiyeon/Library/Logs/Homebrew/tree... (648)
Removing: /Users/yoonjiyeon/Library/Logs/Homebrew/helm... (648)
==>
zsh completions have been installed to:
  /opt/homebrew/share/zsh/site-functions
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro ~ % gh --version
gh version 2.88.1 (2026-03-12)
https://github.com/cli/cli/releases/tag/v2.88.1
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro ~ % gh auth login
? GitHub.com
? Yes
? Login with a web browser

First copy your one-time code:
  to open https://github.com/login/device in your browser...
  Authentication complete.
  gh config set -h github.com git_protocol https
  Configured git protocol
  Logged in as
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro ~ %
  
```

1-2. GitHub 저장소 생성 및 클론

```

(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro ~ % cd Desktop
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro Desktop % git clone https://github.com/de1ayyy/SecPipeline.git
'SecPipeline'에 복제합니다...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
오브젝트를 받는 중: 100% (4/4), 완료.
  
```

```

(base) yoonjiyeon@yunjiyeon-ui-MacBookPro Desktop % cd SecPipeline
(base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % pwd
/Users/yoonjiyeon/Desktop/SecPipeline
(base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % ls -la
total 24
drwxr-xr-x  5 yoonjiyeon  staff   160  3월  24 19:46 .
drwx-----@ 13 yoonjiyeon  staff   416  3월  24 19:46 ..
drwxr-xr-x 12 yoonjiyeon  staff   384  3월  24 19:46 .git
-rw-r--r--  1 yoonjiyeon  staff  4688  3월  24 19:46 .gitignore
-rw-r--r--  1 yoonjiyeon  staff    13  3월  24 19:46 README.md
(base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git remote -v
origin  https://github.com/delayyy/SecPipeline.git (fetch)
origin  https://github.com/delayyy/SecPipeline.git (push)

```

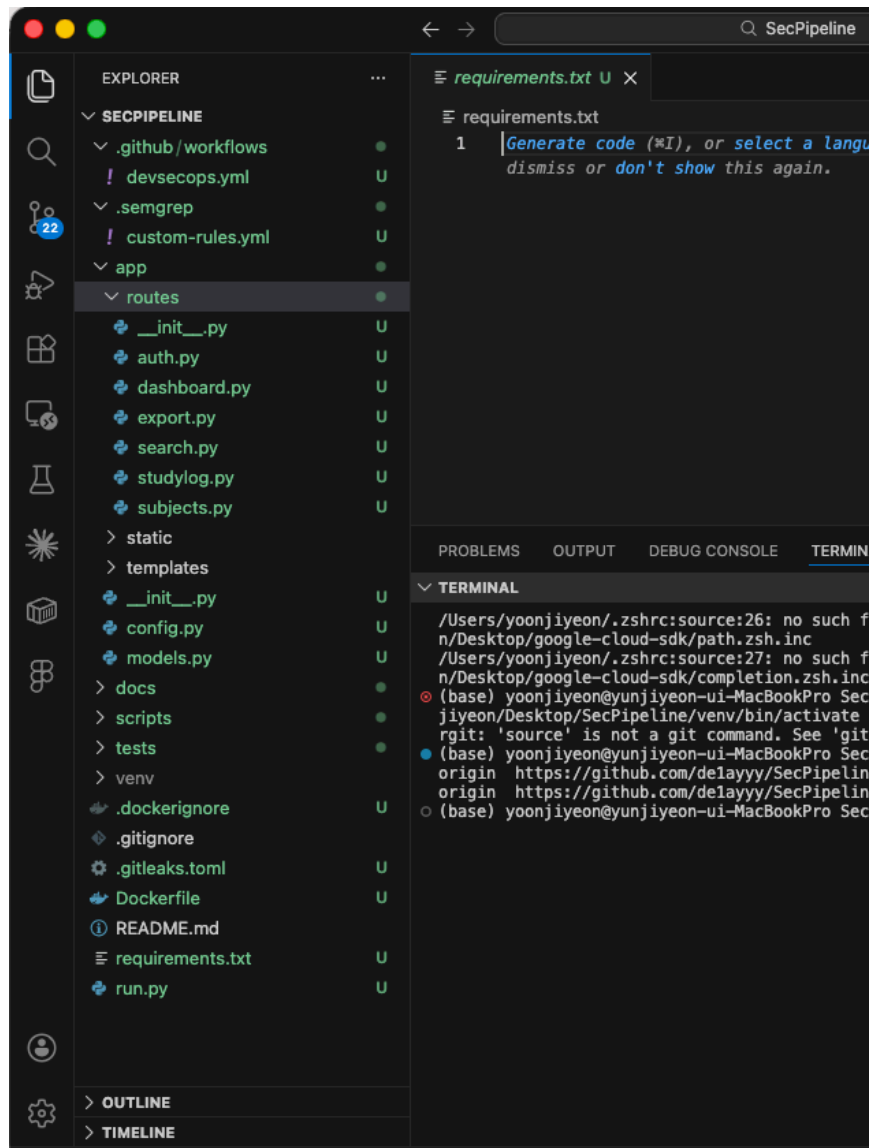
1-3. 가상 환경 생성 및 파일 구조 만들기

프로젝트마다 필요한 라이브러리 버전이 다를 수 있는데, 시스템 전체에 설치하면 프로젝트끼리 충돌이 나기에, 가상환경으로 이 프로젝트만을 위한 독립된 Python 환경을 만들자!

```

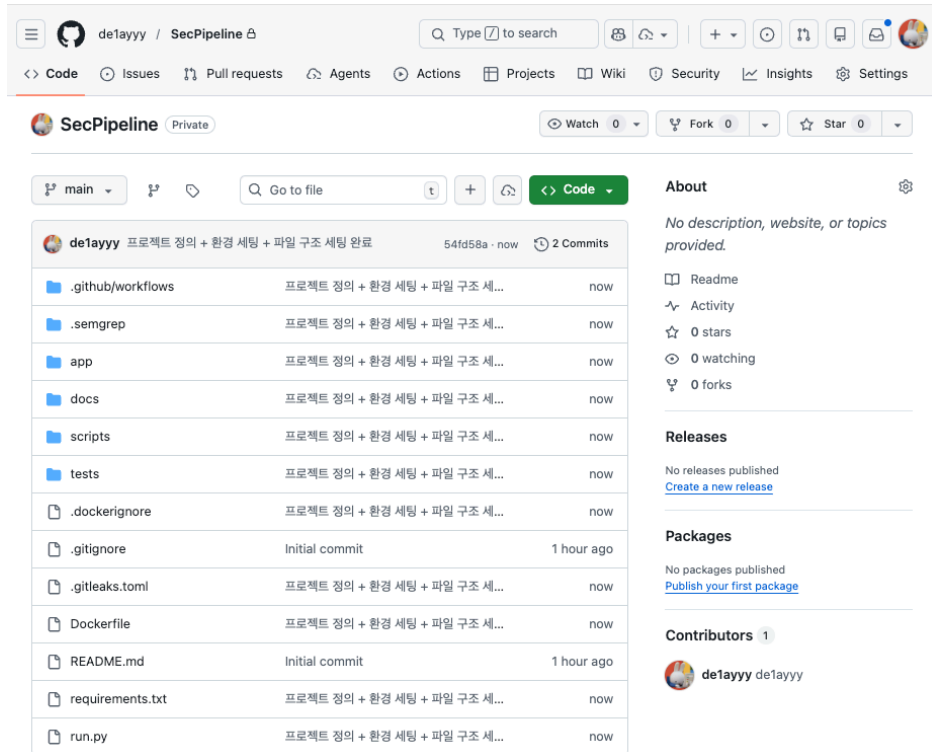
SecPipeline/
├── .github/workflows/devsecops.yml
├── .semgrep/custom-rules.yml
├── .gitleaks.toml
├── app/
│   ├── __init__.py
│   ├── config.py          ← VULN-01
│   ├── models.py
│   ├── routes/
│   │   ├── auth.py
│   │   ├── studylog.py
│   │   ├── subjects.py
│   │   ├── search.py      ← VULN-03a
│   │   ├── dashboard.py  ← VULN-03b
│   │   └── export.py
│   ├── templates/...
│   └── static/...
├── tests/
├── scripts/parse_results.py
├── docs/
├── Dockerfile             ← VULN-04, VULN-05
├── .dockerignore
├── requirements.txt       ← VULN-02
├── run.py
└── README.md

```



이후 git, github에 올리기 완료

```
git remote -v
git add .
git commit -m "프로젝트 정의 + 환경 세팅 + 파일 구조 세팅 완료"
git push origin main
```



1-4. requirements.txt 작성 & 가상 환경에 설치

```

requirements.txt
1 flask==2.2.5
2 jinja2==3.1.2
3 werkzeug==2.2.3
4 gunicorn==21.2.0

```

- 왜 저 패키지들을, 이 버전으로 선택했는가?

패키지	버전	역할	이 버전을 고른 이유
flask	2.2.5	웹 프레임워크 (라우팅, 템플릿, 세션 등)	의도적 취약점 VULN-02: 이 버전에 CVE-2023-30861(세션 쿠키가 cross-site 요청에서 노출되는 취약점)이 존재합니다. pip-audit가 이것을 탐지하도록 일부러 구버전을 사용합니다.
jinja2	3.1.2	HTML 템플릿 엔진 (Flask가 내부적으로 사용)	Flask 2.2.5와 호환되는 버전이면서, 나중에 업그레이드 시 함께 변경할 대상입니다.
werkzeug	2.2.3	WSGI 유틸리티 (Flask의 내부 HTTP 처리 엔진)	Flask 2.2.5가 의존하는 호환 버전입니다.
gunicorn	21.2.0	WSGI 프로덕션 서버	Flask 내장 서버(<code>app.run()</code>)는 개발용이라 동시 요청 처리가 불안정합니다. 실제 배포(Docker, Render)에서는 gunicorn이 Flask 앱 앞에서 요청을 받아 처리합니다.

핵심 포인트: flask==2.2.5 를 쓰는 이유는 단 하나, **CVE-2023-30861**이라는 알려진 취약점이 있는 버전이기 때문입니다. 나중에 Sprint 4에서 flask>=3.1.0 으로 업그레이드하면 pip-audit 경고가 사라지는 것을 확인(Green Run)합니다.

CVE-2023-30861 : Flask 2.2.5 이하에서 웹 브라우저가 Set-Cookie 헤더의 Vary 응답 헤더를 올바르게 받지 못해, 프록시 캐시가 다른 사용자의 세션 쿠키를 반환할 수 있는 취약점입니다. CVSS 7.5 (High). NVD 참조: <https://nvd.nist.gov/vuln/detail/CVE-2023-30861>

본 프로젝트에서는 CVE-2023-30861이 존재하는 Flask 2.2.5를 의도적으로 requirements.txt에 적어 VULN-02를 구성하였음. Schuckert et al. (2023) Insecurity Refactoring의 안전한 코드에 의도적으로 취약점을 삽입하여 보안 도구의 탐

지능력을 검증하는 기법을 참고하여 해당 방법론을 도입하였음. `pip-audit` 가 이 파일을 스캔하여 해당 CVE를 탐지하고, 수정 후 재스캔으로 제거를 확인하는 것이 Gate 2(SCA)의 목적임.

▼ 2. Flask 웹 코드 작성

- `app/config.py` : Flask의 설정 파일, 그 안에 의도적으로 비밀 키를 하드코딩해서 취약점 주입

```
import os
SECRET_KEY = "super-secret-key-12345" # VULN-01: 하드코딩된 시크릿
DATABASE_PATH = os.path.join(os.path.dirname(__file__), '..', 'studylog.db')
```

Flask 앱에는 여러 설정값이 필요한데(세션 암호화에 쓰이는 SECRET_KEY, 데이터베이스 경로, 디버그 모드 여부 등), 이 값들을 `__init__.py`나 `run.py`에 직접 쓰면 코드가 복잡해지고, 환경마다 값을 바꾸기 어려움. 그래서 이 `config.py`를 쓰는 거임.

- 어떤 보안 문제가 있는가?

config.py에 SECRET_KEY에 키를 직접 적고 이 코드를 Git에 커밋하는 순간 Git 히스토리에 영구적으로 기록됨. 저장소가 public 이면 비밀 키가 노출되어 보안 문제가 발생!!

항목	값
CWE	CWE-798: Use of Hard-coded Credentials
OWASP Top 10 2025	A07: Authentication Failures (CWE-798 포함)
CWE Top 25 (2023)	18위 — 상위권 위험 취약점
탐지 도구	Gitleaks (Gate 1: Secret Scan)

의도적 취약점 삽입 & 해결 : VULN-01은 CWE-798(Hard-coded-Credentials)에 해당하며, OWASP Top 10 2025의 A07(Authentication Failures)에 분류됨.

app/config.py에 Flask의 세션 서명 키를 평문으로 하드코딩하여, Gitleaks가 Git 커밋 히스토리에서 해당 시크릿을 탐지하는지 검증! 이후 수정 시에는 12-Factor App의 Config 원칙에 따라 `os.environ.get()`으로 환경변수 주입 방식으로 전환

- `app/models.py` : SQLite 데이터베이스의 스키마를 정의하고, 앱 전체에서 DB 연결을 가져오는 헬퍼 함수 만들기

- 어떤 보안 문제가 있는가?

SQLite는 SQL 쿼리를 직접 문자열로 작성하기 쉬워서, 의도적으로 SQL Injection(VULN-03a)을 삽입하기 좋음.

일단 이 `models.py`에는 취약점이 없음. `models.py`는 테이블 정의와 연결 함수만 있기에 보안 문제가 없음. 하지만 이 파일이 제공하는 `get_db()` 함수를 `search.py`에서 잘못 사용할 때 SQL Injection이 발생함.

- 일단 이 코드는 DB에 연결하고 연결 객체를 반환하는 함수인 `get_db()`, DB는 user 테이블(회원 정보 저장), subjects 테이블(학습 과목), study_logs 테이블(학습 기록의 핵심 테이블)

- `app/__init__.py` : Flask 앱을 생성하고 조립하는 팩토리 함수 `create_app()` 만들기.

- 왜 Application Factory 패턴을 사용하는가?

Flask 공식 문서에 따르면 해당 패턴을 이용해야 `create_app()`을 호출할 때마다 독립된 앱 인스턴스가 만들어지므로, 테스트에서 설정을 바꿔가며 여러 앱을 만들 수 있고 코드가 모듈별로 분리되기에 권장한다고 나와있기에!

- 왜 Blueprint를 사용하는가?

Blueprint는 Flask에서 라우트를 모듈별로 분리하는 기능을 말함. 해당 프로젝트에서는 총 6개의 기능 모듈이 있음.

Blueprint	파일	역할
<code>auth_bp</code>	<code>routes/auth.py</code>	회원이입, 로그인, 로그아웃
<code>studylog_bp</code>	<code>routes/studylog.py</code>	학습 기록 CRUD
<code>subjects_bp</code>	<code>routes/subjects.py</code>	과목 관리
<code>search_bp</code>	<code>routes/search.py</code>	검색 (VULN-03a 위치)
<code>dashboard_bp</code>	<code>routes/dashboard.py</code>	대시보드 (VULN-03b 위치)
<code>export_bp</code>	<code>routes/export.py</code>	CSV 내보내기

Blueprint 없이 하나의 파일에 모든 라우트를 쓰면, Semgrep이나 Bandit이 취약점을 찾았을 때 “어떤 기능의 어떤 부분이 취약한지” 추적하기 어려움. Blueprint로 분리하면 "search.py 15번째 줄에서 SQLi 발견"처럼 **SAST 결과가 기능 단위로 정확히 매핑됨**.

- 각 Blueprint를 앱에 등록한 뒤, 헬스체크 엔드포인트를 넣음

Docker의 HEALTHCHECK 명령이 이 URL을 주기적으로 호출하여 앱이 살아있는지 확인. Render 배포 시에도 이 엔드포인트가 사용됨. `{"status": "ok"}`를 반환하면 정상, 응답이 없으면 컨테이너를 재시작.

```
@app.route('/health')
def health():
    return {"status": "ok"}, 200
```

- `app/run.py`

`run.py`는 Flask Application Factory에서 생성된 앱 인스턴스를 모듈 최상위에 바인딩하여, 개발 환경(python `run.py`)과 운영 환경(gunicorn `run:app`)에서 동일한 진입점을 사용할 수 있게끔 함.

- `run.py`가 필요한 이유

1. Docker, Gunicorn : 운영 환경에서는 python `run.py`가 아닌 gunicorn `run:app`으로 실행함. Gunicorn은 `run.py` 파일에서 `app`이라는 이름의 객체를 찾아서 서빙함. 그래서 `app = create_app()` 이 모듈 최상위에 있어야 함.
2. GitHub Actions/Render : CI/CD에서도 `run:app`이라는 경로로 앱을 참조함.

```
from app import create_app

app = create_app()

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

→ `create_app()`으로 Flask 앱 인스턴스를 생성함. 이 시점에 DB 초기화, Blueprint 등록이 완료됨. python `run.py`로 직접 실행하면 개발 서버로..

- 중간에 잘 돌아가는지 확인

```
(venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % curl http://127.0.0.1:5000/health
{"status": "ok"}
```

- `app/routes/auth.py` : 회원 인증 모듈

`auth.py`는 회원가입, 로그인, 로그아웃 기능을 구현함. 비밀번호는 `werkzeug.security.gnerate_password_hash()`로 bcrypt 해시 처리하여 저장하며, 이는 OWASP Authentication Cheatsheet의 권장 사항을 따름. SQL 쿼리에는 ? 파라미터 바인딩을 사용하여 CWE-89(SQL Injection)를 방지함. 이 모듈은 의도적 취약점을 포함하지 않으며, 이후 `search.py`(VULN-03a)의 취약한 쿼리 패턴과 대비되는 안전한 구현의 기준선 역할을 함.

- 회원가입, 로그인, 로그아웃 기능 만들기. 이 모듈이 만드는 `session['user_id']`와 `session['username']`을 다른 라우트들이 사용하므로, 전체 앱의 인증 기반이 됨.

- `app/routes/studylog.py` : 학습 기록 CRUD

`studylog.py`는 학습 기록의 CRUD를 구현함. 모든 SQL 쿼리에 ? 파라미터 바인딩을 사용하여 CWE-89(SQL Injection)를 방지하며, `session['user_id']`를 WHERE 조건을 포함하여 다른 사용자의 데이터에 접근할 수 없도록 한다. 이 모듈은 `search.py`의 취약한 f-string 쿼리 패턴(VULN-03a)과 동일한 테이블(`study_logs`)에 대해 안전한 구현을 제공하므로 SAST 도구가 두 패턴을 어떻게 다르게 평가하는지 비교하는 기준선이 됨.

- ? 플레이스홀더로 SQLite 드라이버가 값을 안전하게 이스케이프 처리하므로, 사용자가 어떤 입력을 넣어도

- `app/routes/subject.py` : 과목 관리

- 왜 과목 기능이 필요한가?

학습 기록이 쌓이면 “보안”, “네트워크”, “데이터베이스” 등 과목별로 구분하고 싶어짐. subjects 테이블은 study_logs와 1:N 관계로 연결되어, 하나의 과목에 여러 학습 기록이 속함.

라우트	HTTP 메서드	기능
<code>/subjects</code>	GET	과목 목록 조회
<code>/subjects/create</code>	POST	새 과목 추가
<code>/subjects/<id>/delete</code>	POST	과목 삭제

subjects.py는 학습 과목의 추가, 조회, 삭제를 담당하며 모든 쿼리에 파라미터 바인딩을 적용하여 취약점이 없음. SAST 도구가 이 모듈에서 경고를 발생시키지 않는 것은 False Negative가 아닌 True Negative 정도로, `search.py` (VULN-03a)와 `dashboard.py` (VULN-03b)의 탐지 결과와 대비하여 도구의 정밀도(precision)를 평가 가능

- `app/routes/search.py` : 검색 + VULN-03a(SQL Injection)

학습 기록을 키워드로 검색하는 기능을 만들고, 그 안에 의도적으로 SQL Injection 취약점을 삽입할거임.

VULN-03a는 CWE-89(SQL Injection)에 해당하며, OWASP Top 10 2025의 A05(Injection)에 분류됨. `app/routes/search.py`에서 **Python f-string을 사용하여 사용자 검색어 SQL 쿼리에 직접 삽입하는 방식으로 취약점을 구현했음! Semgrep이 Python f-string SQL 패턴을 잘 탐지하는지 확인해야함.**

- SQL Injection이란?

사용자가 입력한 값이 SQL 쿼리 문자열에 직접 삽입되면, 사용자가 SQL 구문 자체를 조작할 수 있음.

```
SELECT * FROM study_logs WHERE title LIKE '%' OR 1=1 --'
```

→ 검색창에 ‘OR 1=1 —’를 입력하면, 쿼리가 저렇게 되어서 모든 사용자의 모든 기록이 반환됨.

항목	값
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)
OWASP Top 10 2025	A05: Injection (2021년에는 A03, 2025년에 A05로 이동했으나 여전히 상위권)
CWE Top 25 (2023)	3위 — 가장 위험한 취약점 중 하나
CVSS 일반 범위	7.5 ~ 9.8 (High ~ Critical)
탐지 도구	Semgrep + Bandit (Gate 3: SAST)

- **취약한 코드 vs 안전한 코드 비교**

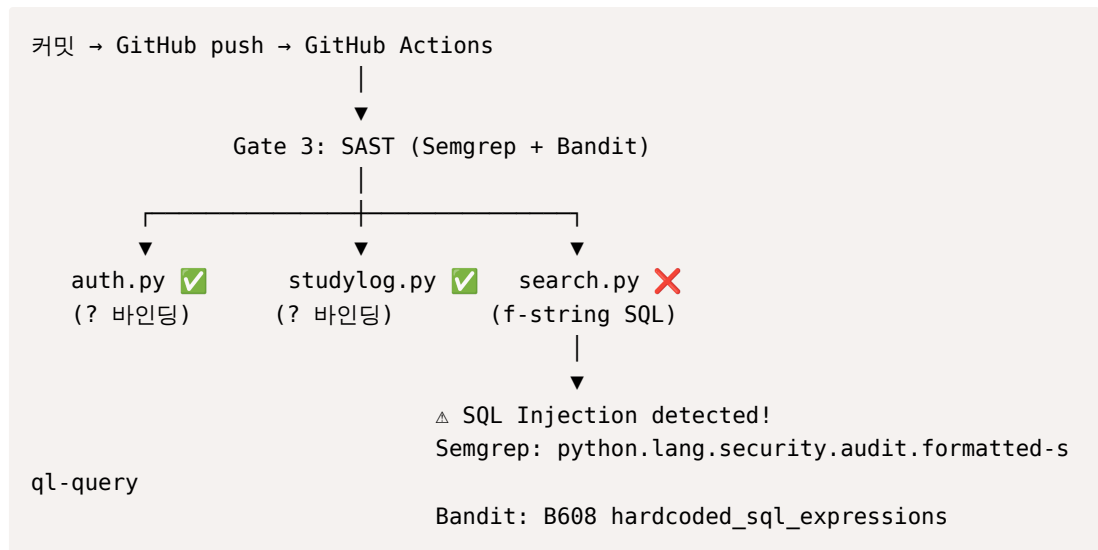
취약한 코드 (해당 파일에서 사용)

```
query = f"SELECT * FROM study_logs WHERE title LIKE '%{keyword}%' AND user_id = {user_id}"
results = db.execute(query).fetchall()
```

안전한 코드 (Sprint 4에서 수정할 때 사용 예정)

```
results = db.execute(
    'SELECT * FROM study_logs WHERE title LIKE ? AND user_id = ?',
    (f'%{keyword}%', user_id)
).fetchall()
```

- 파이프라인에서 어떻게 탐지되는가?



○ 취약점 직접 확인하는 법

서버가 실행 중일 때 브라우저에서

```
http://127.0.0.1:5000/search?q=' OR 1=1 --
```

→ 이렇게 입력하면 모든 학습 기록이 반환됨. 정상적이라면 검색어에 매칭되는 기록만 나와야 함.

- `app/routes/dashboard.py` : 대시보드 + VULN-03b(XSS/SSTI)

로그인한 사용자의 학습 통계(총 기록 수, 총 학습 시간)를 보여주는 대시보드 페이지를 만들었음!

VULN-03b는 CWE-79(XSS)와 CWE-1336(SSTI) 동시에 해당함. `app/routes/dashboard.py` 에서 Python f-string으로 사용자명을 HTML 문자열에 직접 삽입한 뒤 `render_template_string()` 에 전달하는 방식으로 취약점을 구현하였음. 이로 인해 클라이언트 측 스크립트 실행(XSS)과 서버 측 템플릿 구문 실행(SSTI)이 모두 가능함. 수정 시에는 Jinja2의 `{{ username }}` 변수 바인딩을 사용하여 자동 이스케이프를 적용함.

파이프라인 Gate 3에서 `.semgrep/custom-rules.yml`에 정의된 규칙이 이를 탐지하면 성공임.

○ XSS(Cross-Site Scripting)이란?

웹 페이지에 사용자 입력이 HTML로 그대로 삽입되면, 공격자가 JavaScript 코드를 주입할 수 있음.

→ 정상적인 시나리오는 사용자명이 지연일 때 대시보드에 `<h2>지연님의 대시보드</h2>` 가 표시되는데,

→ 공격 시나리오는 회원가입 시 사용자명을 `<script>alert('해킹')</script>` 으로 입력함. 그러면 대시보드에 접속하면 `<h2><script>alert('해킹')</script>님의 대시보드</h2>` 로 나옴. 브라우저가 이것을 HTML로 해석하여 JavaScript가 실행됨. 실제로는 alert 대신 쿠키 탈취, 세션 하이재킹, 피싱 페이지 표시 등을 수행함.

○ SSTI(Server-Side Template Injection)이란?

Flask의 `render_template_string()`은 전달받은 문자열을 Jinja2 템플릿으로 해석함. 사용자의 입력이 이 문자열에 포함되면, 사용자가 Jinja2 구문을 주입하여 서버 측 코드를 실행할 수 있음.

→ 공격 예시 : 사용자명을 `{{ 7*7 }}` 로 등록하면, 대시보드에 `49님의 대시보드` 가 표시됩니다. 즉 **서버에서 연산이 실행된거임**. 더 심각한 공격: `{{ config.SECRET_KEY }}` 를 입력하면 Flask의 시크릿 키가 노출됨. `{{ '.__class__.__mro__[1].__subclasses__()' }}` 같은 페이로드로는 서버의 시스템 명령어까지 실행 가능!!

항목	값
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)
OWASP Top 10 2025	A03: Cross-Site Scripting (2021년 A03 → 2025년에도 A03)
CWE Top 25 (2023)	2위 — SQL Injection(3위)보다 높은 순위
추가 CWE	CWE-1336: Server-Side Template Injection (SSTI)

항목	값
탐지 도구	Semgrep (custom rule) + Bandit (B701 jinja2_autoescape_false)

◦ 취약한 코드 vs 안전한 코드 비교

취약한 코드 (해당 파일에서 일부러 주입)

```
html = f"<h2>{username}님의 대시보드</h2>"
return render_template_string(html + DASHBOARD_BODY, ...)
```

→ username이 f-string으로 HTML 문자열에 직접 삽입되고, 그 결과가 render_template_string()에 전달됨.
그 결과 HTML 이스케이프가 없어서 XSS, Jinja2 템플릿으로 해석되어서 SSTI!

안전한 코드 (Sprint 4에서 수정할 때 사용할 예정)

```
return render_template_string(
    "<h2>{{ username }}님의 대시보드</h2>" + DASHBOARD_BODY,
    username=username, ...
)
```

→ {{ username }}은 Jinja2의 변수 출력 구문임. Jinja2는 기본적으로 자동 이스케이프를 적용하여 <script>를 <script>로 변환함.

◦ 취약점을 직접 확인하는 방법

1. /register 에서 사용자명을 <script>alert('XSS')</script> 로 가입합니다.
2. 로그인 후 /dashboard 에 접속합니다.
3. 브라우저에서 JavaScript alert 창이 뜨면 XSS 성공입니다.

SSTI 확인하는 방법:

1. 사용자명을 {{7*7}}로 가입
2. 대시보드에 49님의 대시보드가 표시되면 SSTI 성공임.

- app/routes/export.py : CSV 내보내기

학습 기록을 CSV 파일로 다운로드하는 기능을 만드는 것.

레퍼런스

- Fu et al. (2025) AI for DevSecOps
- Schuckert et al. (2023) Insecurity Refactoring
- Li et al. (2023) Comparison of SAST Tools
- Bennett et al. (2024) Semgrep Improvements
- OWASP Benchmark, Juliet Test Suite 등

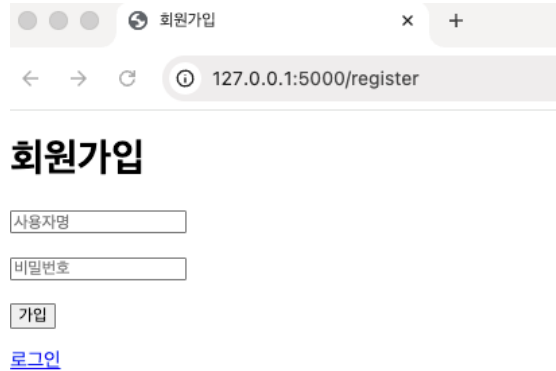
▼ 3. Flask 웹 전체 동작 테스트(중간 점검)

1. 전체 기능 테스트

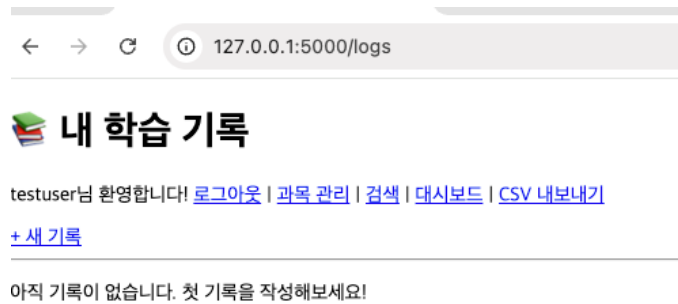
```
# 헬스체크
curl http://127.0.0.1:5000/health

# 회원가입 페이지
curl -s http://127.0.0.1:5000/register | head -5
```

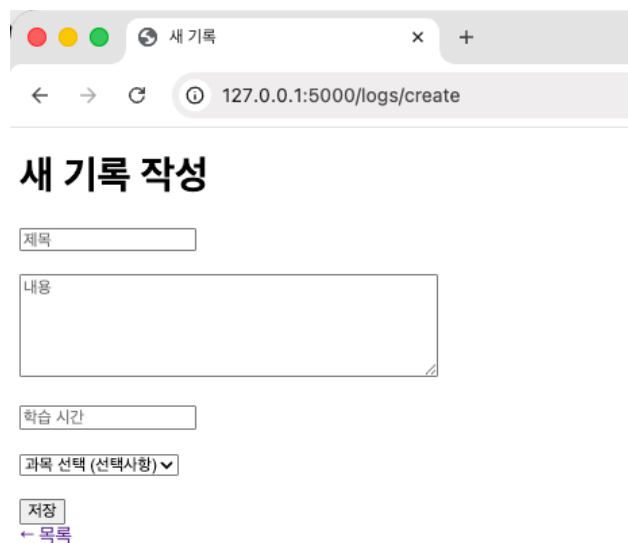
```
# 로그인 페이지
curl -s http://127.0.0.1:5000/login | head -5
```



- ① 사용자명 `testuser`, 비밀번호 `1234` 로 회원가입
- ② 자동으로 로그인 페이지 이동 → 같은 정보로 로그인
- ③ 학습 기록 목록 페이지(`/logs`) 표시 확인



- ④ + 새 기록 클릭 → 제목 `보안 공부`, 내용 `OWASP Top 10 학습`, 시간 `2` 입력 → 저장



⑤ 목록에 기록이 나타나는지 확인

⑥ 검색 클릭 → 보안 입력 → 결과 표시 확인

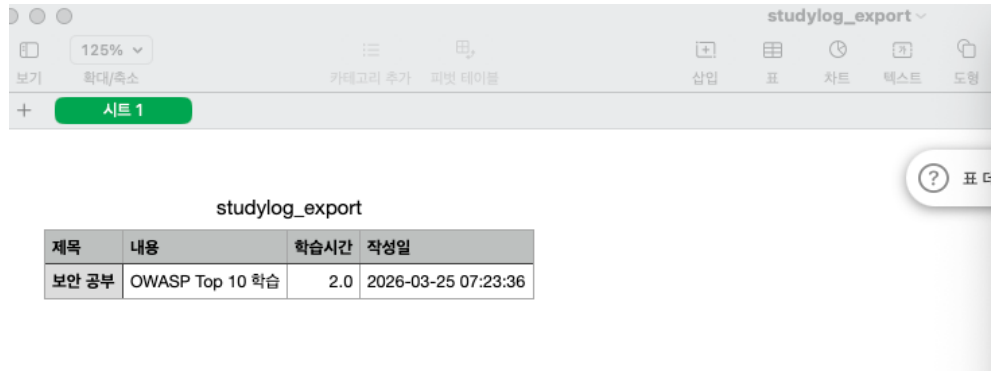
⑦ 대시보드 클릭 → 통계 표시 확인

testuser님의 대시보드

← 학습 기록

총 기록 수	1건
총 학습 시간	2.0시간
등록 과목 수	0개

⑧ CSV 내보내기 클릭 → 파일 다운로드 확인



export.py는 Python 표준 라이브러리(csv, io)만 사용하여 학습 기록을 CSV로 내보내는 기능을 구현함. 외부 의존성이 없고, SQL 쿼리에 파라미터 바인딩을 적용하며, 템플릿 렌더링을 사용하지 않음.

▼ 4. Dockerfile 작성

Flask 앱을 Docker 이미지로 빌드하기 위한 Dockerfile을 작성하고, 그 안에 의도적으로 2가지 취약점을 삽입할 예정

VULN-04 : 구버전 베이스 이미지 (Trivy 탐지)

- 왜 해당 문제를 잡아서 탐지하고자 하는가?

Dockerfile에서의 베이스 이미지를 `python:3.8-slim` 로 설정했음. Python 3.8은 2024.10에 EOL 되어 더 이상 보안 패치가 이루어지지 않기에 이미 알려진 CVE가 수십~수백 개 포함되어 있음. Trivy가 이미지 안의 모든 패키지를 스캔하여 CVE 목록을 출력함.

항목	값
문제 유형	구버전 베이스 이미지에 포함된 알려진 취약점들
탐지 도구	Trivy (Gate 4: Container Scan)
수정 방법	<code>FROM python:3.12-slim</code> 으로 교체

Build Stage에서 Container Image Scanning을 필수 활동으로 정의함. Docker 공식 문서에 따르면 "10 Docker Image Security Best Practices"에서도 최신 안정 버전의 slim/alpine 이미지를 사용을 권장함.

VULN-05 : Dockerfile 베스트 프랙티스 위반 (hadolint 탐지)

- 무엇이 문제인가?

hadolint는 Dockerfile의 작성 규칙 위반을 검사함. (아래와 같이 검사하도록 설정 예정)

hadolint 규칙	위반 내용	위험성
DL3007	<code>FROM python:3.8-slim</code> 대신 <code>latest</code> 태그 미사용은 아니지만, EOL 버전 사용	보안 패치 부재
DL3059	여러 <code>RUN</code> 명령을 합치지 않음	이미지 레이어 불필요 증가

hadolint 규칙	위반 내용	위험성
DL3025	CMD 에서 shell form 사용	시그널 전달 문제
(missing)	USER 지시어 없음 → root로 실행	컨테이너 탈출 시 호스트 root 권한 획득 가능
(missing)	HEALTHCHECK 없음	컨테이너 상태 모니터링 불가

- 특히 Docker 컨테이너 안에서 프로세스가 root로 실행되면, container escape 취약점이 발생했을 때 호스트 시스템의 root 권한을 얻을 수 있음. 그래서 USER appuser로 비특권 사용자로 지정해야함.

- 아래와 같이 보안 분류

항목	값
CWE	CWE-250: Execution with Unnecessary Privileges
탐지 도구	hadolint (Gate 5: Dockerfile Lint)
수정 방법	USER, HEALTHCHECK 추가, 레이어 최적화

Dockerfile (의도적으로 나쁘게 설정한 Dockerfile)

```
#VULN-04 : EOL 버전 Python 3.8 사용 (Trivy가 다수의 CVE 탐지함)
From python:3.8-slim

# VULN-05 : WORKDIR 없이 COPY (hadolint DL3045)
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt

COPY . .
EXPOSE 5000

# VULN-05: USER 지시어 없음 → root로 실행 (CWE-250)
# VULN-05: HEALTHCHECK 없음
# VULN-05: shell form CMD (hadolint DL3025)
CMD python run.py
```

하나씩 톺아보면 ..

- From python:3.8-slim
Docker 이미지는 항상 다른 이미지 위에 만들. 완전히 빈 OS부터 시작할 수는 있으나, 그러면 Python을 직접 컴파일해야 하기에 python이 이미 설치된 이미지를 출발점으로 설정했음.
→ 여기서 VULN-04가 발생하는데, Python 3.8은 EOL이기에 hadolint가 이를 탐지함.
→ 올바르게 수정하기 위해서는 FROM python:3.12-slim
- WORKDIR /app
이 줄이 있어야 모든 파일이 / 루트 디렉토리에 복사가 되어 시스템 파일과 섞이지 않으나, 취약한 Dockerfile을 작성하기 위해서 일부러 빼놓았음. 나중에 hadolint가 DL3045 규칙으로 "WORKDIR 없이 COPY 하면 안된다"고 경고할 것임.
- 전체 코드를 한 번에 복사하지 않고 requirements.txt 미리 먼저 복사해야함.
Docker는 각 줄을 레이어로 저장하고 캐시함. requirements.txt가 바뀌지 않으면 pip install 단계를 캐시에서 재사용함. 하지만 전체 코드를 먼저 복사하면, 코드 한 줄만 바뀌어도 pip install 매번 다시 실행됨. 이는 Docker 빌드 최적화의 기본 임!
- EXPOSE 5000
Flask가 5000번 포트에서 실행되므로, 컨테이너의 5000번 포트를 외부에 노출함. 실제 포트 매핑은 docker run -p 5000:5000으로.
- CMD python run.py

→ CMD ["gunicorn", "--bind", "0.0.0.0:5000", "run:app"] 으로 안전한 방식으로 바꾸어야 함.

- ```
RUN adduser --disabled-password --gecos "" appuser
USER appuser
```

- 안전한 Dockerfile로 바꾸기 위해서는

```

docker build -t studylog:red .
├── "docker" = Docker CLI 프로그램
├── "build" = Dockerfile을 읽어서 이미지를 만들어라
├── "-t" = tag의 약자, 이미지에 이름을 붙여라
├── "studylog" = 이미지 이름
├── "red" = 태그 (우리가 붙이는 버전 이름)
└── "." = 현재 디렉토리에서 Dockerfile을 찾아라

```

- 나머지 모든 코드 (app/, run.py 등) 복사
- 레이어 4 생성

Step 5/5 : CMD python run.py

- "이 이미지를 실행하면 python run.py를 시작하라"는 설정 저장
- 레이어 5 생성

Successfully built abc123def456

Successfully tagged studylog:red

```
(venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % docker build -t studylog:red .
[+] Building 22.2s (9/9) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 437B 0.0s
WARN: ConsistentInstructionCasing: Command 'From' should match the case of th 0.0s
=> [internal] load metadata for docker.io/library/python:3.8-slim 2.7s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 34B 0.0s
=> [1/4] FROM docker.io/library/python:3.8-slim@sha256:1d52838af602b4b5a831beb1 15.4s
=> => resolve docker.io/library/python:3.8-slim@sha256:1d52838af602b4b5a831beb13 0.0s
=> => sha256:0cd45dda32d5387b73cdb7755ea8f31eca0e250f2860b720e9e5155 249B / 249B 0.5s
=> => sha256:56d45b8e140401cb0bb97c76a062697a0502ff1928dad8483 14.51MB / 14.51MB 9.4s
=> => sha256:d847ad1879f2202660d37d9bedf1b8986116079d578b010e46b 3.33MB / 3.33MB 3.1s
=> => sha256:14c9d9d199323cbf0a4c2347a8af85f2875c1f2c26a1558f 29.16MB / 29.16MB 14.7s
=> => extracting sha256:14c9d9d199323cbf0a4c2347a8af85f2875c1f2c26a1558f34dfca7 0.4s
=> => extracting sha256:d847ad1879f2202660d37d9bedf1b8986116079d578b010e46b6917f 0.0s
=> => extracting sha256:56d45b8e140401cb0bb97c76a062697a0502ff1928dad8483d76b108 0.2s
=> => extracting sha256:0cd45dda32d5387b73cdb7755ea8f31eca0e250f2860b720e9e51552 0.0s
=> [internal] load build context 0.2s
=> => transferring context: 16.53MB 0.2s
=> [2/4] COPY requirements.txt requirements.txt 0.1s
=> [3/4] RUN pip install -r requirements.txt 3.3s
=> [4/4] COPY . . . 0.1s
=> => exporting to image 0.6s
=> => exporting layers 0.4s
=> => exporting manifest sha256:d7ea46e47298fb936dce8bc607865ef5d6c2e3fe50bd4523 0.0s
=> => exporting config sha256:22505fe56323f19f0cacd2600c38fb655296cfd1f0861c6ab 0.0s
=> => exporting attestation manifest sha256:5835637779bd329be803de23a38ca09480b7 0.0s
=> => exporting manifest list sha256:aada3de10c7c294721e381de746d37dc5dfc62f9e67 0.0s
=> => naming to docker.io/library/studylog:red 0.0s
=> => unpacking to docker.io/library/studylog:red 0.2s

2 warnings found (use docker --debug to expand):
- ConsistentInstructionCasing: Command 'From' should match the case of the command majori
rity (uppercase) (line 2)
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavio
r related to OS signals (line 14)

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/xrjuyzc
lq64owcy3ldhsuggzy
```

#### • 여기서 발생한 경고

- `JSONArgsRecommended` — `CMD python run.py` (shell form) 대신 `CMD ["python", "run.py"]` (exec form, JSON 형식)를 쓰라는 경고. 이것이 hadolint DL3025 규칙과 동일함. shell form은 OS 시그널이 프로세스에 전달되지 않아 컨테이너가 깔끔하게 종료되지 않을 수 있음.

```
docker run --rm -p 5000:5000 studylog:red
```

```
(venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % docker run --rm -p 5000:5000 studylog:red
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 114-376-152
```

→ 로컬에서 실행하는 건 맥북의 Python, 맥북의 패키지, 맥북의 파일시스템을 직접 사용하는데, Docker 로 실행하는 경우 (docker run studylog:red)는 컨테이너 안의 Python, 컨테이너 안의 패키지, 컨테이너 안의 파일시스템을 이용함. 어떤 컴퓨터든 동일한 환경을 보장함.

```

p/secpipeline/venv/bin/activate
• (venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % curl http://127.0.0.1:5000/health
{
 "status": "ok"
}
○ (venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline %

```

→ Docker 테스트 완료

## ▼ 5. 보안 도구 설정

### 5.1 Gitleaks?

#### Gitleaks란?

개발을 하다 보면 API 키, 데이터베이스 비밀번호, AWS 접근 토큰 같은 민감한 정보를 코드에 직접 적어놓는 실수를 하게 됨. 이러한 실수로 AWS 요금이 수천만 원이 나오거나, 데이터베이스가 통째로 털리는 사고가 빈번하게 발생함.

Gitleaks는 Git 저장소의 모든 커밋 기록을 스캔해서 비밀번호, API 키, 토큰처럼 생긴 문자열을 정규표현식 패턴으로 탐지함. 현재 코드에서 지웠더라도 과거 커밋에 남아있으면 그것까지 찾아냄.

→ 해당 DevSecOps 파이프라인의 Gate 중 하나로 Gitleaks가 들어가는데, 개발자가 코드를 push하면 자동으로 Gitleaks가 돌면서 검사하고, 발견하면 배포를 차단하는 방식임. [.gitleaks.toml](https://github.com/gitleaks/gitleaks) 파일로 구현 예정

### 5.2 .gitleaks.toml

#### • 해당 파일의 역할

Gitleaks는 기본 내장 규칙으로 AWS 키, GitHub 토큰 등등 150개 이상의 패턴을 감지하는데, 실제 프로젝트에서는 오탐이 발생함. 예를 들어 가상환경 안의 라이브러리 코드, 테스트용 데미 키, 문서 속 예시 코드 등이 "시크릿"으로 잘못 탐지됨. 그래서 .gitleaks.toml에서 allowlist를 설정해 이런 오탐 경로를 스캔에서 제외하도록 했음.

### 5.3 .semgrep/custom-rules.yml

#### • 해당 파일의 역할

Semgrep의 `—config auto` 옵션은 수천 개의 기본 규칙을 제공함. VULN-03a는 기본 규칙으로 잘 탐지되지만, VULN-03b(f-string으로 HTML을 조합하는 것)는 기본 규칙만으로는 놓칠 수 있음. Bennett et al. (2024)은 프로젝트 고유의 커스텀 Semgrep 규칙이 기본 규칙 대비 탐지 범위를 유의미하게 확장한다고 했으니 적용해보고자 함.

#### • 규칙 동작 원리

```

auth.py: render_template_string(LOGIN_HTML, ...) → 상수 문자열 → 통과
dashboard.py: render_template_string(header + ..., ...) → 변수 포함 → 경고!

```

pattern은 `render_template_string()` 호출을 모두 찾고, `pattern-not`은 첫 인자가 리터럴 문자열인 경우 제외함. 결과적으로 변수가 포함된 동적 문자열이 전달될 때만 경고함.

## ▼ 6. CI/CD 파이프라인 구축

#### 요약 :

GitHub Actions 기반의 5-Gate DevSecOps 파이프라인을 `.github/workflows/devsecops.yml`에 정의하였음. Fu et al. (2024)의 DevSecOps 5단계 프레임워크 (Code Commit → Build → Test → Deploy → Monitor)에 따라 각 단계에 보안 게이트를 배치하였음.

- Gate 1 (Secret Scan) : gitleaks/gitleaks-action@v2를 사용하여 Git 히스토리 전체에서 하드코딩된 시크릿을 탐지
- Gate 2 (Dependency Scan) : pip-audit로 requirements.txt의 패키지를 PyPI Advisory Database와 대조
- Gate 3 (SAST) : Semgrep와 Bandit을 조합하여 정적 코드 분석을 수행. Semgrep은 기본 규칙(`—config auto`)과 프로젝트 고유 규칙(`—config.semgrep/`)을 동시에 적용한다.
- Gate 4 (Container Scan) : aquasecurity/trivy-action@v0.35.0으로 Docker 이미지 내 OS 패키지 및 언어별 패키지의 알려진 CVE 스캔

- Gate 5 (Dockerfile Lint) : hadolint/hadolint-action@v3.3.0으로 Dockerfile의 모범 사례 위반 검사

모든 Gate의 스캔 결과는 actions/upload-artifact를 통해 JSON 형태로 보존되어, Red Run/Green Run 비교 분석에 활용됨.

## 6.1 일단 개념부터 알아보자

### 1. GitHub Actions란?

GitHub Actions는 GitHub 저장소에 코드를 Push하면 자동으로 원하는 작업을 실행해주는 자동화 서비스임. 개발자가 매번 수동으로 테스트를 돌리고, 빌드하고, 배포하는 것이 아니라, push라는 이벤트가 발생하면 GitHub가 알아서 미리 정해놓은 작업을 순서대로 실행해주는 것임.

### 2. CI/CD란?

CI(Continuous Integration)는 개발자가 코드를 push할 때마다 자동으로 테스트를 돌려서, 새 코드가 기존 코드를 망가뜨리지 않는지 확인하는 것.

CD(Continuous Deployment/Delivery)는 테스트를 통과한 코드를 자동으로 서버에 배포하는 것임. 수동으로 서버에 접속해서 파일 복사하고 재시작하는게 아니라 파이프라인이 알아서 해주는거임.

### 3. GitHub Actions 사용 방법

#### • 핵심 구조

GitHub Actions는 .github/workflows/ 폴더 안에 YAML 파일 하나만 만들면 작동함. 이 경로는 github가 정한 규칙이라 바꿀 수 없음.

#### • 가장 기본적인 예시

```
.github/workflows/ci.yml

name: CI Pipeline # 파이프라인 이름 (자유롭게 지정)

on:
 push: # 언제 실행할지 → push할 때마다
 branches: [main] # main 브랜치에 push할 때만

jobs:
 test: # job 이름 (자유롭게 지정)
 runs-on: ubuntu-latest # 어떤 환경에서 실행할지 → GitHub이 제공하는 Ubuntu VM

 steps: # 실행할 단계들을 순서대로 나열
 - name: 코드 가져오기
 uses: actions/checkout@v4 # GitHub 저장소 코드를 VM에 복사

 - name: Python 설치
 uses: actions/setup-python@v5 # Python 환경 세팅
 with:
 python-version: "3.10"

 - name: 라이브러리 설치
 run: pip install -r requirements.txt # 필요한 패키지 설치

 - name: 테스트 실행
 run: pytest tests/ # 테스트 돌리기
```

이 파일을 저장하고 main 브랜치에 push하면, GitHub이 자동으로 Ubuntu 가상머신을 하나 띄우고, 코드를 복사하고, Python을 설치하고, 라이브러리를 깔고, 테스트를 실행합니다. 결과는 GitHub 저장소의 **Actions 탭**에서 확인할 수 있습니다. 테스트가 실패하면 빨간색, 성공하면 초록색으로 표시됩니다.

- YAML 파일의 핵심 개념 4가지
  - **on** — 언제 실행할지를 정의합니다. `push`, `pull_request`, 심지어 `schedule` (크론잡처럼 특정 시간)도 가능합니다.
  - **jobs** — 실행할 작업 묶음입니다. 여러 job을 정의하면 기본적으로 병렬로 실행되고, 의존관계를 설정하면 순차 실행도 가능합니다.
  - **runs-on** — 작업이 실행될 환경입니다. GitHub이 `ubuntu-latest`, `windows-latest`, `macos-latest` 같은 가상머신을 무료로 제공합니다(공개 저장소 기준).
  - **steps** — job 안에서 순서대로 실행할 개별 명령어들입니다. `uses` 는 남이 만들어놓은 재사용 가능한 액션을 가져다 쓰는 것이고, `run` 은 직접 쉘 명령어를 치는 것입니다.

1. Workflow : `.github/workflows/` 폴더 안의 YAML 파일 하나가 워크플로우 하나임.
2. Event : 워크플로우를 언제 실행할 것인가? 해당 프로젝트에서는 push와 pull\_request시 실행함.
3. Job : 워크플로우 안의 독립된 작업 단위. 각 Job은 별도의 가상 머신(Ubuntu)에서 실행됨. 해당 프로젝트에서는 5개 Gate가 각각의 Job이 됨.
4. Step : Job 안의 개별 명령어임. `uses`로 마켓플레이스 액션을 사용하거나 `run`으로 셸 명령어를 실행하기도 함.

#### 4. DevSecOps에서의 CI/CD

## 6.2 우리 프로젝트에 적용해보자!

```
Workflow (devsecops.yml)
├─ Job: secret-scan (Gate 1)
│ ├── Step: actions/checkout
│ └─ Step: gitleaks/gitleaks-action
├─ Job: dependency-scan (Gate 2)
│ ├── Step: actions/checkout
│ ├── Step: actions/setup-python
│ └─ Step: run pip-audit
├─ Job: sast (Gate 3)
│ ├── Step: actions/checkout
│ ├── Step: actions/setup-python
│ └─ Step: run semgrep + bandit
├─ Job: build-and-scan (Gate 4)
│ ├── Step: actions/checkout
│ ├── Step: docker build
│ └─ Step: aquasecurity/trivy-action
└─ Job: dockerfile-lint (Gate 5)
 ├── Step: actions/checkout
 └─ Step: hadolint/hadolint-action
```

## 전체 코드

```

#
학술 근거: Fu et al. (2024), "AI for DevSecOps"
이 파이프라인은 논문의 5단계 프레임워크
(Code Commit → Build → Test → Deploy → Monitor)를 구현한다.

name: SecPipeline DevSecOps

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 # — Gate 1: Secret Scan (Gitleaks) —————
 secret-scan:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 with:
 fetch-depth: 0
 - uses: gitleaks/gitleaks-action@v2
 env:
 GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

 # — Gate 2: Dependency Scan (pip-audit) —————
 dependency-scan:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-python@v5
 with:
 python-version: "3.12"
 - run: |
 pip install pip-audit
 pip-audit -r requirements.txt --format json --output dep-scan.json || true
 e
 - uses: actions/upload-artifact@v4
 if: always()
 with:
 name: dependency-scan
 path: dep-scan.json

 # — Gate 3: SAST (Semgrep + Bandit) —————
 sast:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-python@v5
 with:
 python-version: "3.12"
 - run: |
 pip install semgrep bandit
 semgrep scan --config auto --config .semgrep/ app/ --json --output semgre

```

```

p.json || true
 bandit -r app/ -f json -o bandit.json || true
- uses: actions/upload-artifact@v4
 if: always()
 with:
 name: sast-results
 path: |
 semgrep.json
 bandit.json

— Gate 4: Build + Container Scan (Trivy) —
build-and-scan:
 runs-on: ubuntu-latest
 needs: [secret-scan, dependency-scan, sast]
 steps:
 - uses: actions/checkout@v4
 - run: docker build -t studylog:${{ github.sha }} .
 - name: Trivy image scan
 uses: aquasecurity/trivy-action@v0.35.0
 with:
 image-ref: studylog:${{ github.sha }}
 format: json
 output: trivy.json
 - uses: actions/upload-artifact@v4
 if: always()
 with:
 name: trivy-results
 path: trivy.json

— Gate 5: Dockerfile Lint (hadolint) —
dockerfile-lint:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - name: Lint Dockerfile
 uses: hadolint/hadolint-action@v3.3.0
 with:
 dockerfile: Dockerfile

```

- Gate 1 : Gitleaks (Secret Scan)
  - runs-on : ubuntu-latest → 이 job을 ubuntu 가상머신에서 실행
  - actions/checkout@v4 → 저장소의 코드를 가상머신으로 복사함. (모든 job의 첫 번째 step)
  - fetch-depth: 0 → Git 히스토리 전체를 가져옴. Gitleaks는 과거 커밋까지 스캔하므로 전체 히스토리가 필요함. 이 옵션이 없으면 최신 커밋 1개만 가져와서 과거에 삽입된 시크릿을 놓침.
  - gitleaks/gitleaks-action@v2 : GitHub Marketplace의 Gitleaks 액션임. 코드를 스캔하여 시크릿 패턴을 찾음.
- Gate 2 : pip-audit (Dependency Scan)
  - actions/setup-python@v5 : 가상머신에 python 3.12를 설치.
  - run: | : 파이프( | ) 뒤에 여러 줄의 셸 명령어를 작성합니다. pip-audit 를 설치하고, requirements.txt 를 스캔하여 JSON으로 결과를 저장
  - || true : pip-audit가 취약점을 발견하면 종료 코드 1(실패)을 반환

- `actions/upload-artifact@v4` : 스캔 결과 파일을 GitHub Actions의 **Artifact**(첨부파일)로 저장합니다. 나중에 Actions 탭에서 다운로드할 수 있습니다. `if: always()` 는 이전 스텝이 실패해도 항상 업로드하라는 의미
- Gate 3 : Semgrep + Bandit (SAST)
  - `semgrep scan --config auto --config .semgrep/ app/` : 두 가지 규칙 소스를 동시에 사용합니다. `--config auto` 는 Semgrep 공식 규칙(수천 개), `--config .semgrep/` 는 우리가 만든 커스텀 규칙입니다. `app/` 는 스캔 대상 디렉토리입니다.
  - `bandit -r app/ -f json -o bandit.json` : Bandit으로 `app/` 폴더를 재귀적(`-r`)으로 스캔하고 JSON 형식으로 저장합니다. Bandit은 Python AST(Abstract Syntax Tree)를 분석하여 B608(SQL Injection), B701(Jinja2 autoescape) 등의 규칙을 검사합니다.

→ 이 두 도구를 같이 써서 서로 다른 관점에서 코드를 분석하도록 함.
- Gate 4 : Docker Build + Trivy (Container Scan)
  - `needs: [secret-scan, dependency-scan, sast]` : Gate 1, 2, 3가 모두 완료된 후에 실행합니다. Docker 빌드는 시간이 걸리므로, 빠른 검사를 먼저 돌리고 통과한 후에 빌드합니다.
  - `{{ github.sha }}` : 현재 커밋의 SHA 해시값입니다. 이미지 태그로 사용하면 "이 이미지가 어떤 커밋에서 빌드되었는지" 추적할 수 있습니다
  - `aquasecurity/trivy-action@v0.35.0` : Trivy로 Docker 이미지를 스캔합니다. **반드시 v0.35.0을 사용합니다.** 2026년 3월 19일 공급망 공격으로 다른 버전 태그가 악성 코드로 오염되었기 때문
- Gate 5 : hadolint (Dockerfile Lint)
  - `hadolint/hadolint-action@v3.3.0` : Dockerfile의 모범 사례 위반을 검사합니다. `USER` 누락, `HEALTHCHECK` 누락, shell form CMD 등을 규칙 번호(DL3002, DL3025 등)와 함께 보고합니다.
  - 이 잡은 다른 잡에 의존하지 않으므로(`needs` 없음) Gate 1, 2, 3과 **동시에** 실행됩니다.

## ▼ 7. Push하고 Red Run 확인하기

지금까지 만든 모든 파일을 Git에 커밋하고 GitHub에 push함. push하는 순간 GitHub Actions가 자동으로 5-Gate 파이프라인을 실행함. 5가지 의도적 취약점이 탐지되는 것을 확인하는 것이 Red Run!@

1. `git status`

```
(venv) (base) yoonjiyeon@yoonjiyeon-ui-MacBookPro SecPipeline % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
 (use "git add <file>..." to update what will be committed)
 (use "git restore <file>..." to discard changes in working directory)
 modified: .github/workflows/devsecops.yml
 modified: .gitignore
 modified: .gitleaks.toml
 modified: .semgrep/custom-rules.yml
 modified: Dockerfile
 modified: app/__init__.py
 modified: app/config.py
 modified: app/models.py
 modified: app/routes/__init__.py
 modified: app/routes/auth.py
 modified: app/routes/dashboard.py
 modified: app/routes/export.py
 modified: app/routes/search.py
 modified: app/routes/studylog.py
 modified: app/routes/subjects.py
 modified: run.py

no changes added to commit (use "git add" and/or "git commit -a")
```

2. `git add .`, `git status`, `git commit -m ..`, `git push origin main`



```

no changes added to commit (use "git add" and/or "git commit -a")
• (venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git add .
• (venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git stage
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'
hint: Disable this message with "git config advice.addEmptyPathsSpec false"
• (venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
 (use "git restore --staged <file>..." to unstage)
 modified: .github/workflows/devsecops.yml
 modified: .gitignore
 modified: .gitleaks.toml
 modified: .semgrep/custom-rules.yml
 modified: Dockerfile
 modified: app/__init__.py
 modified: app/config.py
 modified: app/models.py
 modified: app/routes/__init__.py
 modified: app/routes/auth.py
 modified: app/routes/dashboard.py
 modified: app/routes/export.py
 modified: app/routes/search.py
 modified: app/routes/studylog.py
 modified: app/routes/subjects.py
 modified: run.py

• (venv) (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git commit -m
"Fask 앱 배포 + 5가지 의도적 취약점 삽입 + 5-gate CI/CD 파이프라인 추가"
[main 0de7ea8] Fask 앱 배포 + 5가지 의도적 취약점 삽입 + 5-gate CI/CD 파이프
라인 추가
16 files changed, 712 insertions(+)

```

SecPipeline
Private
Watch 0
Fork 0
Star 0

main
Go to file
Code

de1ayyy

Fask 앱 배포 + 5가지 의도적 취약점 삽입 ...

0de7ea8 · 3 minutes ago

|                   |                           |               |
|-------------------|---------------------------|---------------|
| .github/workflows | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |
| .semgrep          | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |
| app               | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |
| docs              | 프로젝트 정의 + 환경 세팅 + 파일 ...  | yesterday     |
| scripts           | 프로젝트 정의 + 환경 세팅 + 파일 ...  | yesterday     |
| tests             | 프로젝트 정의 + 환경 세팅 + 파일 ...  | yesterday     |
| .dockerignore     | 프로젝트 정의 + 환경 세팅 + 파일 ...  | yesterday     |
| .gitignore        | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |
| .gitleaks.toml    | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |
| Dockerfile        | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |
| README.md         | Initial commit            | yesterday     |
| requirements.txt  | 프로젝트 정의 + 환경 세팅 + 파일 ...  | yesterday     |
| run.py            | Fask 앱 배포 + 5가지 의도적 취약... | 3 minutes ago |

README

About

No description, website, or topics provided.

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

de1ayyy

Languages

- Python 98.0%
- Dockerfile 2.0%

## 2. GitHub Actions 확인해보기

Flask 앱 배포 + 5가지 의도적 취약점 삽입 + 5-gate CI/CD 파이프라인 추가 #2

Summary

All jobs

- secret-scan
- dependency-scan
- sast
- dockerfile-lint
- build-and-scan

Run details

- Usage
- Workflow file

Triggered via push 4 minutes ago

Status: **Failure**

Total duration: **1m 15s**

Artifacts: **4**

devsecops.yml

on: push

secret-scan 10s

dependency-scan 18s

sast 31s

build-and-scan 36s

dockerfile-lint 8s

secret-scan summary

**No leaks detected** ✓

Job summary generated at run-time

Annotations

secret-scan summary

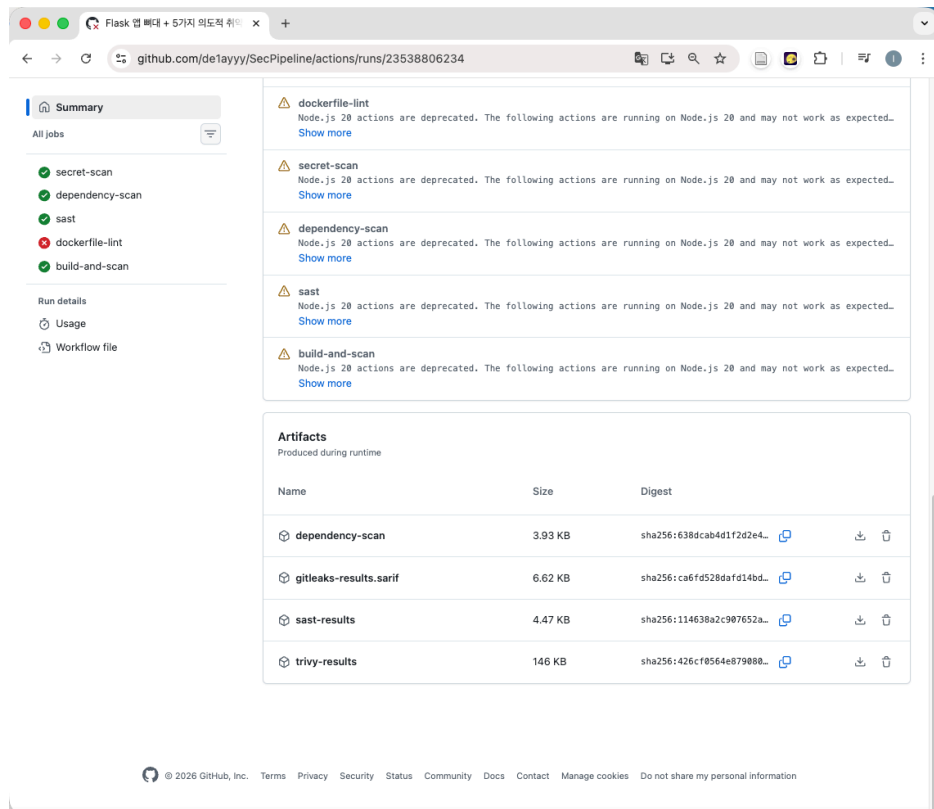
**No leaks detected** ✓

Job summary generated at run-time

Annotations

4 errors and 5 warnings

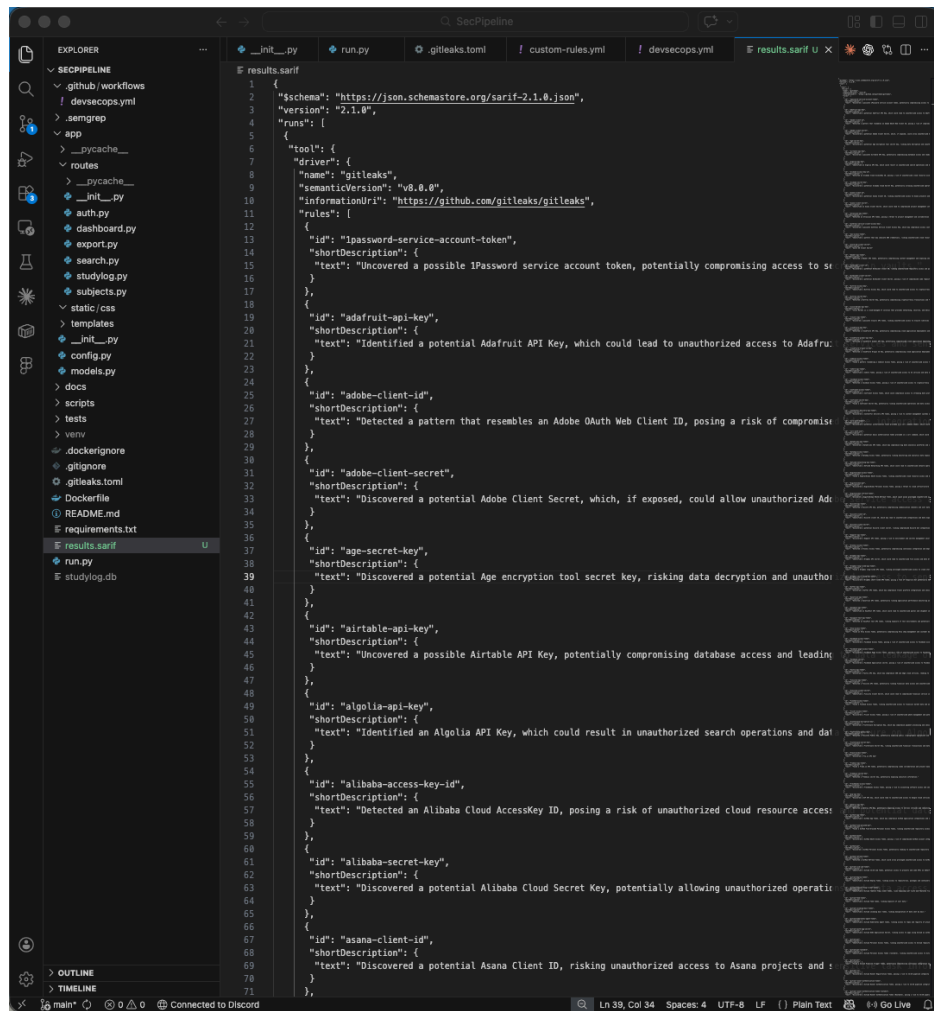
- dockerfile-lint: Dockerfile#L14  
DL3025 warning: Use arguments JSON notation for CMD and ENTRYPOINT arguments
- dockerfile-lint: Dockerfile#L8  
DL3045 warning: 'COPY' to a relative destination without 'WORKDIR' set.
- dockerfile-lint: Dockerfile#L6  
DL3042 warning: Avoid use of cache directory with pip. Use 'pip install --no-cache-dir <packages>'
- dockerfile-lint: Dockerfile#L5  
DL3045 warning: 'COPY' to a relative destination without 'WORKDIR' set.
- dockerfile-lint  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected...  
[Show more](#)
- secret-scan  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected...  
[Show more](#)
- dependency-scan  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected...  
[Show more](#)
- sast  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected...  
[Show more](#)
- build-and-scan  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected...  
[Show more](#)



## ▼ 8. 각 Gate별로 분석하기

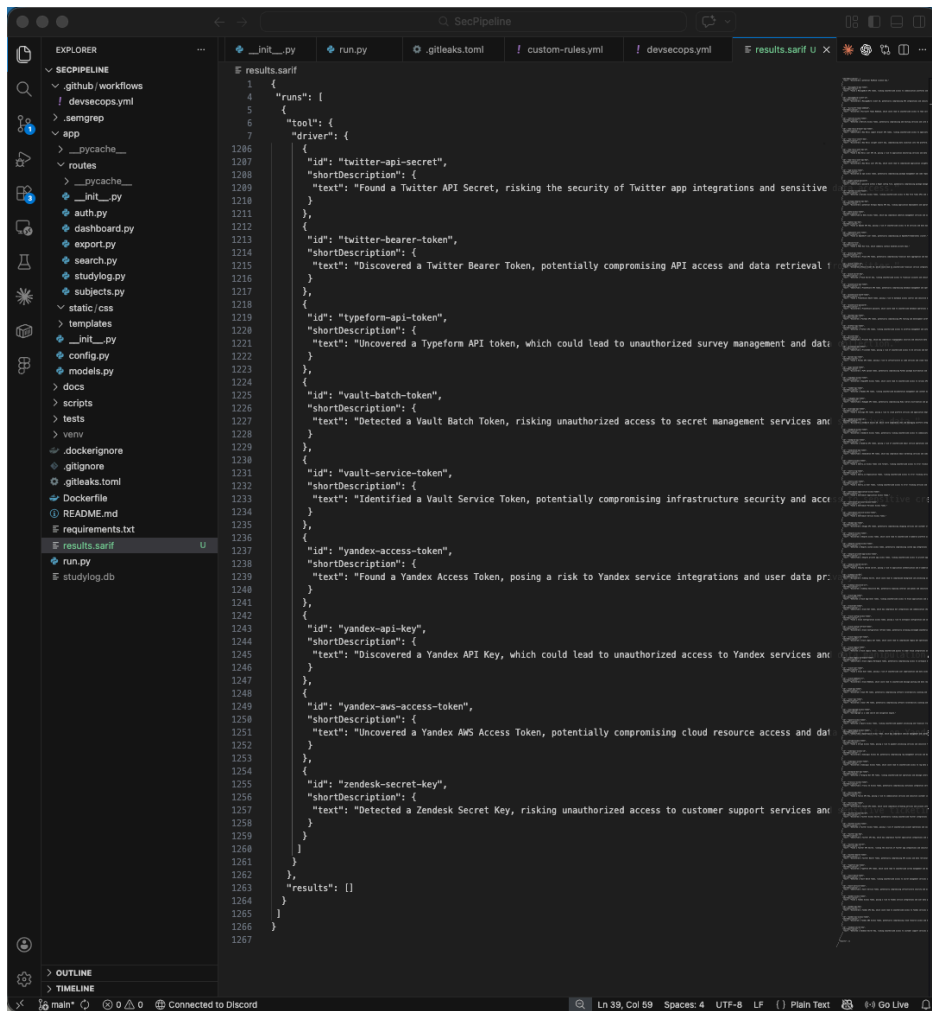
### 8.1 Gate 1 탐지

#### Gate 1 - Secret Scan (Gitleaks)



```
1 {
2 "$schema": "https://json.schemastore.org/sarif-2.1.0.json",
3 "version": "2.1.0",
4 "runs": [
5 {
6 "tool": {
7 "driver": {
8 "name": "gitleaks",
9 "semanticVersion": "v8.0.0",
10 "informationUri": "https://github.com/gitleaks/gitleaks",
11 "rules": [
12 {
13 "id": "1password-service-account-token",
14 "shortDescription": {
15 "text": "Uncovered a possible 1Password service account token, potentially compromising access to se"
16 },
17 },
18 {
19 "id": "adafruit-api-key",
20 "shortDescription": {
21 "text": "Identified a potential Adafruit API Key, which could lead to unauthorized access to Adafru:"
22 },
23 },
24 {
25 "id": "adobe-client-id",
26 "shortDescription": {
27 "text": "Detected a pattern that resembles an Adobe OAuth Web Client ID, posing a risk of compromi"
28 },
29 },
30 {
31 "id": "adobe-client-secret",
32 "shortDescription": {
33 "text": "Discovered a potential Adobe Client Secret, which, if exposed, could allow unauthorized Ad"
34 },
35 },
36 {
37 "id": "age-secret-key",
38 "shortDescription": {
39 "text": "Discovered a potential Age encryption tool secret key, risking data decryption and unautho"
40 },
41 },
42 {
43 "id": "airtable-api-key",
44 "shortDescription": {
45 "text": "Uncovered a possible Airtable API Key, potentially compromising database access and leadin"
46 },
47 },
48 {
49 "id": "algolia-api-key",
50 "shortDescription": {
51 "text": "Identified an Algolia API Key, which could result in unauthorized search operations and dat"
52 },
53 },
54 {
55 "id": "alibaba-access-key-id",
56 "shortDescription": {
57 "text": "Detected an Alibaba Cloud AccessKey ID, posing a risk of unauthorized cloud resource acces"
58 },
59 },
60 {
61 "id": "alibaba-secret-key",
62 "shortDescription": {
63 "text": "Discovered a potential Alibaba Cloud Secret Key, potentially allowing unauthorized operati"
64 },
65 },
66 {
67 "id": "asana-client-id",
68 "shortDescription": {
69 "text": "Discovered a potential Asana Client ID, risking unauthorized access to Asana projects and "
70 },
71 },
72]
73 }
74 }
75]
76 }
```

... 중간 많이 요약 ...



Gitleaks가 아무것도 탐지하지 못했음 !!!

"results": []

마지막을 보면 저렇게 나와있는데, 이것은 Gitleaks가 전체 소스 코드를 스캔했지만, 어떤 시크릿도 발견하지 못했다는 뜻임.

SARIF 파일을 보면 rules 배열에는 약 170개 이상의 탐지 규칙이 정의되어 있음. 이것은 Gitleaks가 "무엇을 찾을 수 있는가"를 나열한 것이지, "무엇을 찾았는가"가 아님. (클라우드 플랫폼 키, CI/CD 토큰, SaaS API 토큰, 암호화 관련, 일반 패턴을 찾을 수 있음.)

- 왜 탐지하지 못했을까?에 대한 생각....

app/config.py 에 삽입한 하드코딩된 시크릿은 `SECRET_KEY = "super-secret-key-12345"` 인데, Gitleaks의 generic-api-key 룰은 일반적으로 다음 패턴을 찾음.

```
(?i)(api[_-]?key|secret[_-]?key|access[_-]?token)\s*[:=]\s*["']([a-zA-Z0-9]{16,}["'])
```

- "super-secret-key-12345"는 하이픈을 포함하고 있어서 길이 조건을 충족하지 못했을수도 있고,,
- Gitleaks v8의 기본 generic-api-key 룰은 높은 엔트로피를 가진 문자열을 타겟으로 하는데, 하드코딩한 시크릿 키는 사람이 읽을 수 있는 단어 조합이어서 엔트로피가 낮음.
- 아니면 .gitleaks.toml의 allowlist 설정이 의도치 않게 해당 경로를 제외했을수도 있음.

- 해결 방안 탐색

- `SECRET_KEY = "f14sk_s3cr3t_8Kx9Qm2vPwLnR7jY5tZa"` 로 일부러 바꾼 뒤에 확인해보기!

바로 수정해서 push!

Gate 1 수정해보려고 하트코딩 시크릿 키 #3

Summary

Triggered via push 1 minute ago

Status: Failure

Total duration: 30s

Artifacts: 3

devsecops.yml

on: push

secret-scan 6s

dependency-scan 20s

sast 26s

dockerfile-lint 13s

build-and-scan 0s

secret-scan summary

**Gitleaks detected secrets**

| Rule ID  | Commit  | Secret URL           | Start Line | Author    | Date     | Email                  |
|----------|---------|----------------------|------------|-----------|----------|------------------------|
| generic- | 331a0ad | <a href="#">View</a> | 2          | jyeon0623 | 2026-03- | a01059003141@gmail.com |

secret-scan summary

**Gitleaks detected secrets**

| Rule ID         | Commit                  | Secret URL                  | Start Line | Author    | Date                 | Email                  |
|-----------------|-------------------------|-----------------------------|------------|-----------|----------------------|------------------------|
| generic-api-key | <a href="#">331a0ad</a> | <a href="#">View Secret</a> | 2          | jyeon0623 | 2026-03-25T11:59:21Z | a01059003141@gmail.com |

Job summary generated at run-time

**Annotations**

4 errors and 5 warnings

- dockerfile-lint: Dockerfile#L14**  
DL3825 warning: Use arguments JSON notation for CMD and ENTRYPOINT arguments
- dockerfile-lint: Dockerfile#L8**  
DL3845 warning: 'COPY' to a relative destination without 'WORKDIR' set.
- dockerfile-lint: Dockerfile#L6**  
DL3842 warning: Avoid use of cache directory with pip. Use 'pip install --no-cache-dir <package>'
- dockerfile-lint: Dockerfile#L5**  
DL3845 warning: 'COPY' to a relative destination without 'WORKDIR' set.
- secret-scan**  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected.  
[Show more](#)
- secret-scan**  
Leaks detected, see job summary for details
- dockerfile-lint**  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected.  
[Show more](#)
- dependency-scan**  
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected.  
[Show more](#)

Gate 1 수장체보러고 하드코딩 시

github.com/de1ayyy/SecPipeline/actions/runs/23539860366

**Summary**

All jobs

- secret-scan
- dependency-scan
- sast
- dockerfile-lint
- build-and-scan

Run details

- Usage
- Workflow file

**dockerfile-lint: Dockerfile#L8**  
DL3845 warning: 'COPY' to a relative destination without 'WORKDIR' set.

**dockerfile-lint: Dockerfile#L6**  
DL3842 warning: Avoid use of cache directory with pip. Use 'pip install --no-cache-dir <packages>'

**dockerfile-lint: Dockerfile#L5**  
DL3845 warning: 'COPY' to a relative destination without 'WORKDIR' set.

**secret-scan**  
Node.js 28 actions are deprecated. The following actions are running on Node.js 28 and may not work as expected.  
[Show more](#)

**secret-scan**  
Leaks detected, see job summary for details

**dockerfile-lint**  
Node.js 28 actions are deprecated. The following actions are running on Node.js 28 and may not work as expected.  
[Show more](#)

**dependency-scan**  
Node.js 28 actions are deprecated. The following actions are running on Node.js 28 and may not work as expected.  
[Show more](#)

**sast**  
Node.js 28 actions are deprecated. The following actions are running on Node.js 28 and may not work as expected.  
[Show more](#)

**Artifacts**  
Produced during runtime

| Name                   | Size    | Digest                     |
|------------------------|---------|----------------------------|
| dependency-scan        | 3.93 KB | sha256:8ae2f743a27e42b8... |
| gitleaks-results.sarif | 7.04 KB | sha256:c88b1ca2e3877232... |
| sast-results           | 4.48 KB | sha256:6fbb2f72f94cd76...  |

result.sarif (두 번째로 나온 결과물)

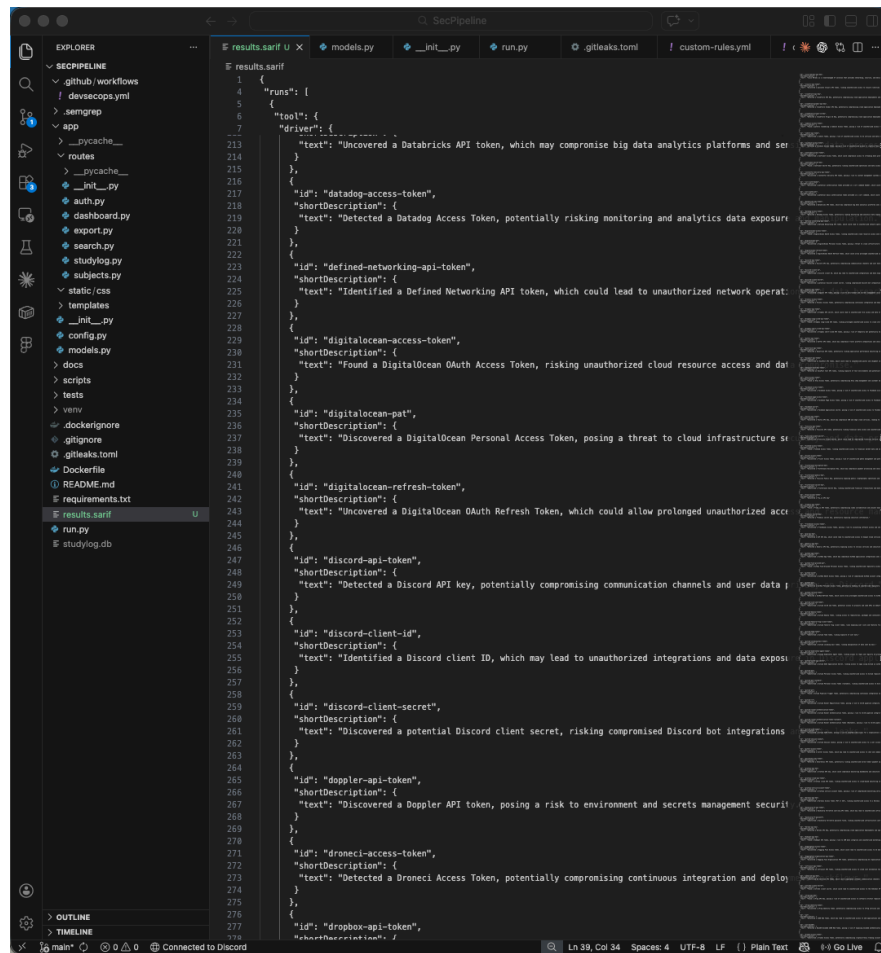
```

1 {
2 "schema": "https://json.schemastore.org/sarif-2.1.0.json",
3 "version": "2.1.0",
4 "runs": [
5 {
6 "tool": {
7 "driver": {
8 "name": "gitleaks",
9 "semanticVersion": "v8.0.0",
10 "informationUri": "https://github.com/gitleaks/gitleaks",
11 "rules": [
12 {
13 "id": "1password-service-account-token",
14 "shortDescription": {
15 "text": "Uncovered a possible 1Password service account token, potentially compromising access to s"
16 },
17 "text": "Uncovered a possible 1Password service account token, potentially compromising access to s"
18 },
19 {
20 "id": "adafruit-api-key",
21 "shortDescription": {
22 "text": "Identified a potential Adafruit API Key, which could lead to unauthorized access to Adafu"
23 },
24 "text": "Identified a potential Adafruit API Key, which could lead to unauthorized access to Adafu"
25 },
26 {
27 "id": "adobe-client-id",
28 "shortDescription": {
29 "text": "Detected a pattern that resembles an Adobe OAuth Web Client ID, posing a risk of compromi"
30 },
31 "text": "Detected a pattern that resembles an Adobe OAuth Web Client ID, posing a risk of compromi"
32 },
33 {
34 "id": "adobe-client-secret",
35 "shortDescription": {
36 "text": "Discovered a potential Adobe Client Secret, which, if exposed, could allow unauthorized Ad"
37 },
38 "text": "Discovered a potential Adobe Client Secret, which, if exposed, could allow unauthorized Ad"
39 },
40 {
41 "id": "age-secret-key",
42 "shortDescription": {
43 "text": "Discovered a potential Age encryption tool secret key, risking data decryption and unautho"
44 },
45 "text": "Discovered a potential Age encryption tool secret key, risking data decryption and unautho"
46 },
47 {
48 "id": "airtable-api-key",
49 "shortDescription": {
50 "text": "Uncovered a possible Airtable API Key, potentially compromising database access and leadin"
51 },
52 "text": "Uncovered a possible Airtable API Key, potentially compromising database access and leadin"
53 },
54 {
55 "id": "algolia-api-key",
56 "shortDescription": {
57 "text": "Identified an Algolia API Key, which could result in unauthorized search operations and da"
58 },
59 "text": "Identified an Algolia API Key, which could result in unauthorized search operations and da"
60 },
61 {
62 "id": "alibaba-access-key-id",
63 "shortDescription": {
64 "text": "Detected an Alibaba Cloud AccessKey ID, posing a risk of unauthorized cloud resource access"
65 },
66 "text": "Detected an Alibaba Cloud AccessKey ID, posing a risk of unauthorized cloud resource access"
67 },
68 {
69 "id": "alibaba-secret-key",
70 "shortDescription": {
71 "text": "Discovered a potential Alibaba Cloud Secret Key, potentially allowing unauthorized operati"
72 },
73 "text": "Discovered a potential Alibaba Cloud Secret Key, potentially allowing unauthorized operati"
74 },
75 {
76 "id": "asana-client-id",
77 "shortDescription": {
78 "text": "Discovered a potential Asana Client ID, risking unauthorized access to Asana projects and "
79 },
80 "text": "Discovered a potential Asana Client ID, risking unauthorized access to Asana projects and "
81 }
82]
83 }
84 }
85]
86 }
87 }

```

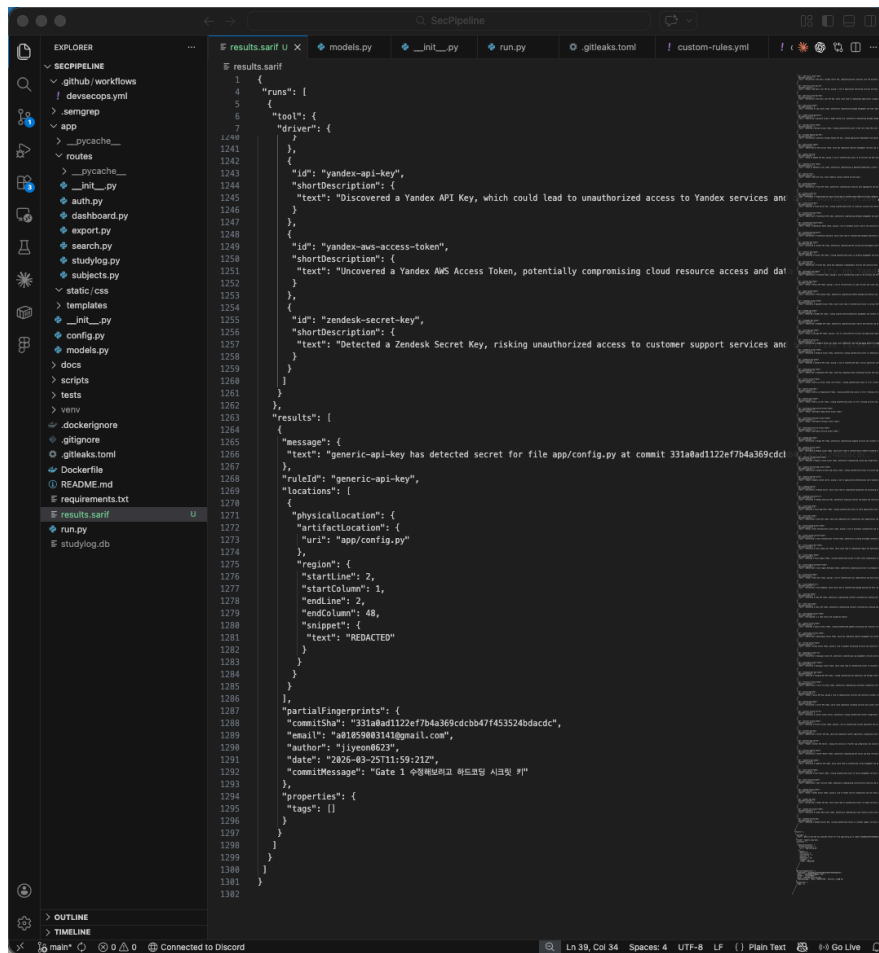






```
1 {
2 "runs": [
3 {
4 "tool": {
5 "driver": {
6 "text": "Uncovered a Databricks API token, which may compromise big data analytics platforms and ser
7 }
8 }
9 },
10 {
11 "id": "datadog-access-token",
12 "shortDescription": {
13 "text": "Detected a Datadog Access Token, potentially risking monitoring and analytics data exposur
14 },
15 {
16 "id": "defined-networking-api-token",
17 "shortDescription": {
18 "text": "Identified a Defined Networking API token, which could lead to unauthorized network operat
19 }
20 },
21 {
22 "id": "digitalocean-access-token",
23 "shortDescription": {
24 "text": "Found a DigitalOcean OAuth Access Token, risking unauthorized cloud resource access and dat
25 }
26 },
27 {
28 "id": "digitalocean-pat",
29 "shortDescription": {
30 "text": "Discovered a DigitalOcean Personal Access Token, posing a threat to cloud infrastructure se
31 }
32 },
33 {
34 "id": "digitalocean-refresh-token",
35 "shortDescription": {
36 "text": "Uncovered a DigitalOcean OAuth Refresh Token, which could allow prolonged unauthorized acc
37 }
38 },
39 {
40 "id": "discord-api-token",
41 "shortDescription": {
42 "text": "Detected a Discord API key, potentially compromising communication channels and user data
43 }
44 },
45 {
46 "id": "discord-client-id",
47 "shortDescription": {
48 "text": "Identified a Discord client ID, which may lead to unauthorized integrations and data exposi
49 }
50 },
51 {
52 "id": "discord-client-secret",
53 "shortDescription": {
54 "text": "Discovered a potential Discord client secret, risking compromised Discord bot integrations
55 }
56 },
57 {
58 "id": "doppler-api-token",
59 "shortDescription": {
60 "text": "Discovered a Doppler API token, posing a risk to environment and secrets management securi
61 }
62 },
63 {
64 "id": "droneci-access-token",
65 "shortDescription": {
66 "text": "Detected a Droneci Access Token, potentially compromising continuous integration and deplo
67 }
68 },
69 {
70 "id": "dropbox-api-token",
71 "shortDescription": {
72 "text": "Detected a Dropbox API token, potentially compromising file storage and user data access
73 }
74 }
75]
76 }
77 }
```

... 중간 요약 ...



## Gate 1 두 번째 시도 정리

### 1. 탐지 결과

```
"results": [
 {
 "message": {
 "text": "generic-api-key has detected secret for file app/config.py at com
mit 331a0ad..."
 },
 "ruleId": "generic-api-key",
 ...
 }
]
```

이전 Red Run에서는 "results" : []로 비어 있었지만, 이번에는 정확히 1건의 탐지 결과가 포함되어 있었음.

### 2. 탐지 세부 정보

- 매칭된 룰 : generic-api-key가 매칭되었음. Gitleaks에 내장된 약 170개 규칙 중 범용 API key / Secret을 탐지하는 규칙으로, 변수명에 secret, key, api, token 등의 키워드가 포함되고 할당된 값이 일정 수준 이상의 엔트로피를 가질 때 매칭됨.
- app/config.py 파일의 2번째 줄에서 `SECRET_KEY = "fl4sk_s3cr3t_8Kx9Qm2vPwLnR7jY5tZa"` 라인 전체에 해당한다고 알려 주었음. SARIF 출력에서 실제 시크릿 값은 `"text": "REDACTED"` 로 마스킹 처리되어 있는데, 이는 Gitleaks가 보안상 탐지된 시크릿을 결과 파일에 평문으로 노출하지 않도록 자동으로 처리한 것임.
- 커밋 메타데이터 : 커밋 SHA는 `331a0ad1122ef7b4a369cdcb47f453524bdacdc`, 작성자는 `jiyeon0623`, 이메일은 `a01059003141@gmail.com`, 커밋 시각은 `2026-03-25T11:59:21Z`, 커밋 메시지는 `"Gate 1 수정해보려고 하드코딩 시크릿"`

키"임. Gitleaks는 단순히 파일 내용뿐 아니라 Git 히스토리의 각 커밋을 순회하며 스캔하기에, 누가 언제 어떤 커밋에서 시크릿을 삽입했는지 정확하게 추적함.

3. 이전 실행과 비교해보았을 때... 어떤 한계점이 있었을까?

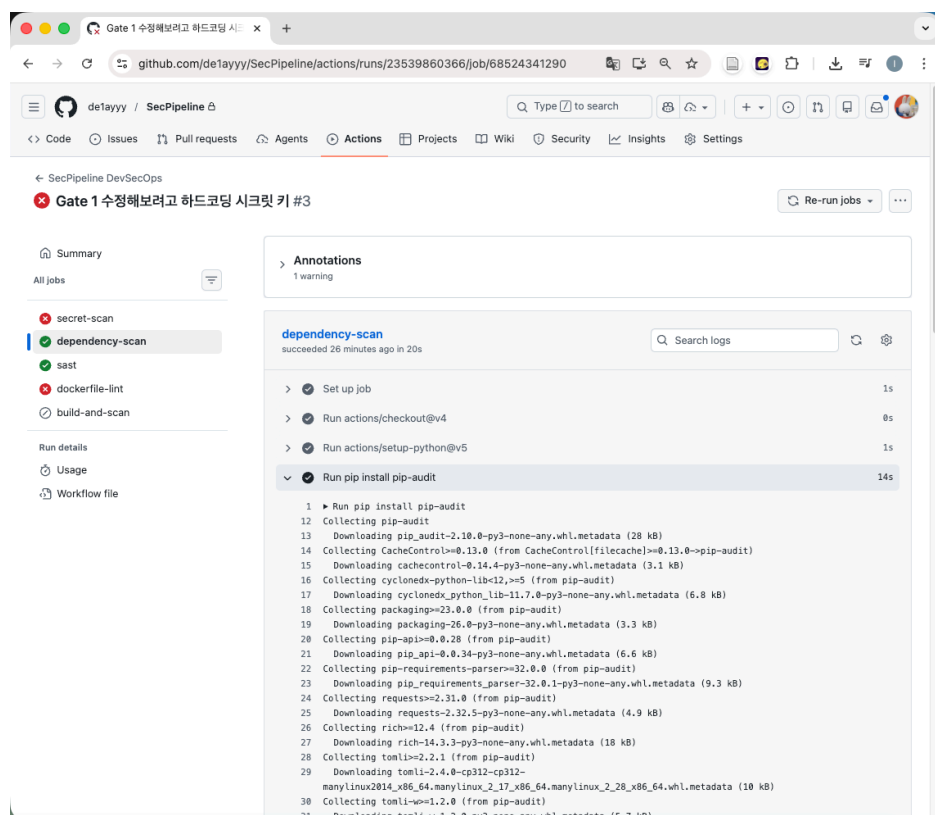
- 이전 Red Run에서 `SECRET_KEY = "super-secret-key-12345"` 는 Gitleaks에 의해 탐지되지 않았습니다. (False Negative)

이번 수정에서 값을 `"fl4sk_s3cr3t_8Kx9Qm2vPwLnR7jY5tZa"` 로 변경하였고, 이 문자열은 대소문자 혼합, 숫자 치환(leet speak), 연속 무작위 문자 배열로 인해 높은 엔트로피를 가집니다. 그 결과 `generic-api-key` 룰이 정상적으로 매칭되어 True Positive 탐지가 이루어졌습니다.

이 과정 자체가 보고서에서 중요한 논의 포인트를 제공합니다. 엔트로피 기반 시크릿 탐지 도구는 사람이 읽을 수 있는 (human-readable) 형태의 시크릿에 대해 사각지대를 가진다는 것을 알 수 있었습니다.

## 8.2 Gate 2 GitHub Actions 결과 분석

### 첫 시도 결과



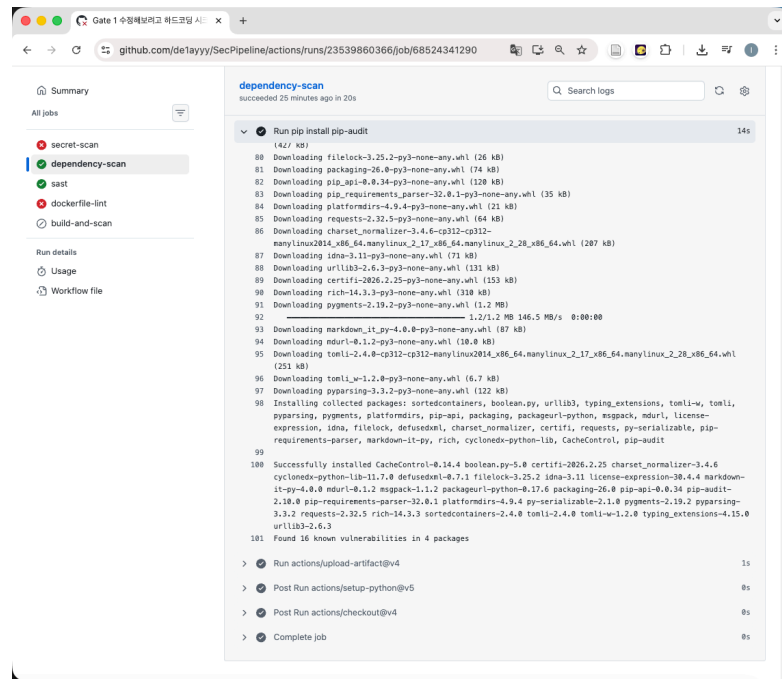
1. 스캔 대상 : requirements.txt

pip-audit가 스캔한 파일은 프로젝트 초기에 작성한 requirements.txt임.

```
flask==2.2.5
jinja2==3.1.2
werkzeug==2.2.3
unicorn==21.2.0
```

의도적으로 구 버전에 고정해놓았으며, VULN-02(CVE-2023-30861)를 포함하여 SCA 도구가 탐지할 수 있는 알려진 취약점을 확보하기 위함이었음.

2. GitHub Actions 로그에서 pip-audit 2.10.0이 설치된 후 스캔을 수행하고, 최종 출력이 한 줄



→ Found 16 Known vulnerabilities in 4 packages 라고 마지막 줄에 있음.

### 3. dep-scan.json 패키지별 상세 분석

push해서 나온 결과물이 dep-scan.json인데, 이를 하나씩 훑아볼 예정

- Flask 2.2.5 - CVE 1건

```

1 {"dependencies": [{"name": "flask", "version": "2.2.5", "vulns": [{"id":
"CVE-2026-27205", "fix_versions": ["3.1.3"], "aliases":
["GHSA-68rp-wp8r-4726"], "description": "When the `session` object is
accessed, Flask should set the `Vary: Cookie` header. This instructs caches
not to cache the response, as it may contain information specific to a
logged in user. This is handled in most cases, but some forms of access such
as the Python `in` operator were overlooked. The severity depends on the
application's use of the session, and the cache's behavior regarding
cookies. The risk depends on all these conditions being met. 1. The
application must be hosted behind a caching proxy that does not ignore
responses with cookies. 2. The application does not set a `Cache-Control`
header to indicate that a page is private or should not be cached. 3. The
application accesses the session in a way that does not access the values,
only the keys, and does not mutate the session."}]}, {"name": "jinja2",

```

→ CVE-2026-27205는 Flask의 session 객체 접근 시 Vary:Cookie 헤더 설정이 누락되는 문제임. 해당 프로젝트의 app/routes/auth.py에서 로그인 시 세션을 사용하고 있습니다. 이때 CVE-2026-27205의 설명에 따르면 session 객체에 in 연산자를 사용하는 것이 Vary: Cookie 헤더 누락의 원인 케이스 중 하나임. 즉, 푸시한 코드가 이 취약점의 정확한 트리거 조건에 해당함. 캐싱 프록시 뒤에서 운영할 경우, 로그인한 사용자 A의 응답이 캐시되어 사용자 B에게 노출될 수 있음.

→ 기대했던건 CVE-2023-30861이었는데, pip-audit가 더 최신의 2026 버전으로 잘 찾아주었음!

- Jinja2 3.1.2 - CVE 5건

```

only the keys, and does not mutate the session.}}}, {"name": "jinja2",
"version": "3.1.2", "vulns": [{"id": "CVE-2024-22195", "fix_versions":
["3.1.3"]}, {"aliases": ["GHSA-h5c8-rqwp-cp95"], "description": "The `xmlattr`
filter in affected versions of Jinja accepts keys containing spaces.
XML/HTML attributes cannot contain spaces, as each would then be interpreted
as a separate attribute. If an application accepts keys (as opposed to only
values) as user input, and renders these in pages that other users see as
well, an attacker could use this to inject other attributes and perform XSS.
Note that accepting keys as user input is not common or a particularly
intended use case of the `xmlattr` filter, and an application doing so
should already be verifying what keys are provided regardless of this
fix."}, {"id": "CVE-2024-34064", "fix_versions": ["3.1.4"], "aliases":
["GHSA-h75v-3vvj-5mfj"], "description": "The `xmlattr` filter in affected
versions of Jinja accepts keys containing non-attribute characters. XML/HTML
attributes cannot contain spaces, `/`, `>`, or `=`, as each would then be
interpreted as starting a separate attribute. If an application accepts keys
(as opposed to only values) as user input, and renders these in pages that
other users see as well, an attacker could use this to inject other
attributes and perform XSS. The fix for the previous GHSA-h5c8-rqwp-cp95
CVE-2024-22195 only addressed spaces but not other characters. Accepting
keys as user input is now explicitly considered an unintended use case of
the `xmlattr` filter, and code that does so without otherwise validating the
input should be flagged as insecure, regardless of Jinja version. Accepting
`_values` as user input continues to be safe."}, {"id": "CVE-2024-56326",
"fix_versions": ["3.1.5"], "aliases": ["GHSA-q2x7-8rv6-6q7h"],
"description": "An oversight in how the Jinja sandboxed environment detects
calls to `str.format` allows an attacker that controls the content of a
template to execute arbitrary Python code. To exploit the vulnerability, an
attacker needs to control the content of a template. Whether that is the
case depends on the type of application using Jinja. This vulnerability
impacts users of applications which execute untrusted templates. Jinja's
sandbox does catch calls to `str.format` and ensures they don't escape the
sandbox. However, it's possible to store a reference to a malicious string's
`format` method, then pass that to a filter that calls it. No such filters
are built-in to Jinja, but could be present through custom filters in an
application. After the fix, such indirect calls are also handled by the
sandbox."}, {"id": "CVE-2024-56201", "fix_versions": ["3.1.5"], "aliases":

```

CVE 요약하면,

```

{
 "name": "jinja2",
 "version": "3.1.2",
 "vulns": [
 {"id": "CVE-2024-22195", "fix_versions": ["3.1.3"]},
 {"id": "CVE-2024-34064", "fix_versions": ["3.1.4"]},
 {"id": "CVE-2024-56326", "fix_versions": ["3.1.5"]},
 {"id": "CVE-2024-56201", "fix_versions": ["3.1.5"]},
 {"id": "CVE-2025-27516", "fix_versions": ["3.1.6"]}
]
}

```

→ Jinja2는 Flask의 템플릿 엔진으로, 프로젝트 전체에서 사용됨. 특히 VULN-03b가 삽입된 app/routes/dashboard.py에서 직접적으로 관련됩니다.

```

app/routes/dashboard.py (VULN-03b: XSS/SSTI)
from flask import render_template_string
...
@dashboard_bp.route('/dashboard')
def dashboard():
 # ...
 html = f"""
 <h1>{session.get('username', 'User')}님의 대시보드</h1>
 <p>총 기록: {total_logs}건</p>

```

```
"""
return render_template_string(html) # ← SSTI 취약점
```

→ 여기서 `render_template_string`은 Jinja2 엔진을 직접 호출하는데, 5건의 CVE가 프로젝트에 미치는 영향을 개별적으로 분석하면 다음과 같음.

- **CVE-2024-22195 / CVE-2024-34064 (xmlattr 필터 XSS)**

`xmlattr` 필터가 공백 및 특수문자(`/`, `>`, `=`)를 포함한 키를 허용하여 속성 주입 → XSS가 가능한 문제

- **CVE-2024-56326 (샌드박스 우회 — `str.format` 간접 호출)**

Jinja2 샌드박스는 `str.format` 호출을 차단하지만, `format` 메서드의 참조를 변수에 저장한 뒤 커스텀 필터를 통해 간접 호출하면 샌드박스를 탈출할 수 있습니다. 프로젝트에서 `render_template_string`을 사용하고 있으므로, 공격자가 템플릿 내용을 제어할 수 있다면 취약점과 연쇄될 수 있습니다.

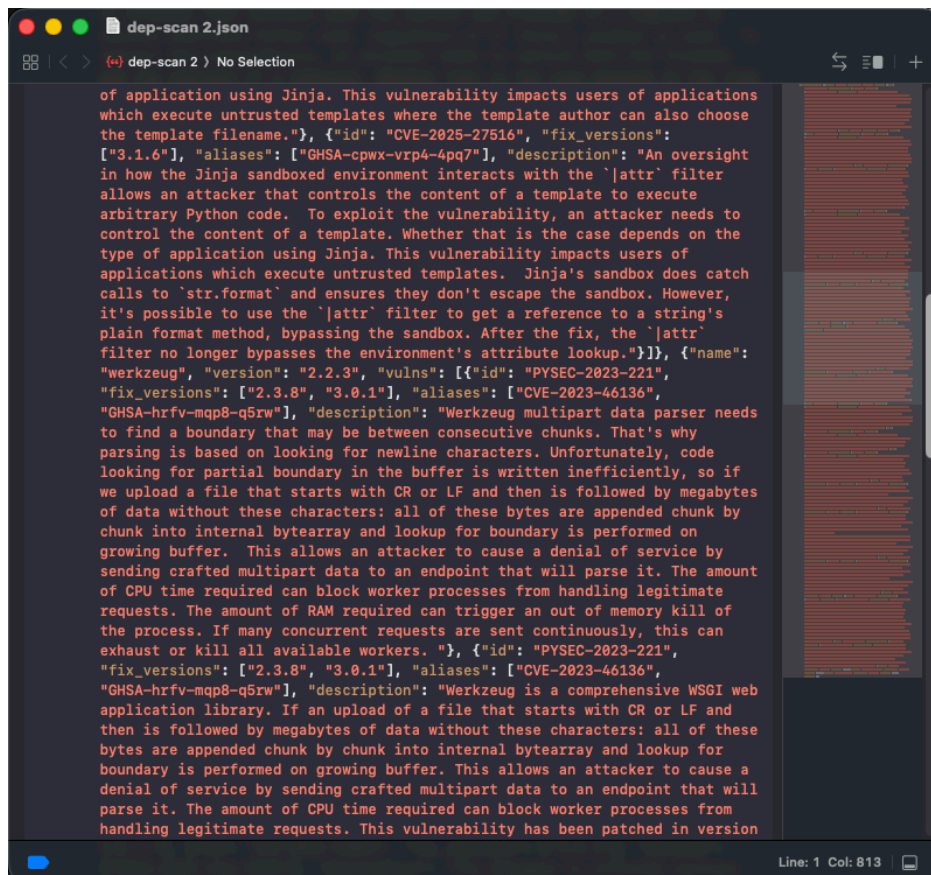
- **CVE-2024-56201 (파일명 + 내용 제어 시 임의 코드 실행)**

템플릿의 파일명과 내용을 모두 제어할 수 있으면 샌드박스와 무관하게 임의 코드 실행이 가능합니다. `render_template_string`은 파일명이 아닌 문자열을 직접 전달하므로 파일명 제어 조건은 충족되지 않지만, 공격 표면이 존재한다는 점에서 유의미합니다.

- **CVE-2025-27516 (`|attr` 필터를 통한 샌드박스 우회)**

`|attr` 필터로 `str.format`의 순수 참조를 얻어 샌드박스를 우회하는 최신 취약점입니다. 이것은 CVE-2024-56326의 수정 이후 발견된 또 다른 우회 경로로, Jinja2의 샌드박스가 반복적으로 우회되는 패턴을 보여줍니다.

- Werkzeug 2.2.3 - CVE 8건



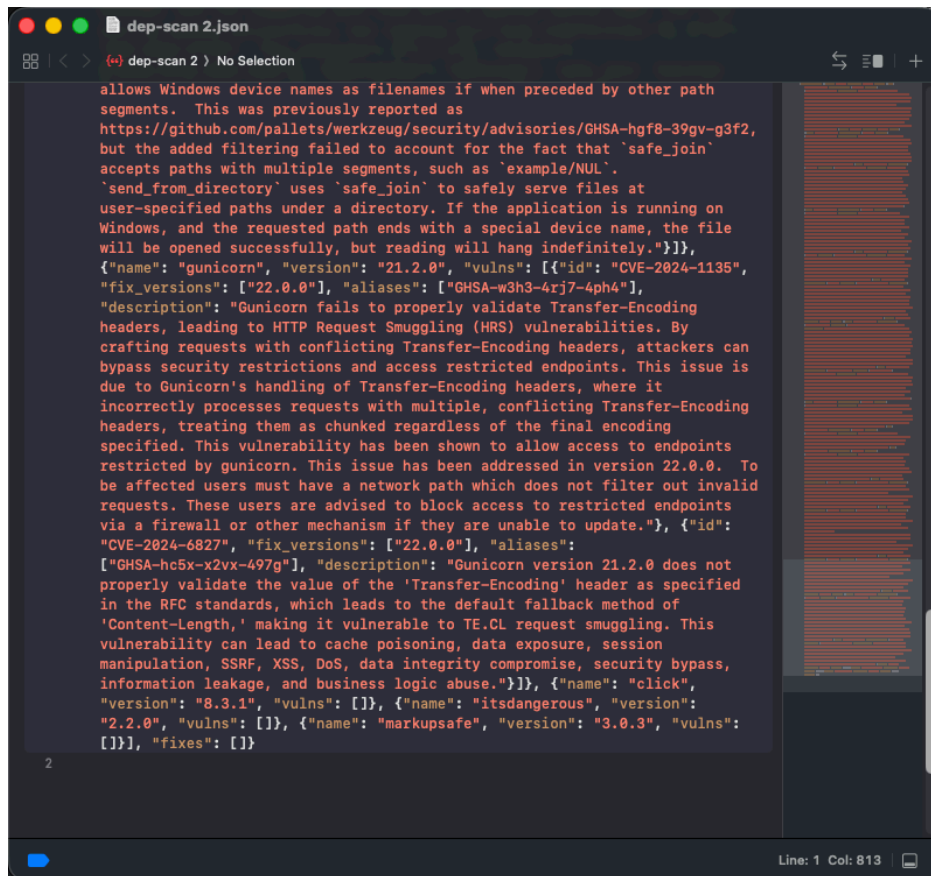


```
dep-scan 2.json
dep-scan 2 } No Selection

the process of many connections requests are sent simultaneously, this can
exhaust or kill all available workers. ", {"id": "PYSEC-2023-221",
"fix_versions": ["2.3.8", "3.0.1"], "aliases": ["CVE-2023-46136",
"GHSA-hrfv-mqp8-q5rw"], "description": "Werkzeug is a comprehensive WSGI web
application library. If an upload of a file that starts with CR or LF and
then is followed by megabytes of data without these characters: all of these
bytes are appended chunk by chunk into internal bytearray and lookup for
boundary is performed on growing buffer. This allows an attacker to cause a
denial of service by sending crafted multipart data to an endpoint that will
parse it. The amount of CPU time required can block worker processes from
handling legitimate requests. This vulnerability has been patched in version
3.0.1.", {"id": "CVE-2024-34069", "fix_versions": ["3.0.3"], "aliases":
["GHSA-2g68-c3qc-8985"], "description": "The debugger in affected versions
of Werkzeug can allow an attacker to execute code on a developer's machine
under some circumstances. This requires the attacker to get the developer to
interact with a domain and subdomain they control, and enter the debugger
PIN, but if they are successful it allows access to the debugger even if it
is only running on localhost. This also requires the attacker to guess a URL
in the developer's application that will trigger the debugger.", {"id":
"CVE-2024-49766", "fix_versions": ["3.0.6"], "aliases":
["GHSA-f9vj-2wh5-fj8j"], "description": "On Python < 3.11 on Windows,
'os.path.isabs()' does not catch UNC paths like '//server/share'. Werkzeug's
'safe_join()' relies on this check, and so can produce a path that is not
safe, potentially allowing unintended access to data. Applications using
Python >= 3.11, or not using Windows, are not vulnerable.", {"id":
"CVE-2024-49767", "fix_versions": ["3.0.6"], "aliases":
["GHSA-q34m-jh98-gwm2"], "description": "Applications using Werkzeug to
parse 'multipart/form-data' requests are vulnerable to resource exhaustion.
A specially crafted form body can bypass the 'Request.max_form_memory_size'
setting. The 'Request.max_content_length' setting, as well as resource
limits provided by deployment software and platforms, are also available to
limit the resources used during a request. This vulnerability does not
affect those settings. All three types of limits should be considered and
set appropriately when deploying an application.", {"id": "CVE-2025-66221",
"fix_versions": ["3.1.4"], "aliases": ["GHSA-hgfh-39gv-g3f2"],
"description": "Werkzeug's 'safe_join' function allows path segments with
Windows device names. On Windows, there are special device names such as
'CON', 'AUX', etc that are implicitly present and readable in every
directory. 'send_from_directory' uses 'safe_join' to safely serve files at
user-specified paths under a directory. If the application is running on
Windows, and the requested path ends with a special device name, the file
will be opened successfully, but reading will hang indefinitely.", {"id":
"CVE-2026-21860", "fix_versions": ["3.1.5"], "aliases":
["GHSA-87hc-h4r6-73f7"], "description": "Werkzeug's 'safe_join' function
allows path segments with Windows device names that have file extensions or
trailing spaces. On Windows, there are special device names such as 'CON',
'AUX', etc that are implicitly present and readable in every directory.
Windows still accepts them with any file extension, such as 'CON.txt', or
trailing spaces such as 'CON '. This was previously reported as
https://github.com/pallets/werkzeug/security/advisories/GHSA-hgfh-39gv-g3f2,
but the fix failed to account for compound extensions such as 'CON.txt.html'
or trailing spaces. It also missed some additional special names.
'send_from_directory' uses 'safe_join' to safely serve files at
user-specified paths under a directory. If the application is running on
Windows, and the requested path ends with a special device name, the file
will be opened successfully, but reading will hang indefinitely.", {"id":
"CVE-2026-27199", "fix_versions": ["3.1.6"], "aliases":
["GHSA-29vq-49wx-vm6x"], "description": "Werkzeug's 'safe_join' function
allows Windows device names as filenames if when preceded by other path
segments. This was previously reported as
https://github.com/pallets/werkzeug/security/advisories/GHSA-hgfh-39gv-g3f2,
but the added filtering failed to account for the fact that 'safe_join'
accepts paths with multiple segments, such as 'example/NUL'.
'send_from_directory' uses 'safe_join' to safely serve files at
user-specified paths under a directory. If the application is running on
Windows, and the requested path ends with a special device name, the file
will be opened successfully, but reading will hang indefinitely."}],
{"name": "unicorn", "version": "21.2.0", "vulns": [{"id": "CVE-2024-1135",
"fix_versions": ["22.0.0"], "aliases": ["GHSA-w3h3-4rj7-4ph4"],
"description": "Gunicorn fails to properly validate Transfer-Encoding
```

```
dep-scan 2.json
dep-scan 2 } No Selection

affect those settings. All three types of limits should be considered and
set appropriately when deploying an application.", {"id": "CVE-2025-66221",
"fix_versions": ["3.1.4"], "aliases": ["GHSA-hgfh-39gv-g3f2"],
"description": "Werkzeug's 'safe_join' function allows path segments with
Windows device names. On Windows, there are special device names such as
'CON', 'AUX', etc that are implicitly present and readable in every
directory. 'send_from_directory' uses 'safe_join' to safely serve files at
user-specified paths under a directory. If the application is running on
Windows, and the requested path ends with a special device name, the file
will be opened successfully, but reading will hang indefinitely.", {"id":
"CVE-2026-21860", "fix_versions": ["3.1.5"], "aliases":
["GHSA-87hc-h4r6-73f7"], "description": "Werkzeug's 'safe_join' function
allows path segments with Windows device names that have file extensions or
trailing spaces. On Windows, there are special device names such as 'CON',
'AUX', etc that are implicitly present and readable in every directory.
Windows still accepts them with any file extension, such as 'CON.txt', or
trailing spaces such as 'CON '. This was previously reported as
https://github.com/pallets/werkzeug/security/advisories/GHSA-hgfh-39gv-g3f2,
but the fix failed to account for compound extensions such as 'CON.txt.html'
or trailing spaces. It also missed some additional special names.
'send_from_directory' uses 'safe_join' to safely serve files at
user-specified paths under a directory. If the application is running on
Windows, and the requested path ends with a special device name, the file
will be opened successfully, but reading will hang indefinitely.", {"id":
"CVE-2026-27199", "fix_versions": ["3.1.6"], "aliases":
["GHSA-29vq-49wx-vm6x"], "description": "Werkzeug's 'safe_join' function
allows Windows device names as filenames if when preceded by other path
segments. This was previously reported as
https://github.com/pallets/werkzeug/security/advisories/GHSA-hgfh-39gv-g3f2,
but the added filtering failed to account for the fact that 'safe_join'
accepts paths with multiple segments, such as 'example/NUL'.
'send_from_directory' uses 'safe_join' to safely serve files at
user-specified paths under a directory. If the application is running on
Windows, and the requested path ends with a special device name, the file
will be opened successfully, but reading will hang indefinitely."}],
{"name": "unicorn", "version": "21.2.0", "vulns": [{"id": "CVE-2024-1135",
"fix_versions": ["22.0.0"], "aliases": ["GHSA-w3h3-4rj7-4ph4"],
"description": "Gunicorn fails to properly validate Transfer-Encoding
```



→ Werkzeug은 Flask의 WSGI 핵심 라이브러리로, HTTP 요청 파싱, 라우팅, 디버거 등 Flask의 거의 모든 저수준 기능을 담당합니다.

총 8개의 CVE가 발견되었는데,

- **PYSEC-2023-221 / CVE-2023-46136 (multipart DoS)**

multipart 데이터 파서의 경계(boundary) 탐색이 비효율적으로 구현되어 있습니다. CR 또는 LF로 시작하고 이후 수 MB의 데이터가 이어지는 파일을 업로드하면, 내부 bytearray에 체크가 계속 추가되면서 경계 검색이 점점 커지는 버퍼에 대해 수행됩니다. 프로젝트의 studylog.py에서 POST 요청을 통해 학습 기록을 생성하는 엔드포인트가 있으므로, multipart 폼 데이터를 처리하는 경로가 존재합니다.

공격자가 엔드포인트에 조작된 multipart 데이터를 전송하면 워커 프로세스의 CPU/메모리를 소진시켜 서비스 거부를 유발할 수 있습니다.

- **CVE-2024-34069 (디버거 원격 코드 실행)**

프로젝트에서 가장 심각한 취약점으로, debug = True로 실행하면 Werkzeug 대화형 디버거가 활성화되는데, 공격자가 개발자를 자신이 제어하는 도메인/서브도메인으로 유도하고 디버거 PIN을 입력하게 만들면, localhost에서만 실행 중인 디버거에도 접근하여 임의의 Python 코드를 실행할 수 있음.

Flask 서버 시작 시 다음 메시지가 출력되는데, 이 PIN이 탈취되면 서버의 Python 인터프리터에서 어떤 코드든 실행 가능함. 따라서 실무에서 debug=True를 프로덕션에 배포하는 것은 절대 안됨.

```
* Debugger is active!
* Debugger PIN: 114-376-152
```

- **CVE-2024-49766 (safe\_join UNC 경로 미검증)**

Windows + Python < 3.11 환경에서 os.path.isabs() 가 //server/share 같은 UNC 경로를 절대 경로로 인식하지 못해 safe\_join() 이 의도하지 않은 파일 접근을 허용합니다. 우리 프로젝트는 macOS/Linux 기반이므로 직접적 영향은 없지만, SCA 도구는 플랫폼 무관하게 보고합니다.

- **CVE-2024-49767 (multipart max\_form\_memory\_size 우회)**



◦ CVE-2025-66221 → CVE-2026-21860 → CVE-2026-27199 (safe\_join Windows 장치명 연쇄 취약점)

이 세 CVE는 보고서에서 특별히 주목할 가치가 있는 연쇄 패치 실패 사례입니다.

CVE-2025-66221: safe\_join이 "CON", "AUX" 등 Windows 장치명을 허용  
→ 수정: 장치명 필터 추가

CVE-2026-21860: "CON.txt.html", "CON " (후행 공백) 등 복합 확장자 우회  
→ 수정: 확장자/공백 처리 추가

CVE-2026-27199: "example/NUL" 등 다중 세그먼트 경로에서 우회  
→ 수정: 모든 경로 세그먼트에 대해 필터링

• Unicorn 21.2.0 - CVE 2건

```
{
 "name": "unicorn",
 "version": "21.2.0",
 "vulns": [
 {"id": "CVE-2024-1135", "fix_versions": ["22.0.0"]},
 {"id": "CVE-2024-6827", "fix_versions": ["22.0.0"]}
]
}
```

Unicorn은 프로덕션 WSGI 서버로, 현재 Dockerfile에서 사용 예정.

◦ CVE-2024-1135 (Transfer-Encoding HRS)

Unicorn이 상충하는 Transfer-Encoding 헤더를 가진 요청을 부적절하게 처리하여, 최종 인코딩과 무관하게 chunked로 취급합니다. 공격 시나리오는 다음과 같음.

```
POST /admin HTTP/1.1
Host: target.com
Transfer-Encoding: chunked
Transfer-Encoding: identity

0

GET /restricted HTTP/1.1
Host: target.com
```

Unicorn 앞의 리버스 프록시는 두 번째 Transfer-Encoding: identity 를 보고 본문 전체를 하나의 요청으로 전달하지 만, Unicorn은 첫 번째 Transfer-Encoding: chunked 를 보고 0\r\n\r\n 이후의 내용을 별도의 두 번째 요청으로 해석 합니다. 이를 통해 접근 제한된 /restricted 엔드포인트에 도달할 수 있습니다.

◦ CVE-2024-6827 (TE.CL Request Smuggling 변형)

CVE-2024-1135와 유사하지만 Transfer-Encoding 검증 실패 시 Content-Length 로 폴백하는 동작을 악용하는 TE.CL 유형입니다. 캐시 포이즈닝, 세션 조작, SSRF, XSS, DoS 등 광범위한 공격이 가능합니다.

## Gate 2는 정확히 작동했는가?

- **True Positive:** 4개 패키지에서 16건의 CVE를 모두 정확하게 탐지했습니다. 의도적으로 삽입한 VULN-02(Flask 세션 취약점 계열)를 포함하여, 예상을 크게 초과하는 15건의 추가 취약점이 발견되었습니다.
- **True Negative:** click, itsdangerous, markupsafe에서 취약점 없음을 올바르게 판정했습니다.
- **False Positive:** 0건입니다. 모든 탐지 결과가 NVD/GHSA/PyPI Advisory에 등록된 실제 CVE에 근거합니다.
- **False Negative:** 확인 가능한 범위 내에서 누락된 알려진 취약점은 없습니다.

→ 이 결과로 보고서에 시사하는 바는 크게 두 가지입니다. 첫째, 오래된 패키지를 사용하면 단일 취약점이 아닌 시간에 비례하여 취약점이 누적된다는 점.

둘째, Green Run에서 flask를 3.13 버전 이상, jinja2를 3.1.6, werkzeug를 3.1.6, gunicorn을 22.0.0이상으로 업그레이드하면 16건 → 0건으로 감소로 정량적으로 제시 가능합니다!

### 8.3 Gate 3 - SAST (Semgrep + Bandit)

#### 실행 요약

The screenshot shows a GitHub Actions workflow run for the repository 'de1ayyy/SecPi...' with the workflow name 'Gate 1 수정해보려고 하드코딩 시크릿 키 #3'. The status is 'Failure' and the total duration is '30s'. The workflow was triggered via a push 18 hours ago by user 'de1ayyy' on the 'main' branch with commit '331a0ad'.

**Summary**

- secret-scan (Failure)
- dependency-scan (Success)
- sast (Success)
- dockerfile-lint (Failure)
- build-and-scan (Success)

**Run details**

- Usage
- Workflow file

**devsecops.yml**

on: push

secret-scan (6s) → build-and-scan (0s)

dependency-scan (20s)

sast (26s)

dockerfile-lint (13s)

**secret-scan summary**

Branches · de1ayyy/jnu-pipe- x Gate 1 수정해보려고 하드코딩 시크릿 키 #3

github.com/de1ayyy/SecPi... 업데이트 완료

de1ayyy / SecPipeline

Code Issues Pull requests Agents Actions Projects Wiki Security Insights Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

← SecPipeline DevSecOps

Gate 1 수정해보려고 하드코딩 시크릿 키 #3 Re-run jobs

Summary

All jobs

- secret-scan
- dependency-scan
- sast
- dockerfile-lint
- build-and-scan

Run details

- Usage
- Workflow file

Annotations

1 warning

Complete job

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and.. [Show more](#)

sast

succeeded 18 hours ago in 26s Search logs

Set up job 1s

- Current runner version: '2.332.0'
- Runner Image Provisioner
- Operating System
- Runner Image
- GITHUB\_TOKEN Permissions
- Secret source: Actions
- Prepare workflow directory
- Prepare all required actions
- Getting action download info
- Download action repository 'actions/checkout@v4' (SHA:34e114876b0b11c390a56381ad16bd13914f8d5)
- Download action repository 'actions/setup-python@v5' (SHA:a26af69be951a213d495a4c3e4e4022e16d87065)
- Download action repository 'actions/upload-artifact@v4' (SHA:ea165f8d65b6e75b540449e92b4886f43607fa02)
- Complete job name: sast

Run actions/checkout@v4 1s

Branches · de1ayyy/jnu-pipe- x Gate 1 수정해보려고 하드코딩 시 x +

github.com/de1ayyy/SecPi... 업데이트 완료

Summary

All jobs

- secret-scan
- dependency-scan
- sast**
- dockerfile-lint
- build-and-scan

Run details

- Usage
- Workflow file

**sast**  
succeeded 18 hours ago in 26s

Search logs

Run actions/checkout@v4 1s

```

1 ▶ Run actions/checkout@v4
16 Syncing repository: de1ayyy/SecPipeline
17 ▶ Getting Git version info
21 Temporarily overriding HOME='/home/runner/work/_temp/71cf7864-4d2b-406a-91eb-45b6ce8c2463' before making global git config changes
22 Adding repository directory to the temporary git global config as a safe directory
23 /usr/bin/git config --global --add safe.directory /home/runner/work/SecPipeline/SecPipeline
24 Deleting the contents of '/home/runner/work/SecPipeline/SecPipeline'
25 ▶ Initializing the repository
42 ▶ Disabling automatic garbage collection
44 ▶ Setting up auth
52 ▶ Fetching the repository
56 ▶ Determining the checkout info
57 /usr/bin/git sparse-checkout disable
58 /usr/bin/git config --local --unset-all extensions.worktreeConfig
59 ▶ Checking out the ref
63 /usr/bin/git log -1 --format=%H
64 331a0ad1122ef7b4a369cdcb47f453524bdacdc

```

Run actions/setup-python@v5 0s

```

1 ▶ Run actions/setup-python@v5
9 ▶ Installed versions

```

Run pip install semgrep bandit 20s

```

1 ▶ Run pip install semgrep bandit
13 Collecting semgrep
14 Downloading semgrep-1.156.0-cp310.cp311.cp312.cp313.cp314.py310.py311.py312.py313.py314-none-manylinux2014_x86_64.whl.metadata (22 kB)
15 Collecting bandit
16 Downloading bandit-1.9.4-py3-none-any.whl.metadata (7.1 kB)
17 Collecting attrs>=21.3 (from semgrep)
18 Downloading attrs-26.1.0-py3-none-any.whl.metadata (8.8 kB)
19 Collecting boltons>=21.0 (from semgrep)
20 Downloading boltons-21.0.0-py2.py3-none-any.whl.metadata (1.5 kB)
21 Collecting click-option-group>=0.5 (from semgrep)
22 Downloading click_option_group-0.5.9-py3-none-any.whl.metadata (5.8 kB)
23 Collecting click>=8.1.8 (from semgrep)
24 Downloading click-8.1.8-py3-none-any.whl.metadata (2.3 kB)
25 Collecting colorama>=0.4.0 (from semgrep)
26 Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
27 Collecting exceptiongroup>=1.2.0 (from semgrep)
28 Downloading exceptiongroup-1.2.2-py3-none-any.whl.metadata (6.6 kB)
29 Collecting glom>=23.3 (from semgrep)
30 Downloading glom-25.12.0-py3-none-any.whl.metadata (5.2 kB)
31 Collecting jsonschema>=4.25.1 (from semgrep)
32 Downloading jsonschema-4.25.1-py3-none-any.whl.metadata (7.6 kB)

```

Branches · de1ayyy/jnu-pipe- x Gate 1 수정해보려고 하드코딩 시...

github.com/de1ayyy/SecPi... 업데이트 완료

Summary

All jobs

- secret-scan
- dependency-scan
- sast**
- dockerfile-lint
- build-and-scan

Run details

- Usage
- Workflow file

**sast**  
succeeded 18 hours ago in 26s

Search logs

Run pip install semgrep bandit 20s

pydantic-2.12.5 pydantic-core-2.41.5 pydantic-settings-2.13.1 pygments-2.19.2  
pyjwt-2.12.1 python-dotenv-1.2.2 python-multipart-0.0.22 referencing-0.37.0  
requests-2.32.5 rich-14.3.3 rps-py-0.30.0 ruamel.yaml-0.19.1 ruamel.yaml.clib-  
0.2.14 semantic-version-2.10.0 semgrep-1.156.0 sse-starlette-3.3.3 starlette-1.0.0  
stevedore-5.7.0 toml-2.0.2 typing-extensions-4.15.0 typing-inspection-0.4.2  
urllib3-2.6.3 uvicorn-0.42.0 wcmatch-8.5.2 wrapt-1.17.3 zipp-3.23.0

268  
269  
270  
271 Scan Status  
272  
273 Scanning 10 files tracked by git with 1064 Code rules:  
274  
275

| Language    | Rules | Files | Origin    | Rules |
|-------------|-------|-------|-----------|-------|
| python      | 244   | 10    | Community | 1063  |
| <multilang> | 47    | 10    | Custom    | 1     |

276  
277  
278  
279  
280  
281  
282  
283 Scan Summary  
284  
285 Scan completed successfully.  
286 • Findings: 16 (16 blocking)  
287 • Rules run: 291  
288 • Targets scanned: 10  
289 • Parsed lines: ~100.0%  
290 • Scan was limited to files tracked by git  
291 • For a detailed list of skipped files and lines, run semgrep with the --verbose  
flag  
292 Ran 291 rules on 10 files: 16 findings.  
293 [main] INFO profile include tests: None  
294 [main] INFO profile exclude tests: None  
295 [main] INFO cli include tests: None  
296 [main] INFO cli exclude tests: None  
297 [json] INFO JSON output written to file: bandit.json

Run actions/upload-artifact@v4 1s

1 Run actions/upload-artifact@v4  
18 Multiple search paths detected. Calculating the least common ancestor of all paths  
19 The least common ancestor is /home/runner/work/SecPipeline/SecPipeline. This will  
be the root directory of the artifact  
20 With the provided path, there will be 2 files uploaded  
21 Artifact name is valid!  
22 Root directory input is valid!  
23 Beginning upload of artifact content to blob storage  
24 Uploaded bytes 4588  
25 Finished uploading artifact content to blob storage!  
26 SHA256 digest of uploaded artifact zip is

Branches · de1ayyy/jnu-pipe- x Gate 1 수정해보려고 하드코딩 시 x +

← → ↺ github.com/de1ayyy/SecPi... 🔍 📄 📁 📌 📄 📄 📄 업데이트 완료 :

Summary

All jobs

- secret-scan
- dependency-scan
- sast**
- dockerfile-lint
- build-and-scan

Run details

- Usage
- Workflow file

**sast**

succeeded 18 hours ago in 26s 🔍 Search logs 🔄 ⚙️

- Run actions/upload-artifact@v4 1s

```

1 ▶ run actions/upload-artifact@v4
18 Multiple search paths detected. Calculating the least common ancestor of all paths
19 The least common ancestor is /home/runner/work/SecPipeline/SecPipeline. This will
 be the root directory of the artifact
20 With the provided path, there will be 2 files uploaded
21 Artifact name is valid!
22 Root directory input is valid!
23 Beginning upload of artifact content to blob storage
24 Uploaded bytes 4588
25 Finished uploading artifact content to blob storage!
26 SHA256 digest of uploaded artifact zip is
 6fbb2f72f04cd765deb4f0aac7e06dbd6c4e408c439f5380438ff6d4c002c6d
27 Finalizing artifact upload
28 Artifact sast-results.zip successfully finalized. Artifact ID 6100559408
29 Artifact sast-results has been successfully uploaded! Final size is 4588 bytes.
 Artifact ID is 6100559408
30 Artifact download URL:
 https://github.com/de1ayyy/SecPipeline/actions/runs/23539860366/artifacts/6100559408

```
- Post Run actions/setup-python@v5 0s

```

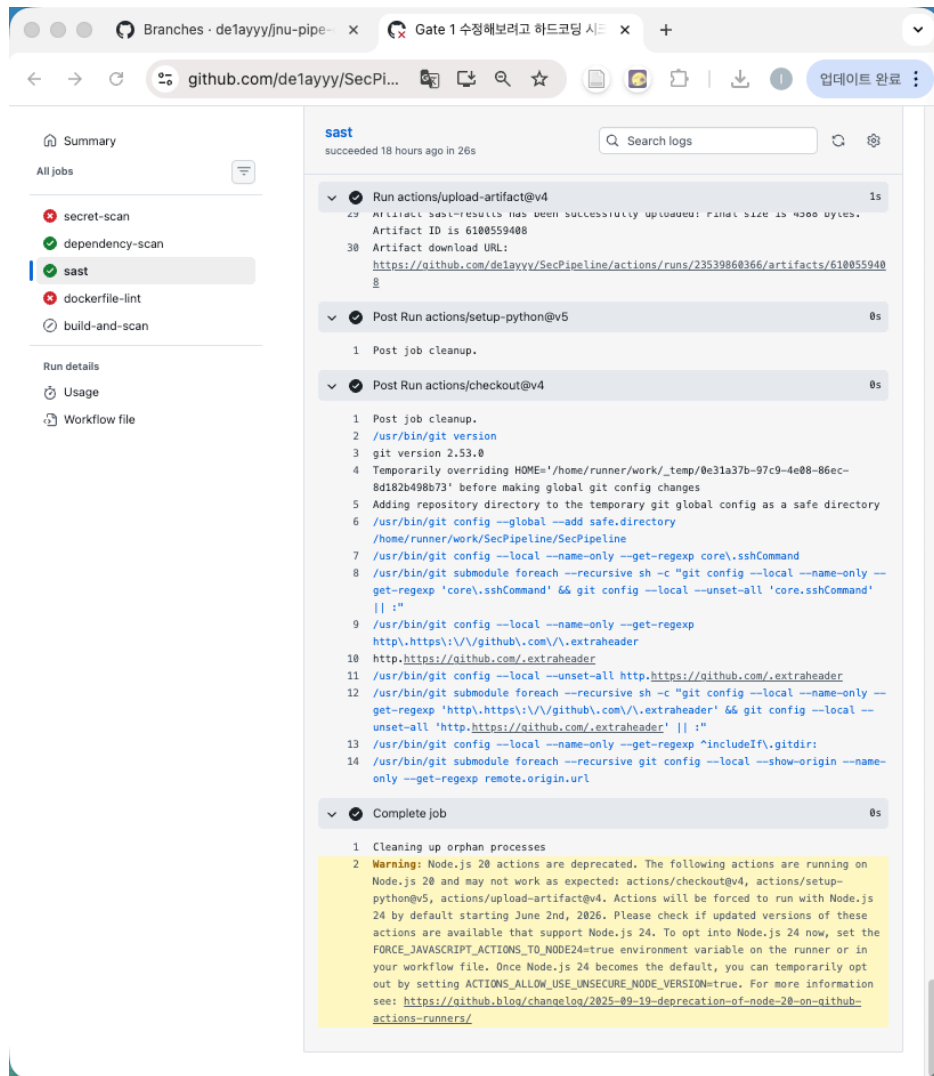
1 Post job cleanup.

```
- Post Run actions/checkout@v4 0s

```

1 Post job cleanup.
2 /usr/bin/git version
3 git version 2.53.0
4 Temporarily overriding HOME='/home/runner/work/_temp/0e31a37b-97c9-4e08-86ec-8d182b498b73' before making global git config changes
5 Adding repository directory to the temporary git global config as a safe directory
6 /usr/bin/git config --global --add safe.directory
 /home/runner/work/SecPipeline/SecPipeline
7 /usr/bin/git config --local --name-only --get-regexp core.sshCommand
8 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --
 get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand'
 || :"
9 /usr/bin/git config --local --name-only --get-regexp
 http.https://github.com/.extraheader
10 http.https://github.com/.extraheader
11 /usr/bin/git config --local --unset-all http.https://github.com/.extraheader
12 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --
 get-regexp 'http.https://github.com/.extraheader' && git config --local --
 unset-all 'http.https://github.com/.extraheader' || :"
13 /usr/bin/git config --local --name-only --get-regexp ^includeIf\.\gitdir:
14 /usr/bin/git submodule foreach --recursive git config --local --show-origin --name-
 only --get-regexp remote.origin.url

```
- Complete job 0s



→ Gate 2와 마찬가지로 continue-on-error:true로 설정되어 있어 취약점이 발견되어도 Job 자체는 통과합니다. 두 도구의 실행 결과는 다음과 같습니다.

- Bandit : 총 459줄의 Python 코드를 스캔하여 2건의 취약점을 탐지했습니다. 1건은 SEVERITY.LOW / CONFIDENCE.MEDIUM, 1건은 SEVERITY.MEDIUM / CONFIDENCE.LOW입니다.
- Semgrep : 10개 Python 파일을 스캔하여 16건의 경고를 탐지했습니다. 이 중 ERROR 등급 3건, WARNING 등급 13건입니다.

## Bandit 결과 상세 분석

### 1. 탐지 결과 (GitHub Actions CI/CD 결과로 나온 bandit.json 분석 결과)

- 탐지 결과 #1 : VULN-01 하드코딩 시크릿

```

},
"results": [
 {
 "code": "1 import os\n2 SECRET_KEY = \"fl4sk_s3cr3t_8Kx9Qm2vPwLnR7jY5tZa\" # VULN-01: \\ud558\\ub4dc\\uc",
 "col_offset": 13,
 "end_col_offset": 48,
 "filename": "app/config.py",
 "issue_confidence": "MEDIUM",
 "issue_cwe": {
 "id": 259,
 "link": "https://cwe.mitre.org/data/definitions/259.html"
 },
 "issue_severity": "LOW",
 "issue_text": "Possible hardcoded password: 'fl4sk_s3cr3t_8Kx9Qm2vPwLnR7jY5tZa'",
 "line_number": 2,
 "line_range": [
 2
]
 }
]

```

→ Bandit의 B105 (hardcoded\_password\_string) 룰이 변수명에 SECRET, PASSWORD, KEY 등의 키워드가 포함되고 문자열 리터럴이 할당된 패턴을 탐지했습니다. CWE-259(Use of Hard-coded Password)로 분류되었으며, 이는 Gate 1의 GitLeaks(CWE-798)와 같은 취약점을 **다른 CWE 관점**에서 탐지한 것입니다.

GitLeaks는 엔트로피 기반으로 “이 문자열이 실제 시크릿처럼 보이는가?”를 판단하고, Bandit은 AST 기반으로 “비밀번호/키 변수에 하드코딩된 값이 할당되었는가?”를 판단합니다.

그리고 결과를 보면 등급이 LOW / Confidence MEDIUM으로 분류되었는데, 그 이유는 Bandit이 “SECRET\_KEY라는 변수명에 문자열이 할당되었지만, 이것이 반드시 프로덕션 시크릿인지는 확인할 수 없다”고 판단하기 때문입니다. 테스트 코드나 더미 값일 수도 있지만, 프로젝트에서는 이것이 실제로 Flask 세션 암호화에 사용되므로 실질적 위험도는 훨씬 높음.

- 탐지 결과 #2 : VULN-03a SQL Injection

## ▼ 9. 부족한 CI에 테스트 자동화 코드 넣기

이번에 한 일

```

(base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % python -m pytest tests/ -v
tests/test_auth.py: 264 warnings
tests/test_health.py: 88 warnings
tests/test_search.py: 88 warnings
tests/test_studylog.py: 88 warnings
/opt/anaconda3/lib/python3.12/site-packages/werkzeug/routing/rules.py:748: DeprecationWarning: ast.Str is deprecated and will be removed in Python 3.14; use ast.Constant instead
 _convert(elem) if is_dynamic else ast.Str(s=elem)
tests/test_auth.py: 264 warnings
tests/test_health.py: 88 warnings
tests/test_search.py: 88 warnings
tests/test_studylog.py: 88 warnings
/opt/anaconda3/lib/python3.12/site-packages/ast.py:507: DeprecationWarning: Attribute s is deprecated and will be removed in Python 3.14; use value instead
 return Constant(*args, **kwargs)
tests/test_auth.py: 372 warnings
tests/test_health.py: 124 warnings
tests/test_search.py: 124 warnings
tests/test_studylog.py: 124 warnings
/opt/anaconda3/lib/python3.12/site-packages/werkzeug/routing/rules.py:755: DeprecationWarning: ast.Str is deprecated and will be removed in Python 3.14; use ast.Constant instead
 if isinstance(p, ast.Str) and isinstance(ret[-1], ast.Str):
tests/test_auth.py: 312 warnings
tests/test_health.py: 104 warnings
tests/test_search.py: 104 warnings
tests/test_studylog.py: 104 warnings
/opt/anaconda3/lib/python3.12/site-packages/werkzeug/routing/rules.py:756: DeprecationWarning: Attribute s is deprecated and will be removed in Python 3.14; use value instead
 ret[-1] = ast.Str(ret[-1].s + p.s)
tests/test_auth.py: 156 warnings
tests/test_health.py: 52 warnings
tests/test_search.py: 52 warnings
tests/test_studylog.py: 52 warnings
/opt/anaconda3/lib/python3.12/site-packages/werkzeug/routing/rules.py:756: DeprecationWarning: ast.Str is deprecated and will be removed in Python 3.14; use ast.Constant instead
 ret[-1] = ast.Str(ret[-1].s + p.s)
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
6 passed, 2917 warnings in 0.46s

```

CI(GitHub Actions)에 테스트 자동화 파이프라인 통합 이전 단계에서 로컬 환경을 통해 **pytest** 가 100% 정상 동작함을 확인했다.

하지만 진정한 의미의 DevSecOps 자동화를 달성하려면 개발자가 수동으로 테스트하는 것이 아니라, 코드가 푸시될 때마다 서버에서 자동으로 테스트가 실행되어야 한다.

이를 위해 GitHub Actions 워크플로우( `.github/workflows/devsecops.yml` )에 **unit-test** 라는 새로운 Job을 만들었다. 이 Job은 가상 환경에서 Python 3.12를 세팅하고, `requirements.txt` 의 패키지들을 설치한 뒤 **pytest** 를 실행하여 애플리케이션

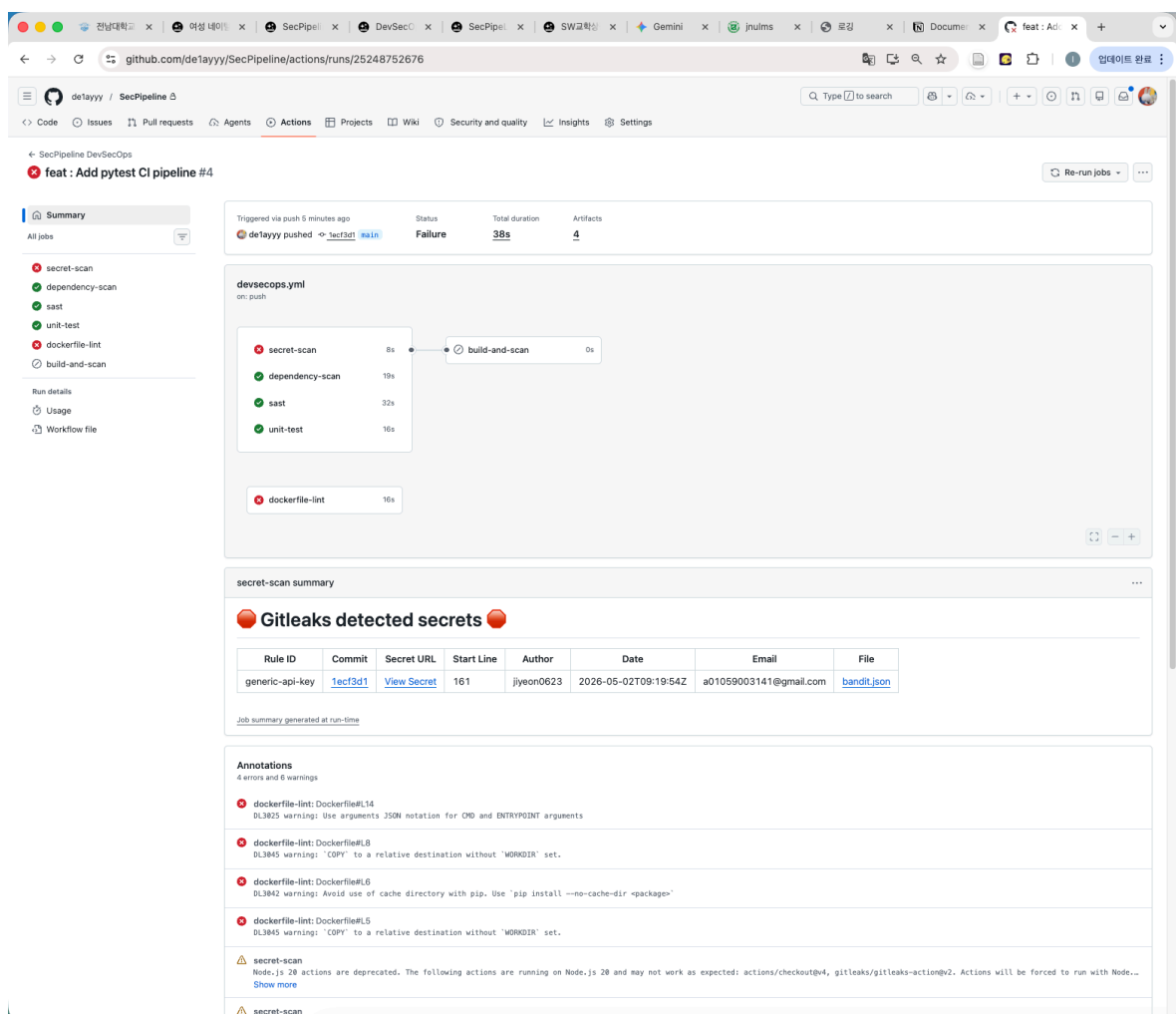


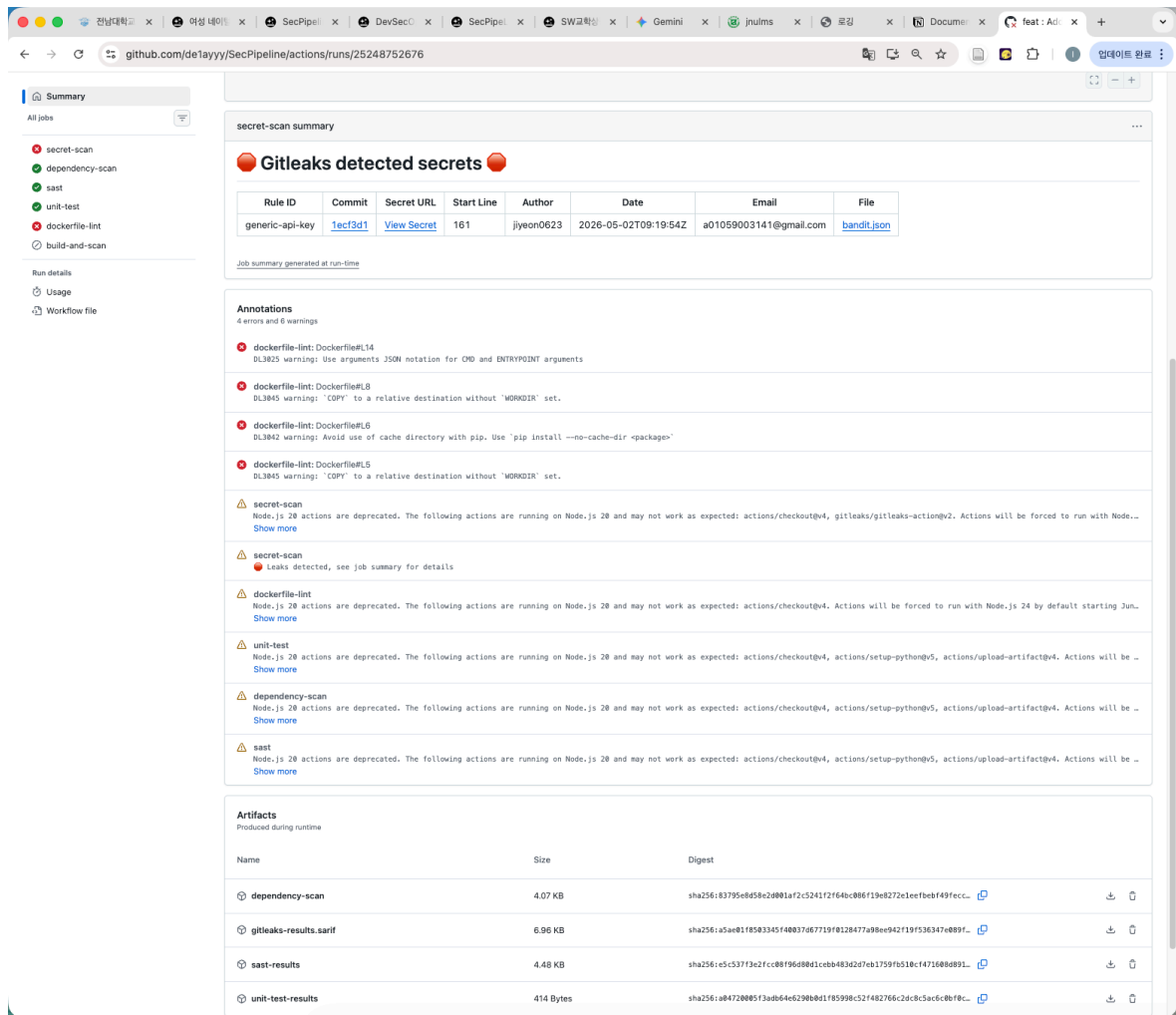
션의 정상 동작을 검증한다. 만약 이 단위 테스트가 하나라도 실패하면 다음 단계인 빌드( `build-and-scan` ) 작업으로 넘어가지 못하도록 방어하는 품질 게이트 역할을 수행하도록 의존성( `needs` )을 설정했다. 테스트 결과는 `test-results.xml` 형태의 Artifact로 업로드되도록 구성했다.

한 번 확인해보면,

```
(base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git add .
git
(base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git commit -m "feat : Add pytests CI pipeline"
[main 1ecf3d1] feat : Add pytest CI pipeline
20 files changed, 1631 insertions(+), 2 deletions(-)
create mode 100644 ".8324_352\260\234\353\260\234.txt"
create mode 100644 "SecPipeline_352\260\234\353\260\234_353\260\234_352\260\200\354\235\264\353\223\234_353\254\270\354\204\234_2.txt"
create mode 100644 analysis_report.md
create mode 100644 bandit.json
create mode 100644 results.sarif
create mode 100644 semgrep.json
create mode 100644 tests/confest.py
create mode 100644 tests/test_health.py
create mode 100644 tests/test_search.py
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\0324_352\260\234\353\260\234.pdf"
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\5_355\205\214\354\212\244\355\212\270\354\236\220\353\217\231\355\231\224_48-66_2).pdf"
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\6_CI\354\231\200\GitHub Actions (33-56) (1).pdf"
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\7_1_353\260\260\355\217\254\354\231\200\Render (14-32).pdf"
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\7_2_353\217\204\354\273\244\354\231\200_353\260\260\355\217\254_13-29).pdf"
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\9_353\241\234\352\271\205_24-35_1).pdf"
create mode 100644 "\354\260\270\352\263\240_354\236\220\353\243\214\SecPipeline_352\260\234\353\260\234_352\260\200\354\235\264\353\223\234_353\254\270\354\204\234_2.pdf"
f"
```

GitHub Actions 돌아간걸 확인해보면,





- 의도적 취약점 탐지 확인 (Red Run) CI 파이프라인을 구축한 후, 사전에 기획하여 삽입해 둔 5가지 의도적 보안 취약점 (VULN-01~05)이 파이프라인의 각 보안 게이트에서 정상적으로 탐지되고 빌드를 차단하는지 확인하기 위해 'Red Run'을 수행했습니다.
- GitHub Actions 실행 결과, secret-scan Job에서는 하드코딩된 **SECRET\_KEY** (VULN-01)를 탐지하여 실패(Failure)를 반환했고, **dockerfile-lint** Job에서는 권한 및 캐시 최적화가 누락된 Dockerfile(VULN-04, 05)의 안티패턴을 정확히 탐지해 내었다. 이로써 취약한 코드가 Main 브랜치나 운영 환경으로 배포되는 것을 CI 단계에서 원천 차단하는 DevSecOps 파이프라인이 성공적으로 동작함을 증명하였다.
- 자세하게 **dependency-scan**, **gitleaks-results.sarif**, **sast-results**, **unit-test-results** 결과를 살펴보면,
  - test-results.xml을 살펴보면 총 6개의 단위 테스트가 100% 통과하여 애플리케이션의 핵심 비즈니스 로직에는 결함이 없음을 확인했습니다.
  - SCA 스캔 결과를 보면, flask, jinja2 등의 구버전 패키지에서 도합 16건의 High/Critical 수준 CVE가 탐지되었습니다. 이를 통해, 기능이 완벽하게 동작하더라도 코드의 보안 수준이 미달되면 파이프라인 단계에서 배포를 철저히 차단하는 DevSecOps의 방어 체계가 기획한 대로 완벽하게 작동함을 입증했습니다.
  - 직접 다운로드 받은 것(GitHub Actions에서)

```
{
 "dependencies": [
 {
 "name": "flask",
 "version": "2.2.5",
 "vulns": [
 {
 "id": "CVE-2026-27205",
 "fix_versions": ["3.1.3"],
 "aliases": ["GHSA-68rp-wp8r-4726"],
 "description": "When the `session` object is accessed, Flask should set the `Vary: Cookie` header. This instructs caches not to cache the response, as it may contain information specific to a logged in user. This is handled in most cases, but some forms of access such as the Python `in` operator were overlooked. The severity depends on the application's use of the session, and the c"
 }
]
 }
]
}
```

ache's behavior regarding cookies. The risk depends on all these conditions being met. 1. The application must be hosted behind a caching proxy that does not ignore responses with cookies. 2. The application does not set a `Cache-Control` header to indicate that a page is private or should not be cached. 3. The application accesses the session in a way that does not access the values, only the keys, and does not mutate the session."}}], {"name": "jinja2", "version": "3.1.2", "vulns": [{"id": "CVE-2024-22195", "fix\_versions": ["3.1.3"], "aliases": ["GHSA-h5c8-rqwp-cp95"], "description": "The `xmlattr` filter in affected versions of Jinja accepts keys containing spaces. XML/HTML attributes cannot contain spaces, as each would then be interpreted as a separate attribute. If an application accepts keys (as opposed to only values) as user input, and renders these in pages that other users see as well, an attacker could use this to inject other attributes and perform XSS. Note that accepting keys as user input is not common or a particularly intended use case of the `xmlattr` filter, and an application doing so should already be verifying what keys are provided regardless of this fix."}, {"id": "CVE-2024-34064", "fix\_versions": ["3.1.4"], "aliases": ["GHSA-h75v-3vvj-5mfj"], "description": "The `xmlattr` filter in affected versions of Jinja accepts keys containing non-attribute characters. XML/HTML attributes cannot contain spaces, `/`, `>`, or `=`, as each would then be interpreted as starting a separate attribute. If an application accepts keys (as opposed to only values) as user input, and renders these in pages that other users see as well, an attacker could use this to inject other attributes and perform XSS. The fix for the previous GHSA-h5c8-rqwp-cp95 CVE-2024-22195 only addressed spaces but not other characters. Accepting keys as user input is now explicitly considered an unintended use case of the `xmlattr` filter, and code that does so without otherwise validating the input should be flagged as insecure, regardless of Jinja version. Accepting `\_values\_` as user input continues to be safe."}, {"id": "CVE-2024-56326", "fix\_versions": ["3.1.5"], "aliases": ["GHSA-q2x7-8rv6-6q7h"], "description": "An oversight in how the Jinja sandboxed environment detects calls to `str.format` allows an attacker that controls the content of a template to execute arbitrary Python code. To exploit the vulnerability, an attacker needs to control the content of a template. Whether that is the case depends on the type of application using Jinja. This vulnerability impacts users of applications which execute untrusted templates. Jinja's sandbox does catch calls to `str.format` and ensures they don't escape the sandbox. However, it's possible to store a reference to a malicious string's `format` method, then pass that to a filter that calls it. No such filters are built-in to Jinja, but could be present through custom filters in an application. After the fix, such indirect calls are also handled by the sandbox."}, {"id": "CVE-2024-56201", "fix\_versions": ["3.1.5"], "aliases": ["GHSA-gmj6-6f8f-6699"], "description": "A bug in the Jinja compiler allows an attacker that controls both the content and file name of a template to execute arbitrary Python code, regardless of if Jinja's sandbox is used. To exploit the vulnerability, an attacker needs to control both the filename and the contents of a template. Whether that is the case depends on the type of application using Jinja. This vulnerability impacts users of applications which execute untrusted templates where the template author can also choose the template filename."}, {"id": "CVE-2025-27516", "fix\_versions": ["3.1.6"], "aliases": ["GHSA-cpwx-vrp4-4pq7"], "description": "An oversight in how the Jinja sandboxed environment interacts with the `|attr` filter allows an attacker that controls the content of a template to execute arbitrary Python code. To exploit the vulnerability, an attacker needs to control the content of a template. Whether that is the case depends on the type of application using Jinja. This vulnerability impacts users of applications which execute untrusted templates. Jinja's sandbox does catch calls to `str.format`

` and ensures they don't escape the sandbox. However, it's possible to use the `|attr` filter to get a reference to a string's plain format method, bypassing the sandbox. After the fix, the `|attr` filter no longer bypasses the environment's attribute lookup."}}], {"name": "werkzeug", "version": "2.2.3", "vulns": [{"id": "PYSEC-2023-221", "fix\_versions": ["2.3.8", "3.0.1"], "aliases": ["CVE-2023-46136", "GHSA-hrfv-mqp8-q5rw"], "description": "Werkzeug multipart data parser needs to find a boundary that may be between consecutive chunks. That's why parsing is based on looking for newline characters. Unfortunately, code looking for partial boundary in the buffer is written inefficiently, so if we upload a file that starts with CR or LF and then is followed by megabytes of data without these characters: all of these bytes are appended chunk by chunk into internal bytearray and lookup for boundary is performed on growing buffer. This allows an attacker to cause a denial of service by sending crafted multipart data to an endpoint that will parse it. The amount of CPU time required can block worker processes from handling legitimate requests. The amount of RAM required can trigger an out of memory kill of the process. If many concurrent requests are sent continuously, this can exhaust or kill all available workers."}], {"id": "PYSEC-2023-221", "fix\_versions": ["2.3.8", "3.0.1"], "aliases": ["CVE-2023-46136", "GHSA-hrfv-mqp8-q5rw"], "description": "Werkzeug is a comprehensive WSGI web application library. If an upload of a file that starts with CR or LF and then is followed by megabytes of data without these characters: all of these bytes are appended chunk by chunk into internal bytearray and lookup for boundary is performed on growing buffer. This allows an attacker to cause a denial of service by sending crafted multipart data to an endpoint that will parse it. The amount of CPU time required can block worker processes from handling legitimate requests. This vulnerability has been patched in version 3.0.1."}], {"id": "CVE-2024-34069", "fix\_version": ["3.0.3"], "aliases": ["GHSA-2g68-c3qc-8985"], "description": "The debugger in affected versions of Werkzeug can allow an attacker to execute code on a developer's machine under some circumstances. This requires the attacker to get the developer to interact with a domain and subdomain they control, and enter the debugger PIN, but if they are successful it allows access to the debugger even if it is only running on localhost. This also requires the attacker to guess a URL in the developer's application that will trigger the debugger."}], {"id": "CVE-2024-49766", "fix\_versions": ["3.0.6"], "aliases": ["GHSA-f9vj-2wh5-fj8j"], "description": "On Python < 3.11 on Windows, `os.path.isabs()` does not catch UNC paths like `//server/share`. Werkzeug's `safe\_join()` relies on this check, and so can produce a path that is not safe, potentially allowing unintended access to data. Applications using Python >= 3.11, or not using Windows, are not vulnerable."}], {"id": "CVE-2024-49767", "fix\_version": ["3.0.6"], "aliases": ["GHSA-q34m-jh98-gwm2"], "description": "Applications using Werkzeug to parse `multipart/form-data` requests are vulnerable to resource exhaustion. A specially crafted form body can bypass the `Request.max\_form\_memory\_size` setting. The `Request.max\_content\_length` setting, as well as resource limits provided by deployment software and platforms, are also available to limit the resources used during a request. This vulnerability does not affect those settings. All three types of limits should be considered and set appropriately when deploying an application."}], {"id": "CVE-2025-66221", "fix\_versions": ["3.1.4"], "aliases": ["GHSA-hgf8-39gv-g3f2"], "description": "Werkzeug's `safe\_join` function allows path segments with Windows device names. On Windows, there are special device names such as `CON`, `AUX`, etc that are implicitly present and readable in every directory. `send\_from\_directory` uses `safe\_join` to safely serve files at user-specified paths under a directory. If the application is running on Windows, and the requested path ends with a special device name, the file will be opened successfully, but read"

```

ding will hang indefinitely."}, {"id": "CVE-2026-21860", "fix_versions": ["3.1.5"], "aliases": ["GHSA-87hc-h4r5-73f7"], "description": "Werkzeug's `safe_join` function allows path segments with Windows device names that have file extensions or trailing spaces. On Windows, there are special device names such as `CON`, `AUX`, etc that are implicitly present and readable in every directory. Windows still accepts them with any file extension, such as `CON.txt`, or trailing spaces such as `CON `. This was previously reported as https://github.com/pallets/werkzeug/security/advisories/GHSA-hgf8-39gv-g3f2, but the fix failed to account for compound extensions such as `CON.txt.html` or trailing spaces. It also missed some additional special names. `send_from_directory` uses `safe_join` to safely serve files at user-specified paths under a directory. If the application is running on Windows, and the requested path ends with a special device name, the file will be opened successfully, but reading will hang indefinitely."}, {"id": "CVE-2026-27199", "fix_versions": ["3.1.6"], "aliases": ["GHSA-29vq-49wr-vm6x"], "description": "Werkzeug's `safe_join` function allows Windows device names as filenames if when preceded by other path segments. This was previously reported as https://github.com/pallets/werkzeug/security/advisories/GHSA-hgf8-39gv-g3f2, but the added filtering failed to account for the fact that `safe_join` accepts paths with multiple segments, such as `example/NUL`. `send_from_directory` uses `safe_join` to safely serve files at user-specified paths under a directory. If the application is running on Windows, and the requested path ends with a special device name, the file will be opened successfully, but reading will hang indefinitely."}], [{"name": "unicorn", "version": "21.2.0", "vulns": [{"id": "CVE-2024-1135", "fix_versions": ["22.0.0"], "aliases": ["GHSA-w3h3-4rj7-4ph4"], "description": "Gunicorn fails to properly validate Transfer-Encoding headers, leading to HTTP Request Smuggling (HRS) vulnerabilities. By crafting requests with conflicting Transfer-Encoding headers, attackers can bypass security restrictions and access restricted endpoints. This issue is due to Gunicorn's handling of Transfer-Encoding headers, where it incorrectly processes requests with multiple, conflicting Transfer-Encoding headers, treating them as chunked regardless of the final encoding specified. This vulnerability has been shown to allow access to endpoints restricted by gunicorn. This issue has been addressed in version 22.0.0. To be affected users must have a network path which does not filter out invalid requests. These users are advised to block access to restricted endpoints via a firewall or other mechanism if they are unable to update."}, {"id": "CVE-2024-6827", "fix_versions": ["22.0.0"], "aliases": ["GHSA-hc5x-x2vx-497g"], "description": "Gunicorn version 21.2.0 does not properly validate the value of the 'Transfer-Encoding' header as specified in the RFC standards, which leads to the default fallback method of 'Content-Length', making it vulnerable to TE.CL request smuggling. This vulnerability can lead to cache poisoning, data exposure, session manipulation, SSRF, XSS, DoS, data integrity compromise, security bypass, information leakage, and business logic abuse."}], [{"name": "pytest", "version": "9.0.3", "vulns": []}, {"name": "pluggy", "version": "1.6.0", "vulns": []}, {"name": "pytest-cov", "version": "7.1.0", "vulns": []}, {"name": "httpx", "version": "0.28.1", "vulns": []}, {"name": "httpcore", "version": "1.0.9", "vulns": []}, {"name": "click", "version": "8.3.3", "vulns": []}, {"name": "coverage", "version": "7.13.5", "vulns": []}, {"name": "h11", "version": "0.16.0", "vulns": []}, {"name": "iniconfig", "version": "2.3.0", "vulns": []}, {"name": "itsdangerous", "version": "2.2.0", "vulns": []}, {"name": "markupsafe", "version": "3.0.3", "vulns": []}, {"name": "pygments", "version": "2.20.0", "vulns": []}, {"name": "anyio", "version": "4.13.0", "vulns": []}, {"name": "idna", "version": "3.13",

```

```
"vulns": [], {"name": "typing-extensions", "version": "4.15.0", "vulns": []}, {"name": "certifi", "version": "2026.4.22", "vulns": []}, {"fixes": []}
```

```
<?xml version="1.0" encoding="utf-8"?><testsuites name="pytest tests"><testsuite name="pytest" errors="0" failures="0" skipped="0" tests="6" time="0.632" timestamp="2026-05-02T09:20:25.843712+00:00" hostname="runner-vmeorf1"><testcase classname="tests.test_auth" name="test_login_page" time="0.033" /><testcase classname="tests.test_auth" name="test_register_page" time="0.011" /><testcase classname="tests.test_auth" name="test_auth_flow" time="0.173" /><testcase classname="tests.test_health" name="test_health" time="0.013" /><testcase classname="tests.test_search" name="test_search" time="0.170" /><testcase classname="tests.test_studylog" name="test_studylog_crud" time="0.178" /></testsuite></testsuites>
```

## ▼ 10. 발견된 보안 취약점 수정 및 시큐어 코딩 적용하기

의도적 실패로 Red Run을 통해 탐지된 4건의 보안 취약점에 대해 다음과 같이 시큐어 코딩을 적용하여 리팩토링을 완료해 Green Run으로 수정하고자 합니다. (이는 의도적으로 개발 상황에서 흔히 일어날 수 있는 보안 취약점이 있는데도 배포하는 것을 막고자 DevSecOps로 방지 하기 위해 개발한 내용들입니다.)

1. VULN-01 (시크릿 하드코딩) : `app/config.py` 의 평문 시크릿을 `os.environ.get()` 환경변수 호출 방식으로 변경하여 코드베이스에서 시크릿을 완전히 분리했습니다.

a. 보안 취약점 : 하드코딩된 시크릿이 GitHub 저장소에 푸시되면, 소스 코드 열람 권한이 있는 누구든 세션 데이터를 변조하거나 복호화할 수 있기에 치명적인 피해 유발할 수 있기에 필수로!! 검증해야 합니다.

b. 구체적인 조치 내역 :

```
- 기존: `SECRET_KEY = "fl4sk_s3cr3t..."`
- 변경: `SECRET_KEY = os.environ.get("SECRET_KEY", "dev-only-fallback")`
```

실제 배포 환경에서는 관리자 대시보드에서 환경변수를 안전하게 주입할 예정입니다.

2. VULN-02 (취약한 의존성) : 16개의 CVE가 발견된 flask, werkzeug 등의 구버전 패키지를 최신 버전으로 상향 업데이트하여 공급망 공격 위험을 차단했습니다.

a. 보안 취약점 :

i. `requirements.txt`에 명시된 `flask==2.2.5`, `jinj2==3.1.2`, `werkzeug==2.2.3` 패키지들이 알려진 취약점(CVE)을 다수 내포한 구버전으로 고정되어 있었습니다.

ii. CI 과정의 Gate 2 (pip-audit) 스캔 결과, 총 16건의 취약점(예: CVE-2024-56326 등)이 탐지되며 빌드가 실패했습니다.

b. 공격자가 이 구버전 패키지들의 알려진 취약점을 이용해 임의 코드 실행, 서버 자원 고갈 등의 공격을 수행하여 서버 전체의 권한을 탈취할 수 있는 매우 위험한 상태

c. 구체적인 수정 내역 : 기존 `requirements.txt`의 버전을 최신 안정화 버전으로 명시적으로 상향 업데이트했습니다.

```
- `flask>=3.1.0`
- `jinj2>=3.1.4`
- `werkzeug>=3.1.0`
```

3. VULN-03a (SQL Injection) : `search.py` 에서 f-string을 통해 문자열을 조합하던 SQL 쿼리를 파라미터 바인딩 방식으로 교체하여 인젝션 공격을 완전 차단했습니다.

a. 보안 취약점

i. `app/routes/search.py` 모듈에서 사용자가 입력한 검색어(`keyword`)를 SQL 쿼리 문자열에 f-string으로 직접 연결(Concatenation)하고 있었습니다.



ii. CI 과정의 Gate 3 (Semgrep / Bandit)에서 'B608 hardcoded\_sql\_expressions' 룰과 'formatted-sql-query' 룰에 의해 발견되었습니다.

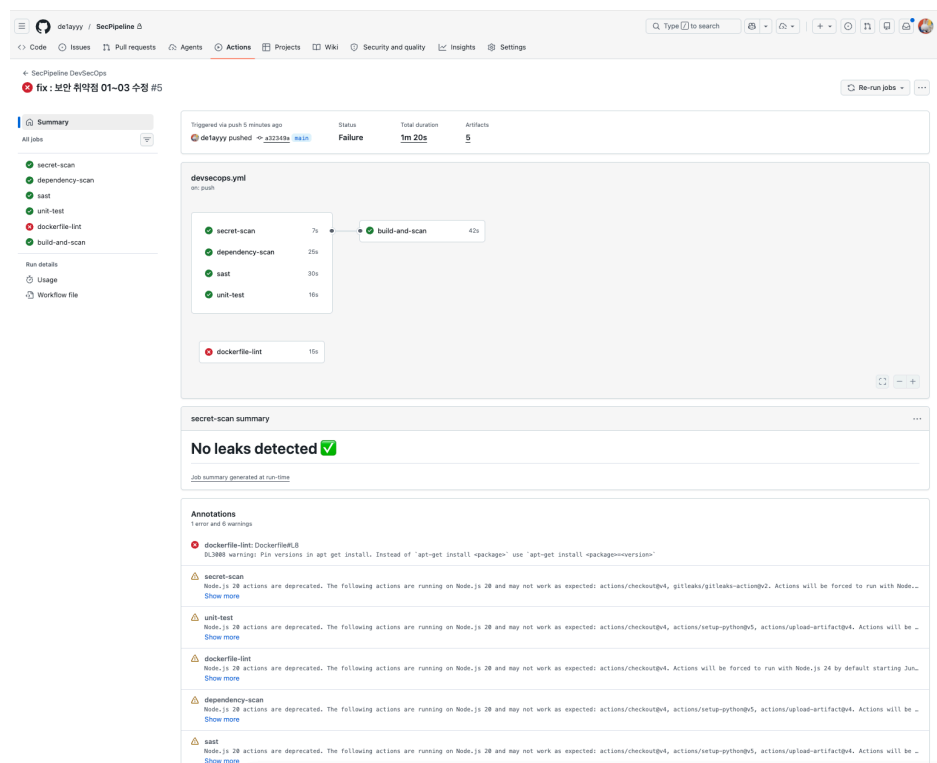
b. 구체적인 수정 내역

a. 데이터베이스 쿼리를 실행할 때, 문자열 포매팅 방식을 제거하고 DB 드라이버에서 제공하는 파라미터 바인딩 방식 인 ?로 교체하였습니다.

4. VULN-03b (XSS/SSTI) : `dashboard.py` 에서 템플릿에 직접 렌더링되던 사용자 입력값을 Jinja2 템플릿 엔진의 `{{ }}` 변수 렌더링 방식으로 리팩토링

```
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro SecPipeline % git add .
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro SecPipeline % git commit -m "fix : 보안 취약점 01~03 수정"
[main a32349a] fix : 보안 취약점 01~03 수정
19 files changed, 1648 insertions(+), 19 deletions(-)
create mode 100644 docs/incident-reports/fix-01-secret.md
create mode 100644 docs/incident-reports/fix-02-dependency.md
create mode 100644 docs/incident-reports/fix-03a-sqli.md
create mode 100644 docs/incident-reports/fix-03b-xss-ssti.md
create mode 100644 docs/incident-reports/fix-04-05-docker.md
create mode 100644 docs/scan-results/red-run/README.md
create mode 100644 docs/scan-results/red-run/dep-scan 3.json
create mode 100644 docs/scan-results/red-run/work 3/.DS_Store
create mode 100644 docs/scan-results/red-run/work 3/SecPipeline/.DS_Store
create mode 100644 docs/scan-results/red-run/work 3/SecPipeline/SecPipeline/results.sarif
create mode 100644 docs/scan-results/red-run/work 3/sast-results (2)/bandit.json
create mode 100644 docs/scan-results/red-run/work 3/sast-results (2)/semgrep.json
create mode 100644 docs/scan-results/red-run/work 3/test-results 2.xml
(base) yoonjiyeon@yoonjiyeon-ui-MacBookPro SecPipeline % git push origin main
Enumerating objects: 40, done.
Counting objects: 100% (40/40), done.
Delta compression using up to 11 threads
Compressing objects: 100% (28/28), done.
Writing objects: 100% (30/30), 25.04 KiB | 8.34 MiB/s, done.
Total 30 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To https://github.com/dejayyy/SecPipeline.git
1ecf3d1..a32349a main -> main
```

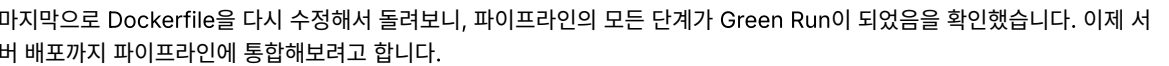
수정 후에 git에 올리고 actions 확인해보면,



의도적 취약점 VULN-01~03의 코드를 안전하게 리팩토링한 후 GitHub에 Push하여 CI 파이프라인의 변화를 확인했습니다.

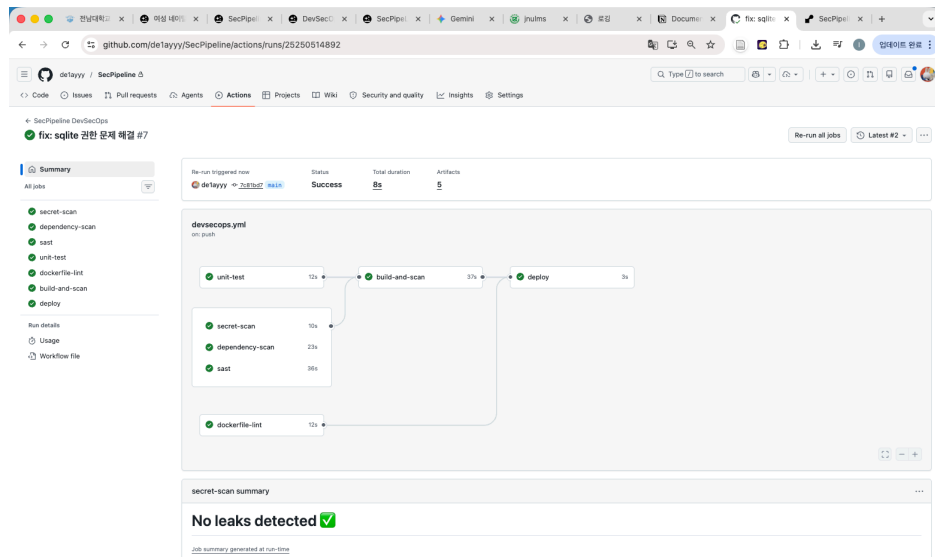
- **secret-scan 통과**: `SECRET_KEY` 를 환경변수로 분리하여, Gitleaks에서 하드코딩된 시크릿이 더 이상 탐지되지 않음을 확인했습니다. (화면상 "No leaks detected")
- **dependency-scan 통과**: `flask` , `werkzeug` 등 의존성을 최신 버전으로 패치하여 16건의 치명적 CVE 탐지가 모두 사라지고 정상 통과했습니다.

- 단, 아직 **Dockerfile**에 대한 하드닝(VULN-04, 05)을 로컬에만 반영하고 Push하기 전이므로 **dockerfile-lint** 게이트에서는 여전히 실패(Failure)하고 있음을 의도적으로 확인했습니다. 이를 통해 파이프라인이 단 하나의 결함이라도 남아있으면 끝까지 배포를 막아주는 간간한 문지기 역할을 수행하고 있음을 확인했습니다.



앞선 단계에서 모든 보안 취약점 게이트를 통과(Green Run)하는 안전한 컨테이너 이미지를 확보했습니다. 이제 CI의 마지막 단계로 외부에서 접근 가능한 실제 서비스로 배포하는 파이프라인을 완성하고자 합니다.

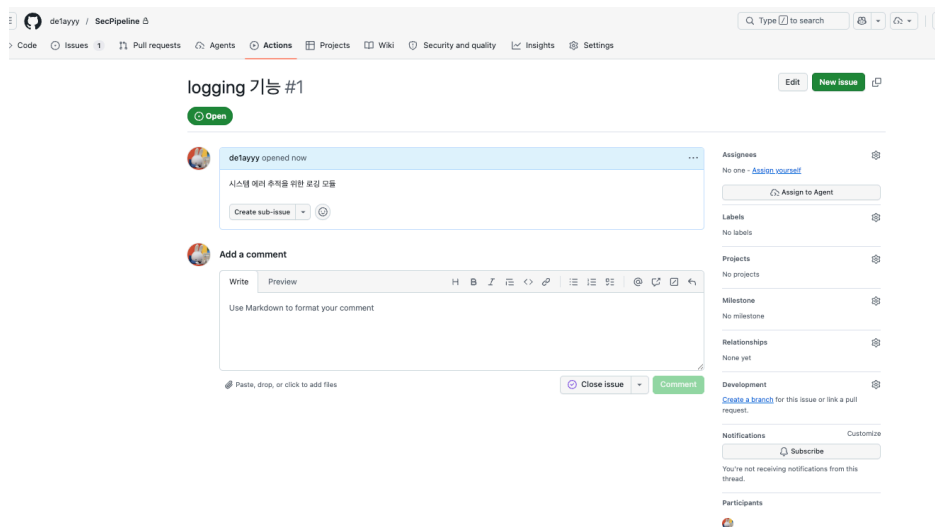




지금까지 구축한 DevSecOps 파이프라인을 확인해보면, GitHub의 main 브랜치에 push하는 즉시 다음의 과정을 자동으로 수행합니다.

1. Unit Test : pytest 기반 6개의 로직 정상 동작하는지? 검증
2. SCA & Secret Scan : Gitleaks 및 pip-audit을 병렬 실행하여 시크릿 노출 및 의존성 CVE 취약점 차단
3. SAST : Semgrep 및 Bandit을 병렬 실행하며 SQLi, XSS 등 소스코드 내부의 취약점 안티패턴 탐지
4. **Container Scan & Lint**: hadolint로 Dockerfile의 권한 및 최적화 상태를 검증하고, Trivy로 빌드된 이미지의 OS 레벨 취약점을 검사
5. **Deploy**: 위 모든 품질/보안 게이트(Quality Gate)를 단 하나의 에러 없이 통과(Green Run)했을 경우에만, Render Webhook을 호출하여 클라우드 환경에 컨테이너를 무중단 배포한다.

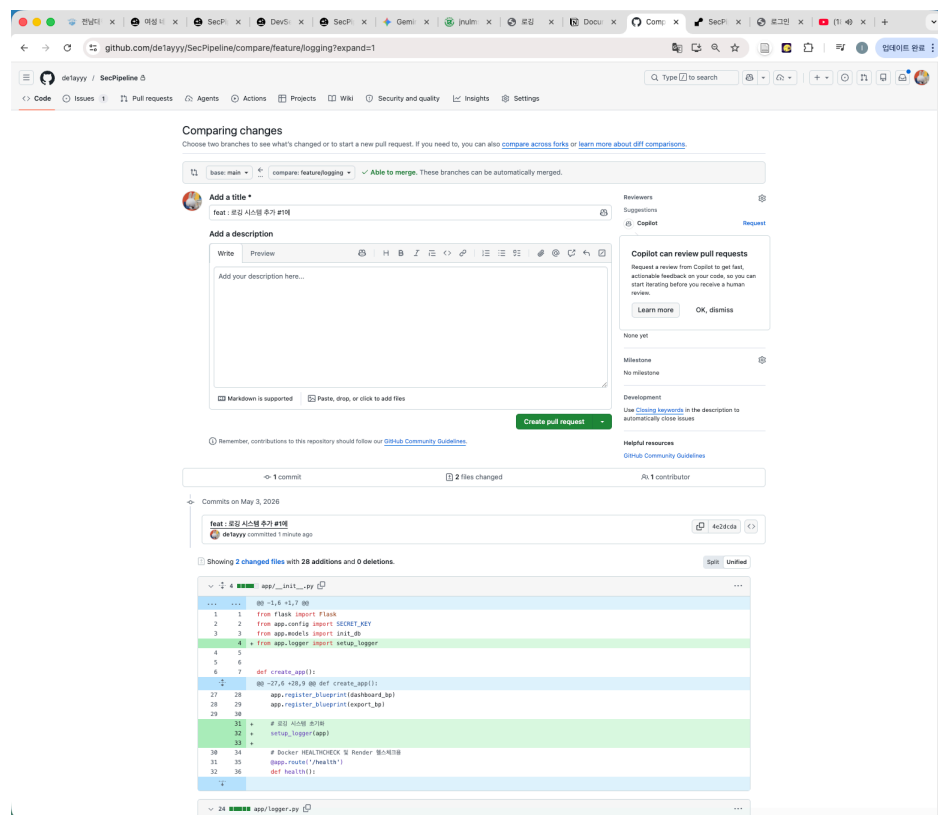
## ▼ 12. 시스템 로깅 모듈 및 PR



```

• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git checkout -b feature/logging
Switched to a new branch 'feature/logging'
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git add .
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git commit -m "feat : 로깅 시스템 추가 #10"
[feature/logging 4e2dcda] feat : 로깅 시스템 추가 #10
2 files changed, 28 insertions(+)
create mode 100644 app/logger.py
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git push -u origin feature/logging
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 11 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.12 KiB | 1.12 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'feature/logging' on GitHub by visiting:
remote: https://github.com/delayyy/SecPipeline/pull/new/feature/logging
remote:
To https://github.com/delayyy/SecPipeline.git
 * [new branch] feature/logging -> feature/logging
branch 'feature/logging' set up to track 'origin/feature/logging'.

```



Pull Request를 만들면,

de'ayyy / SecPipeline

Code Issues 1 Pull requests 1 Agents Actions Projects Wiki Security and quality Insights Settings

## feat : 로깅 시스템 추가 #1에 #2

Open de'ayyy wants to merge 1 commit into main from feature/logging

Conversation 0 Commits 1 Checks 0 Files changed 2 +28

de'ayyy commented now

No description provided.

Mention @copilot in a comment to make changes to this pull request.

Feat : 로깅 시스템 추가 #1에 #2

Some checks haven't completed yet  
2 in progress, 3 queued checks

3 pending checks

- SecPipeline DevSecOps / dockerfile-lint (pull\_request) Queued — Waiting to run this check...
- SecPipeline DevSecOps / sast (pull\_request) Queued — Waiting to run this check...
- SecPipeline DevSecOps / secret-scan (pull\_request) Queued — Waiting to run this check...

2 in progress checks

- SecPipeline DevSecOps / dependency-scan (pull\_request) Started now — This check has started...
- SecPipeline DevSecOps / unit-test (pull\_request) Started now — This check has started...

Checking for the ability to merge automatically...  
Hang in there while we check the branch's status.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

Write Preview

Add your comment here...

Markdown is supported Paste, drop, or click to add files

Close pull request Comment

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

ProTip! Add `.patch` or `.diff` to the end of URLs for Git's plaintext views.

Reviewers Suggestions

@ Copilot [Request](#)

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications Customize

Unsubscribe

You're receiving notifications because you authored the thread.

1 participant

Lock conversation

Ci 파이프라인이 새롭게 추가한 로깅 코드가 main에 들어가도 안전한지 검사합니다.

## feat : 로깅 시스템 추가 #1에 #2

Open de1ayyy wants to merge 1 commit into `main` from `feature/logging`

Conversation 0 Commits 1 Checks 7 Files changed 2



de1ayyy commented 2 minutes ago

No description provided.



Mention @copilot in a comment to make changes to this pull request.

feat : 로깅 시스템 추가 #1에

4e2dcda



Some checks were not successful  
1 failing, 2 skipped, 4 successful checks

2 skipped checks

SecPipeline DevSecOps / build-and-scan (pull\_request) Skipped 1 minute ago

SecPipeline DevSecOps / deploy (pull\_request) Skipped 1 minute ago

4 successful checks

SecPipeline DevSecOps / dependency-scan (pull\_request) Successful in 23s

SecPipeline DevSecOps / dockerfile-lint (pull\_request) Successful in 12s

SecPipeline DevSecOps / sast (pull\_request) Successful in 28s

SecPipeline DevSecOps / unit-test (pull\_request) Successful in 12s

No conflicts with base branch  
Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)



Add a comment

merge해도 괜찮으니, Merge pull request를 눌러 안전하게 PR하면 됩니다!

### ▼ 13. 운영 환경 동적 보안 스캔(DAST) 파이프라인에 합치기

정적 애플리케이션 보안 테스트(SAST)와 컨테이너 스캔만으로는 런타임 환경에서 일어나는 취약점을 완벽하게 잡아낼 수 없기에, 배포가 완료된 직후 DAST 단계를 구축했습니다.

OWASP ZAP을 GitHub Actions에 통합하여, Render에 배포된 라이브 서비스 URL을 타겟으로 실제 해킹 시나리오와 유사한 Baseline Scan을 자동으로 수행하도록 설정했습니다. 이로써 본 프로젝트는 Git push → CI 테스트 → SCA/SAST 정적 스캔 → 컨테이너 빌드 및 하드닝 검사 → 배포 → DAST 동적 스캔으로 DevSecOps 파이프라인 아키텍처를 완성했습니다.

```

hint: See the 'Note about fast-forwards' in 'git push --help' for details.
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git checkout main
Switched to branch 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
 (use "git pull" to update your local branch)
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git pull origin main
gFrom https://github.com/delayyy/SecPipeline
 * branch main -> FETCH_HEAD
Updating 7c81bd7..f8c4592
Fast-forward
 app/__init__.py | 4 ++++
 app/logger.py | 24 ++++++++++++++++++++++
 2 files changed, 28 insertions(+)
 create mode 100644 app/logger.py
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git merge feature/logging
Merge made by the 'ort' strategy.
 .github/workflows/devsecops.yml | 16 ++++++++-----
 1 file changed, 15 insertions(+), 1 deletion(-)
• (base) yoonjiyeon@yunjiyeon-ui-MacBookPro SecPipeline % git push origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 11 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 770 bytes | 770.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/delayyy/SecPipeline.git
 f8c4592..66b04cc main -> main

```

정적으로 보안 테스트를 돌리면 실제로 코드가 서버에서 구동될 때(Runtime) 발생하는 취약점이나 서버 설정 누락 등을 탐지하는 데에는 한계가 있습니다. 그래서 파이프라인의 완성도를 더 올리기 위해 배포 단계 직후에 작동하는 DAST를 추가

## ▼ 웹 설명

### 1. 회원가입

SecPipeline
로그인
회원가입

회원가입

사용자명

사용할 아이디

비밀번호

비밀번호

비밀번호 강도: 입력해주세요

가입완료

이미 계정이 있으신가요? [로그인](#)

### 2. 로그인

SecPipeline로그인회원가입

로그인

사용자명아이디를 입력하세요

비밀번호비밀번호를 입력하세요

로그인

계정이 없으신가요? [회원가입](#)

### 3. 첫 대시보드

SecPipelinejiyeon0623님기록과목검색대시보드로그아웃

📖 내 학습 기록

+ 새 기록 작성

아직 작성된 학습 기록이 없습니다.

첫 번째 학습 기록을 남겨보세요!

### 4. 공부 기록 새롭게 작성하기

## 새 기록 작성

제목

오늘은 인공지능파이프라인 공부 했어요 ㅎㅎ

내용

render로 배포해보았습니다!

학습 시간 (시간)

30분

1시간

2시간

3시간

2

과목 선택 (선택사항)

-- 과목을 선택하세요 --

저장하기

취소

## 5. 과목 관리

## 과목 관리

새로운 과목 이름을 입력하세요

추가하기

● 소프트웨어개발론

삭제

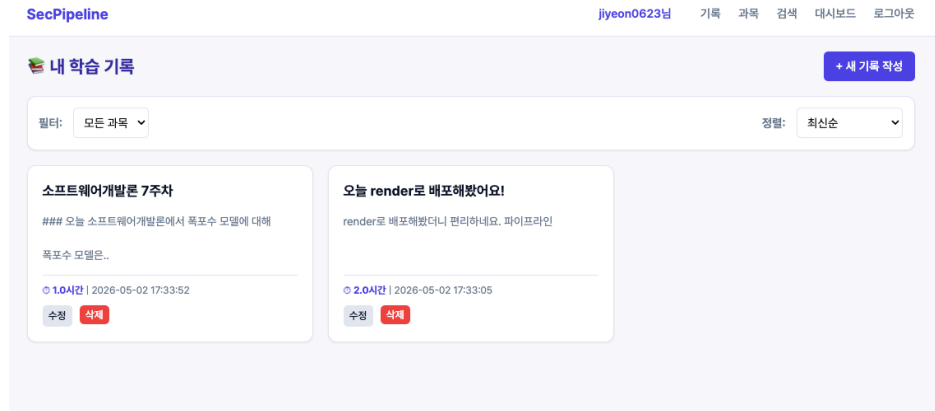
● 알고리즘

삭제

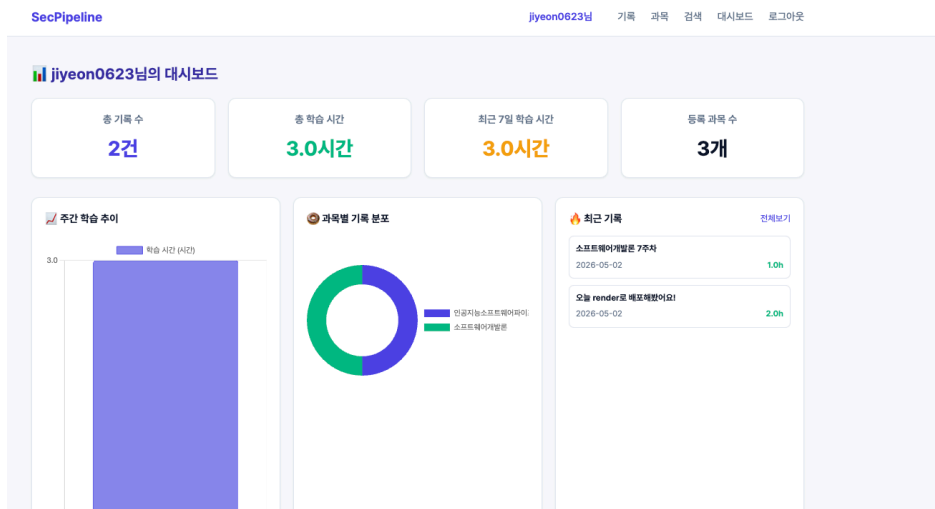
● 인공지능소프트웨어파이프라인개발

삭제

## 6. 기록 관리



## 7. 대시보드



→ 대시보드에서 자신의 공부 기록 통계를 확인할 수 있습니다.